universität wien

MASTERARBEIT | MASTER'S THESIS

Titel | Title

Hierarchical Organization of Behavioral Encodings across Model Organisms

verfasst von | submitted by

Julian Jakob Zimmermann B.Sc. (TUM)

angestrebter akademischer Grad | in partial fulfilment of the requirements for the degree of

Master of Science (MSc)

Wien | Vienna, 2026

Studienkennzahl lt. Studienblatt | Degree programme code as it appears on the student record sheet:	UA 066 875
Studienrichtung lt. Studienblatt | Degree programme as it appears on the student record sheet:	Masterstudium Bioinformatik
Betreut von | Supervisor:	Univ.-Prof. Dr. Manuel Zimmer

Acknowledgements

I want to wholeheartedly thank my mentor Charles Fieseler for his continuous support. His positivity, competence in the field and ability to communicate criticism made it a joy to discuss the topics and field as a whole, brainstorm new ideas and work together towards the finalization of this work. It has been a privilege!
I also want to thank my Professor Manuel Zimmer from the University of Vienna for this opportunity and for building a constructive and professional atmosphere.
Additionally, I want to thank Professor Elly Tanaka, Scientific Director of the Institute of Molecular Biotechnology, for referring me to Professor Zimmer. Without her, I might not have been able to find such a suitable environment for my thesis.

Last but not least, I want to thank all the people I've met during my time in the lab: Eva for mental support, Maria for the neverending stream of chocolate, Liana for answering all my questions, Zehra who to this day still makes jokes about how much I love loss functions, Itamar for our chats about life, Chanuka for the biological discussions and everyone else I've spent fun times with!

The computational results of this work have been achieved using the Life Science Compute Cluster (LiSC) of the University of Vienna.

Kurzfassung

Neuronale Aufnahmen auf großer Skala haben es ermöglicht, Denkmuster in Populationsaktivität über Modellorganismen hinweg zu analysieren. Im Nematoden *Caenorhabditis elegans* wurde gezeigt, dass flächendeckende Signale im Gehirn, welche die zentrale Sequenz von Bewegungskommandos kodieren, neuronalen Trajektorien auf niedrigdimensionalen Mannigfaltigkeiten folgen. Neuere Arbeiten bestätigten, dass solche Mannigfaltigkeiten zusätzlich komplexe Verhaltensmuster organisieren, indem sie sich Schranken-Dynamiken entlang einer Hierarchie von Zeitskalen bedienen, bei der übergeordnete Mannigfaltigkeitssignale untergeordnete Signale modulieren, die detailliertere verhaltensbezogene Informationen kodieren. Obwohl gehirnweite Repräsentationen von anhaltendem Verhalten auch in Fliegen und Mäusen gefunden wurden, ist bislang nur unvollständig verstanden, in welchem Ausmaß diese bei ähnlichen modulatorischen Interaktionsmustern eine Rolle spielen. In dieser Arbeit generalisieren wir die Befunde aus *C. elegans* auf andere Modellorganismen. Dazu stellen wir ein computergestütztes Hierarchical Gating Model (HGM) vor, das moderne Methoden des maschinellen Lernens kombiniert, um neuronale Kodierungen von Verhalten zu lernen, die durch ein parallel gelerntes neuronales Mannigfaltigkeitssignal moduliert werden. Wir trainieren und evaluieren das Modell auf neurobehavioralen Datensätzen von *C. elegans*, *Drosophila melanogaster* und Mäusen, die wir aus öffentlich zugänglichen wissenschaftlichen Quellen recherchiert haben. Bezüglich Fliegen konnte das Modell keine zuverlässige Kodierung solch detaillierten Verhaltens finden, aller Wahrscheinlichkeit nach aufgrund von Unterschieden zwischen den Zeitskalen der Aufnahmen. Unsere Ergebnisse deuten jedoch darauf hin, dass die modellierte Struktur bei Mäusen eine relevante Rolle spielt, was nahelegt, dass Mäuse möglicherweise eine hierarchische Organisation von Verhaltenskodierungen aufweisen, der der in Würmern ähnelt.

Abstract

Large-scale neuronal recordings made it possible to analyse population activity patterns across model organisms. In the nematode *Caenorhabditis elegans*, brain-wide signals encoding the major behavioral action sequence have been shown to follow neuronal trajectories on low-dimensional manifolds. Recent work confirmed such manifolds to additionally organize complex behavior patterns by adopting gating dynamics along a time-scale hierarchy, where higher-level manifold signals modulate lower-level signals that encode more granular behavior-related information. While brain-wide representations of ongoing behavior have also been found in flies and mice, the extent to which they play a role in similar modulatory interaction patterns remains incompletely understood. In this thesis, we generalize the findings in *C. elegans* to other model organisms. To do so, we introduce a computational Hierarchical Gating Model (HGM), that combines modern machine learning techniques to learn behavioral encodings as modulated by a simultaneously learned neuronal manifold. We train and evaluate the model on neuro-behavioral datasets of *C. elegans*, *Drosophila melanogaster* and mice that we researched from publicly available academic sources. For flies, the model was not able to reliably find an encoding of such granular behavior, likely due to differences in recording time-scales. However, our results indicate the modelled structure to play a relevant role in mice, suggesting that mice might show a hierarchical organization of behavioral encodings similar to worms.

Contents

Acknowledgements iii

Kurzfassung v

Abstract vii

Contents ix

1. Introduction 1
- 1.1. Understanding the Brain . . . 1
- 1.2. The Neuronal Manifold . . . 1
- 1.3. Context Dependency . . . 3

2. Preliminaries 7
- 2.1. Biological datasets . . . 7
  - 2.1.1. *C. elegans* data . . . 7
  - 2.1.2. *Drosophila* data . . . 8
  - 2.1.3. Mouse data . . . 9
- 2.2. Computational Tools . . . 10
  - 2.2.1. Dimensionality reduction . . . 10
  - 2.2.2. Linear Regression . . . 13
  - 2.2.3. Multi-Layered Perceptron . . . 15
  - 2.2.4. Optimization . . . 15
  - 2.2.5. Training . . . 16
  - 2.2.6. Significance Assessment . . . 17

3. Related Work 19
- 3.1. State-dependent encoding in *C. elegans* . . . 19
- 3.2. Modelling neuronal activity . . . 19
  - 3.2.1. Strictly Linear Models . . . 21
  - 3.2.2. Nonlinear Models . . . 22
  - 3.2.3. Discrete Modulation Models . . . 23
  - 3.2.4. Continuous Modulation Models . . . 24

4. Results 27
- 4.1. Model Architecture . . . 27
  - 4.1.1. Manifold-Concrete Model . . . 27
  - 4.1.2. Hierarchical Gating Model . . . 29

4.1.3. Residual Subspace Extension . . . 32
4.2. Literature Review . . . 34
4.3. *C. elegans* Analysis . . . 34
4.3.1. Exploratory Analysis . . . 35
4.3.2. Model Results . . . 39
4.4. *Drosophila* Analysis . . . 56
4.4.1. Exploratory Analysis . . . 56
4.4.2. Model Results . . . 62
4.5. Mouse Analysis . . . 65
4.5.1. Exploratory Analysis . . . 65
4.5.2. Model Results . . . 70

5. Methods **81**
5.1. Preprocessing . . . 81
5.1.1. Fly . . . 81
5.1.2. Mice . . . 81
5.2. Toy Data generation . . . 82
5.3. Exploratory Methods . . . 82
5.4. Model Design . . . 83
5.4.1. Residual Regression . . . 83
5.4.2. Linear Autoencoder . . . 84
5.4.3. Nonlinear Gating Extension . . . 85
5.4.4. Loss function . . . 85
5.4.5. Post-processing . . . 86
5.5. Neuron scoring . . . 87
5.5.1. Out-of-sample R2 . . . 87
5.5.2. Shuffle Control . . . 87
5.6. Training . . . 87
5.6.1. Cosine Decay . . . 88
5.6.2. Early Stopping . . . 88
5.6.3. Stratified k-Fold Cross-Validation . . . 88
5.7. Sweeping . . . 89
5.8. JAX . . . 90

6. Discussion **91**

Overview of generative AI tools used **95**

Übersicht verwendeter Hilfsmittel **97**

List of Figures **99**

List of Tables **107**

Bibliography **109**

A. Appendix **115**

1. Introduction

1.1. Understanding the Brain

The brain is the central organ of sensory processing, cognition and motor control. Its highly complex dynamics shape our perception and understanding of and interaction with the world around us. It is therefore no surprise that the central goal of neuroscience is understanding the brain.
Over the last century, biology has often tried to close in on this goal from the structural side, by studying the biochemical fabric that underlies brain dynamics. The neuron cell was defined as its central unit of information and different types of interactions between them were identified, like fast gap junctions ([Ben66]), slower chemical synapses ([KM65]), or neuromodulation over longer timescales ([Mar12]). In this context, structural wiring maps of neural connections, termed connectomes, became of interest. Even though it is a difficult technological feat to map brain-wide connectomes due to the sheer amount of neurons and their interactions, the connectomes of a few model organisms have been fully mapped as of now, including *C. elegans* ([WSTB86]) and *Drosophila melanogaster* ([DMS$^+$24]).
A complementary approach is offered by the idea of functional recordings to capture the actual activity of neurons over time. This has been classically achieved by electrode-based methods that provide high temporal resolution by leveraging the electrophysiological properties of neurons ([Kha22]).
Over time, the dogma of single-neuron information encoding has repeatedly been challenged by the idea that instead, populations of neurons encode information in a multiplexed manner ([EH21]). When these ideas were novel, they were not testable due to the technological state of recording techniques and computational resources ([SC19]).
However, due to major progress in imaging technology and microscopy in recent decades, recording neuronal activity became feasible on a population level ([UDLC22]). Therefore, this alternative approach of studying the functional dynamics underlying brain activity plays a central role in modern neuroscience and there are nowadays several established methods to capture large-scale high-resolution population activity in real time. A technique falling into the category of optical imaging and commonly used across model organisms is to record fluorescent calcium indicators in transgenic animals ([LS16]).

1.2. The Neuronal Manifold

With the rise of optical imaging, neuroscientists are able to measure activity from more neurons than ever before (Figure 1.1), creating a whole new field of application for

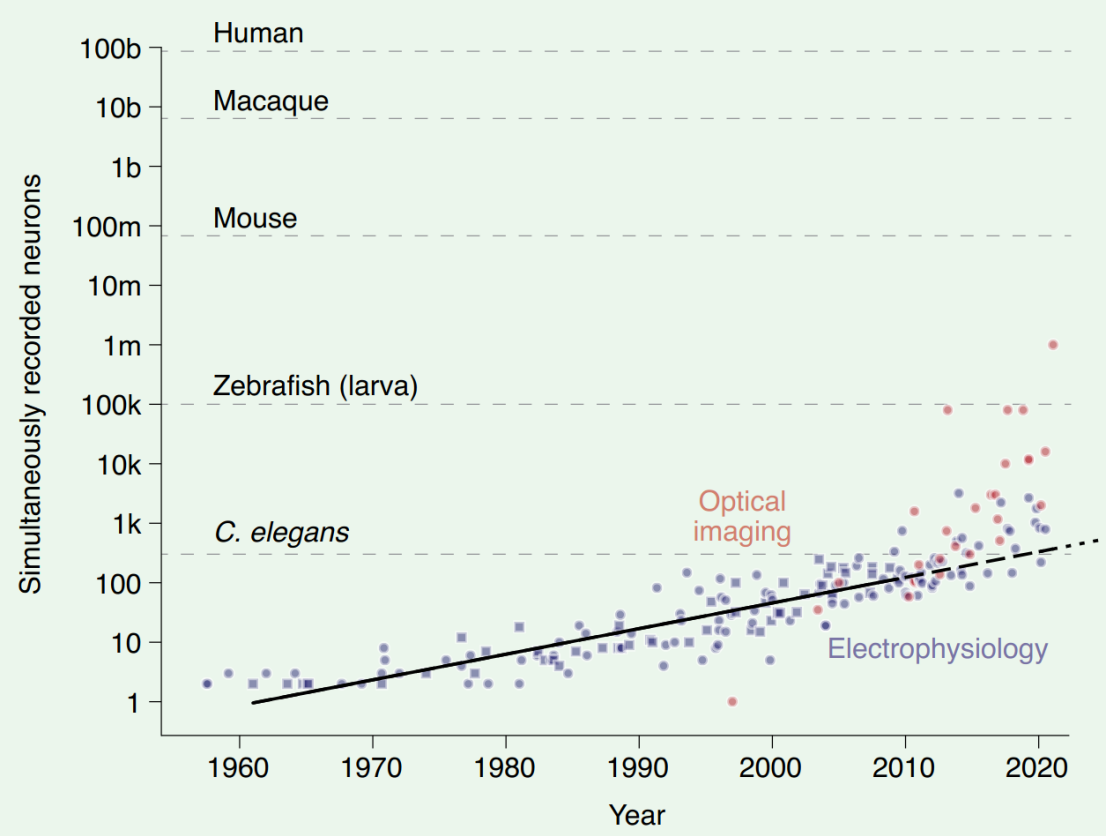


Figure 1.1.: **The rise of optical imaging.** The blue dots indicate electrophysiology data up until 2011, published in [SK11]. The blue squares indicate new data since then, compiled in [UDLC22]. Red dots indicate new optical imaging data. The black line denotes an exponential fit to the original data until 2011, extrapolated to the present/future where dashed/dotted. The figure was adapted from [UDLC22].

data science, machine learning and computational methods in order to analyze these sometimes brain-wide recordings. In this context, several studies have uncovered a surprising common feature in worms ([KKS+15]), flies ([AKJ+19], [SMW+23]) and mice ([SPS+19]): Brain-wide signals related to ongoing behavior that even pervade neuronal populations which were usually associated with sensory processing. This led to the suggestion that the brain-wide representation of ongoing behavior might be a universal principle in neuroscience ([KZ20]).

A landmark study in this area was the identification of a low-dimensional representation of such brain-wide activity in the nematode *Caenorhabditis elegans* ([KKS+15]). With 302 neurons, the space of its possible neuronal population states could in theory be as huge as 2^{302} when already constraining neurons to a binary state. However, the actually assumed brain states appear to be far less and a large portion of activity can be

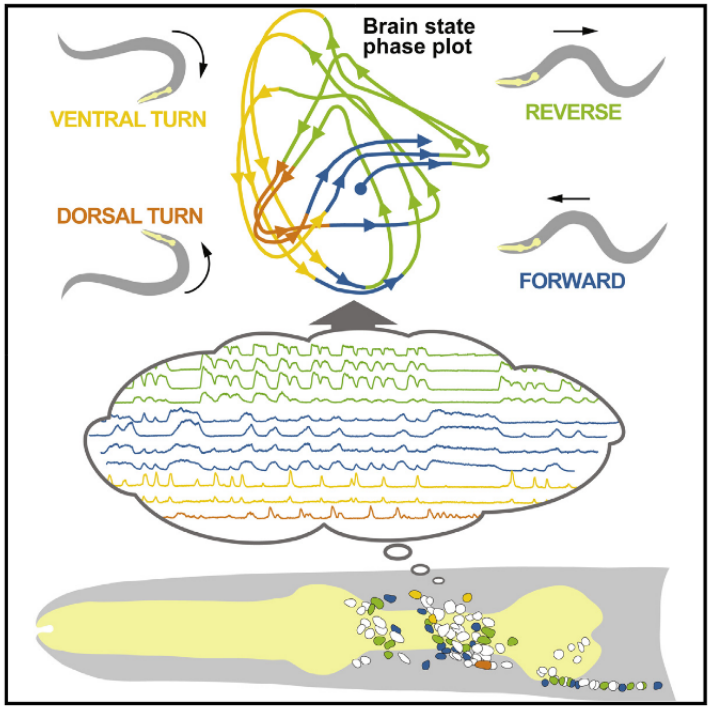


Figure 1.2.: **The neuronal manifold encodes the global action sequence in *C. elegans*.** Brainwide neuronal activity can be reduced to a low-dimensional brain state signal that encodes high level behavioral commands. Adapted from [KKS+15].

modeled with as little as two to three dimensions only. In *C. elegans*, this low-dimensional widespread signal, termed the "Neuronal Manifold", can even be explicitly mapped to the animals' major motor commands (Figure 1.2).
That manifolds can encode relevant aspects of behavior has also been demonstrated in primate motor cortical areas, where both the planning and execution of arm reaches have been functionally linked to neuronal trajectories on low-dimensional manifolds [CCK+12]. Due to their capacity to give meaningful insights into the underlying structure and function of neuronal activity, finding neuronal manifolds by utilizing methods of dimensionality reduction (see 2.2.1) became a common procedure when analysing large-scale recordings.
However, the functions of neuronal manifolds and the mechanisms generating them remain incompletely understood.

1.3. Context Dependency

Additional to representing ongoing behavior, one function of this low-dimensional distributed brain-wide activity could be to organize complex behavior patterns. As established, the major motor command action sequence in *C. elegans*, consisting primarily of forward

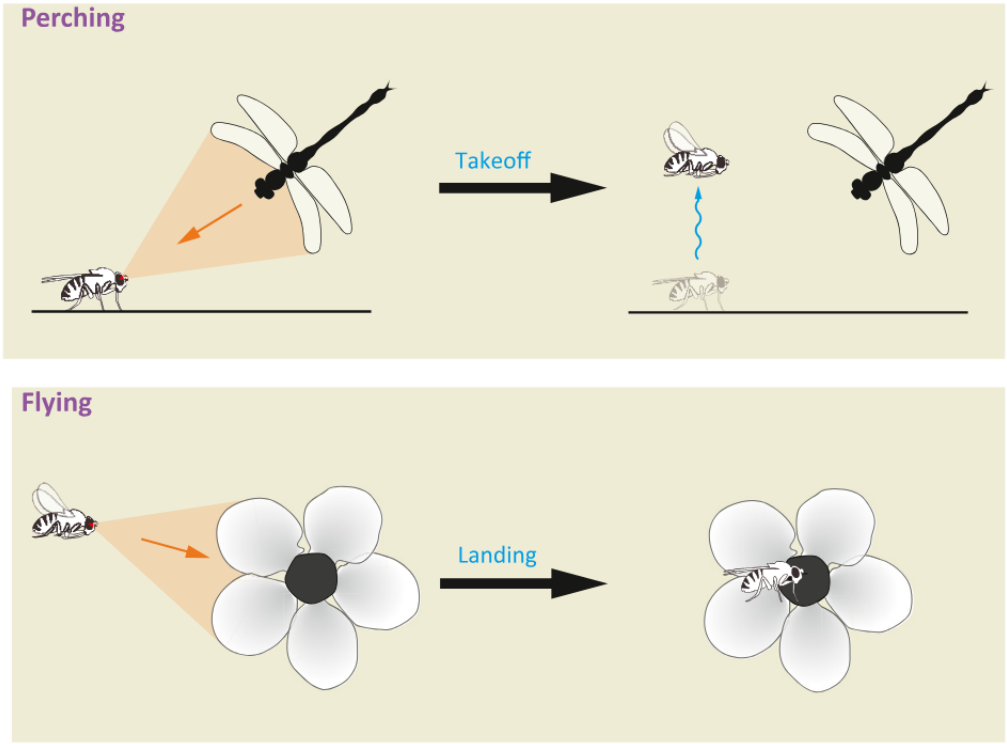


Figure 1.3.: **Context dependency in *Drosophila*.** Different looming stimuli can leave similar images on the retina - however, depending on the currently exhibited high-level state, the encoding of this sensory input changes, resulting in different behavioral responses downstream. Adapted from [OC22].

and backward movement followed by turns, is known to be encoded in the global manifold signal [KKS+15]. However, the worm exhibits additional, more granular behavior patterns, occurring within these motor commands.
In order to interact with their environment, an animal must be able to adopt behavioral patterns that serve direct purposes, like movement, sampling input from the environment via sensory organs, or food intake. However, to pursue longer-term behavioral goals, it must also be able to keep up a more stable overarching behavioral state. A behavioral hierarchy can be established, placing these more abstract global internal states of behavior that change on a slower times-scale at a higher level and more granular, concrete behavior that plays out on a faster time-scale at a lower level.
How are lower level aspects of behavior that are not encoded within the manifold signal represented in the brain? Furthermore, is there an interaction between levels of behavior? Studies have found that in *C. elegans*, co-occurring behaviors at different time-scale levels are also represented on the neuronal level via different time-scales of neuronal dynamics. These are combined into nested representations, sometimes within the same neuron, encoding corresponding levels of behavior in a multiplexed manner [KTKZ20].

Furthermore, higher levels of behavior seem to modulate lower levels. For instance, the worm exhibits fast head casts only during forward crawling. Head-casts therefore exhibit a type of state dependency.

This interaction, more broadly referred to as context-dependency, has also been found to extend to the encoding level ([FLR+25]): The way information is represented in the brain, i.e. the encoding itself, changes depending on the context. In the brain of *C. elegans*, the encoding of the instantaneous body posture during crawling movement changes depending on the slower global manifold signal.
The mechanism is not bound to *C. elegans* only. It has been shown that in *Drosophila melanogaster*, the way sensory information is processed depends on the higher level behavioral state (Figure 1.3). Presenting a looming stimulus during flight triggered a landing response, whereas presenting a similar stimulus during perching resulted in a takeoff response. Even though both stimuli left a similar image on the retina, the encoding of this sensory information changed depending not on the sensory input, but on the internal higher level state the fly exhibited at the time, resulting in a significantly different behavioral decision downstream [OC22].

In this thesis, we build on previous work in hierarchical organization of behavioral encodings in *C. elegans* that showed these context-dependent dynamics occurring on an encoding level [FLR+25]. With the purpose of generalizing the approach, we present a computational Hierarchical Gating Model (HGM), that decomposes neuronal population activity into a global manifold component and a component that encodes other information, in our case low-level behavior. We combine both components in a nonlinear manner to predict a state-dependent representation of this information on a neuronal level. The HGM achieves this by learning the manifold, behavior encoding and interaction signals simultaneously, via joint optimization. This manifold-learning approach constitutes a notable improvement over the previous work in the field which commonly relies on pre-computed manifold signals. Additionally, the HGM is designed to be compatible across model organisms, without the need to identify neurons across datasets, as it can be done in *C. elegans*. For implementation and training, we use the high-performance computing and machine learning library JAX [BFH+18].
In the second part of this thesis we therefore apply the model to whole-brain freely moving *C. elegans* calcium imaging data [FLR+25], as well as calcium imaging data from head-fixed flies [SMW+23] and mice [SPS+19], and present the results. Research, exploratory analysis and suitability assessment of the selected fly and mice datasets were also part of the thesis.

2. Preliminaries

2.1. Biological datasets

In this work we will analyze datasets from *C. elegans* [FLR$^+$25], *Drosophila melanogaster* [SMW$^+$23] and mice [SPS$^+$19]. In all three cases, a calcium imaging technique is used to record neuronal activity. This is done by using transgenic lines expressing the biosensor protein GCaMP, that fluoresces when bound to Calcium ions (Ca^2+). As a result, GCaMP expression causes neuron fluorescence when exposed to laser light, which can be imaged as a proxy for neural activity. These signals are then globally normalized and corrected for motion artifacts as well as an effect called photobleaching, where the fluorescence intensity decreases over time due to light exposure.
Additionally, each dataset contains varying quantifications of behavior acquired during the recording sessions. We consider more abstract global internal states, often discrete, as high-level behavior and more concrete, granular and faster continuous quantifications as low-level behavior.

2.1.1. *C. elegans* data

Neuronal data

34 unstimulated individual worms were imaged at 3.5 Hz for 8 minutes while freely moving on a 2% agarose pad under a cover glass. *C. elegans* is known to have a canonical connectome with 302 neurons in total, enabling identification of individual neurons across worms. On average, 120 neurons were recorded per worm, located in the nerve ring and head ganglia, out of which 40% could be identified. A GCaMP7b green channel was used, together with an additional mScarlet red channel, that fluoresces independently of calcium influx.
Both were respectively bleach-corrected via an exponential fit. The signal was then corrected to only capture activity-related intensity via

$$R = \frac{F_{green} - F_{red}}{F_{red}} \tag{2.1}$$

and the final normalized trace was computed as

$$\Delta R/R = \frac{R - R_{50}}{R_{50}} \tag{2.2}$$

where the baseline signal R_{50} is estimated as the median R over the full recording session.

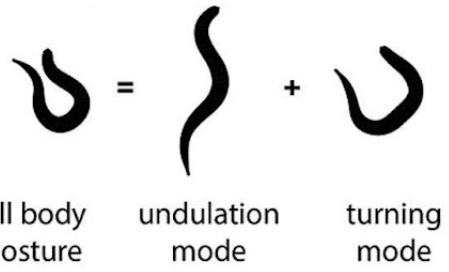


Figure 2.1.: **Eigenworm posture reconstruction.** The full body posture can be decomposed into modes by applying PCA to the curvature timeseries.

Analogously, 8 additional recordings were obtained under similar conditions, but using unmodified GFP instead of GCaMP7b. These calcium-independent recordings are useful as a control group, e.g. to correct for possible motion artifacts.

Behavioral quantifications

While freely moving, the worms was recorded via a high-speed camera at ∼83 frames per second. After segmentation, the worm was reduced to a 1D midline curve, represented by a sequence of points along its body. By computing the relative local curvature at each point, a bending-based timeseries of body posture was obtained. Applying PCA (see 2.2) to this timeseries yields the principal building blocks from which a full body posture can be reconstructed, termed eigenworms Figure 2.1 [FLR$^+$25].
The timeseries of instantaneous body posture as projected onto the eigenworms (i.e. the modes of the PCA, see 2.2) were used as the main behavioral quantification for *C. elegans* in this work and henceforth also referred to as "low-level behavior".

Additionally, discrete behavioral states were quantified. The worm follows a stereotyped sequence of motor commands, consisting a Forward Crawling state that can be interrupted by a so-called pirouette event, in which it changes to a short reverse crawl, followed by either a Ventral or Dorsal Turn, after which it usually resumed forward crawling [KKS$^+$15]. Additionally to these states, a Slowing and Pause state were quantified, resulting in a total of 8 labels: "FWD", "REV", "VENTRAL_TURN", "DORSAL_TURN", "SLOWING", "PAUSE", "TRACKING_FAILURE" and "UNKNOWN" if none of the earlier apply. The timeseries for each of these labels were obtained in various ways from the eigenworm projection timeseries or raw speed (see methods in [FLR$^+$25]).
These discrete states will henceforth also be referred to as "high-level behavior".

2.1.2. *Drosophila* data

Neuronal data

18 individual head-fixed flies were imaged at 8-12 volumes per second for 18 minutes while behaving freely on a spherical treadmill. On average, 1419 ± 78 stable ROIs were

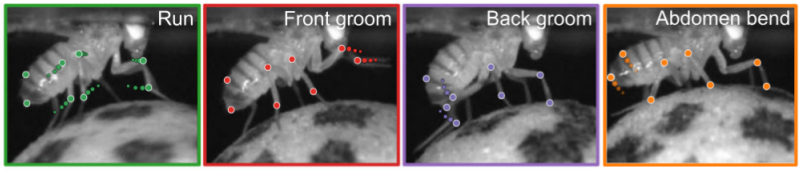


Figure 2.2.: **DeepGraphPose markers.** A semi-supervised sequence model classifies marker positions into high-level behavioral states. Adapted from [SMW+23].

identified per animal, located in the dorsal third of the central brain. A GCaMP6s green channel was used, together with a dsRed red channel.
The ratiometric signal was then computed via (2.1) using the raw signals and bleach correction was applied simultaneously with normalization, by using a best-fit exponential baseline in (2.2), instead of a simple median.

Behavioral quantifications

The low-level behavioral data for *Drosophila* consists of eight tracked points (markers) on the body of the fly, obtained using the deep learning-based pose estimation toolbox DeepGraphPose (see methods in [SMW+23]) on a ∼70 frames per second behavioral video recording during imaging (Figure 2.2). The DeepGraphPose output is a timeseries of absolute image x and y coordinates for each marker, resulting in 16 timeseries per fly.

Similar to the worm data, a set of high-level discrete behavior labels was segmented from the DGP markers, using a semi-supervised sequence model (see methods in [SMW+23]). Five salient behavioral states were hand-labeled and used to train the DGP model: "walk", which refers to running, "still", which refers to quiescence, as well as "front-groom", "back-groom", "abdomen-move" and "background" if none of the earlier apply.

2.1.3. Mouse data

Neuronal data

6 individual head-fixed awake mice were recorded at 2.5 Hz over 9 sessions for 70-160 minutes while free to run on an air-floating ball. About ∼11.000 neurons were recorded per session, located in the primary visual cortex (V1).
The raw intensity from the single green GCaMP6s channel was neuropil corrected and directly normalized via (2.2), estimating the baseline activity from a running 60s window.

Whereas most signals in *C. elegans* are propagated in the form of graded potentials, mammalian brains are dominated by all-or-none action potentials. Therefore, a nonnegative spike deconvolution was applied to the slow calcium signals, using the OASIS algorithm with a fixed time scale of calcium indicator decay of 2s [FZP17].

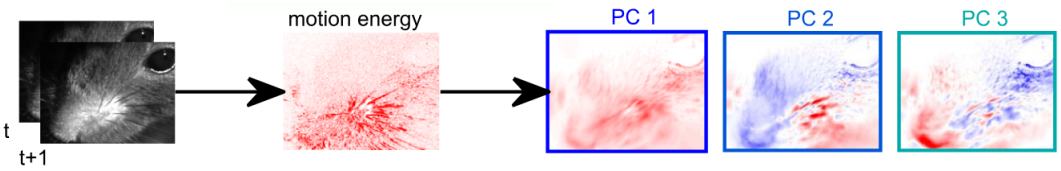


Figure 2.3.: **Facial video processing.** PCA is applied to the Motion Energy of the face video. Adapted from [SPS+19]

Behavioral quantifications

The behavioral quantification for mice significantly differs from the other two species. The speed of the ball was separately measured using optical computer mice and a 30 fps video of the face was recorded during imaging for each session. From this video, three further behavioral variables were extracted: Pupil diameter, whisking and facial movement. Pupil area was estimated by a simple threshold for pixel brightness, whereas whisking and facial movement was quantified using the absolute motion energy (see 2.2) of the whisker area and full video, respectively. A 1-dimensional whisking trace was defined as the sum of pixel values for each motion energy frame and the full face motion energy was reduced to 1000 Dimensions via PCA (Figure 2.3).
This results in a collection of three one-dimensional variables "running", "pupil area" and "whisking", claimed to be related to general arousal [SPS+19], and the additional more complex facial movement quantification, capturing more fine-grained behavior.

Therefore, we consider the three arousal variables high- and the 1000 face motion principal components as low-level behavior.

2.2. Computational Tools

2.2.1. Dimensionality reduction

When dealing with a high number of features k, a classic approach to find underlying structure in the data is to reduce its dimensionality, i.e. to reduce the amount of features while preserving information. The dimensionality reduction techniques used in this work all have in common that they reduce dimensionality in a linear manner, transforming a set $X \in \mathbb{R}^{n \times m}$ of m-dimensional variables into a set $Z \in \mathbb{R}^{n \times l}$ of l-dimensional latent variables, usually with $l << m$.
This is achieved by learning a linear transformation defined by a matrix W. This matrix is fitted on internal metrics of the resulting latent variables. Therefore, this type of dimensionality reduction falls into the category of unsupervised learning.
Each of the following techniques pursues a unique goal by optimizing a different metric in the latent variable space.

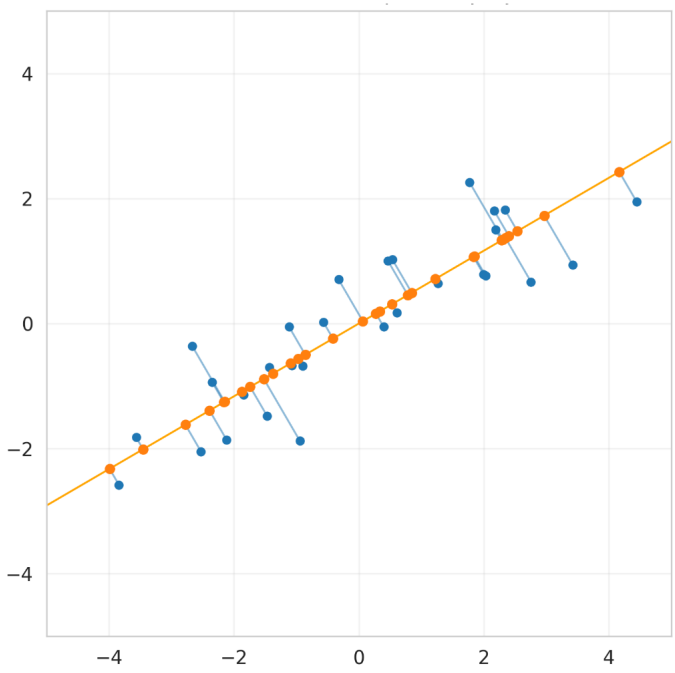


Figure 2.4.: **Geometric intuition of PCA.** The variance of the data the latent space (orange) is maximized.

Principal Component Analysis

Arguably the most well-known technique is called Principal Component Analysis (PCA) [Jol02]. It learns an orthogonal matrix W that transforms the variables into a lower dimension while maximizing the variance of the variables in the latent space:

$$W = \arg\max_{W} \; Var(XW) \quad \text{s.t. } W^T W = I \tag{2.3}$$

It can be shown that maximizing the variance of the variables in the latent space $Z = XW$ is equivalent to minimizing the error $||X - \hat{X}||_F^2$ of reconstructing the original variables from the latent variables via $\hat{X} = ZW^+$, with W^+ denoting the Moore-Penrose inverse of W.
Geometrically, PCA can be interpreted as a rotation of the original data points (blue), followed by a projection into a lower-dimensional space, such that they cover the most area (Figure 2.4). The reconstruction (orange) is the embedding of this lower-dimensional space in the original data space.
The so-called loadings of a PCA denote how much each original variable is aligned with each latent variable. The latent variables are also called Modes or Principal Components (PCs).

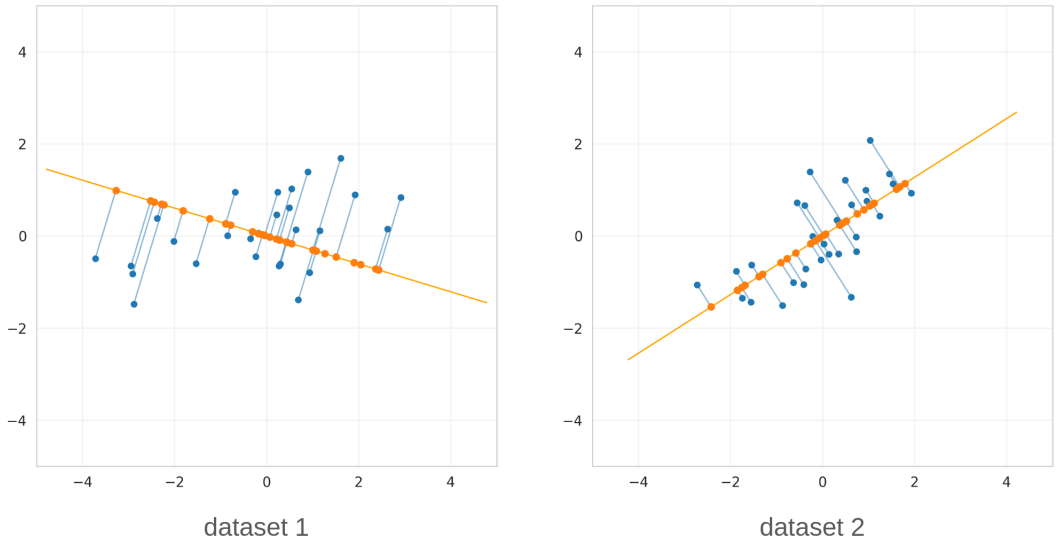


Figure 2.5.: **Geometric intuition of CCA.** The correlation between the data two latent spaces (orange) is maximized.

Canonical Correlation Analysis

While PCA optimizes variance, Canonical Correlation Analysis (CCA) optimizes cross-correlation. Accordingly, it requires two datasets $X \in \mathbb{R}^{n \times m}$ and $Y \in \mathbb{R}^{n \times k}$ as input. Instead of finding a single weight matrix W, it finds two weight matrices $W_X \in \mathbb{R}^{m \times l}$ and $W_Y \in \mathbb{R}^{k \times l}$ and learns two low-dimensional projections $Z_X = XW_X \in \mathbb{R}^{n \times l}$ and $Z_Y = YW_Y \in \mathbb{R}^{n \times l}$. These two weight matrices are learned by optimizing cross-correlation between the two latent spaces:

$$W_X, W_Y = \arg\max_{W_X, W_Y} \; Corr(Z_X W_X, Z_Y W_Y) \tag{2.4}$$

Geometrically, CCA can be interpreted as a whitening of the original data points (blue), followed by a rotation and then a projection into a lower-dimensional space (reconstruction in orange), such that they look most similar (Figure 2.5).
The loadings of a CCA similarly denote how much each original variable is aligned with each latent variable. The latent variables are also called Modes or Canonical Variates. Note that, different to PCA, the reconstructed embeddings of the Canonical Variates are not necessarily orthogonal in the multidimensional case.

Partial Least-Squares Canonical

A rather similar approach to CCA is Partial Least-Squares Canonical (PLS). It also requires two datasets, but instead of optimizing cross-correlation, it optimizes raw cross-covariance.

$$W_X, W_Y = \arg\max_{W_X, W_Y} \; Cov(Z_X W_X, Z_Y W_Y) \quad \text{s.t. } W_X^T W_X = I, W_Y^T W_Y = I \tag{2.5}$$

PLS can be loosely interpreted as a mixture of PCA and CCA, as it takes variance as well as correlation into account.
In the context of this work, PLS is only used with the goal to give additional visual intuition and is therefore not the focus of further explanations.

Linear Autoencoder

The least constrained method within the field of linear dimensionality reduction is a simple Linear Autoencoder. It simultaneously learns an encoder matrix $W \in \mathbb{R}^{m \times l}$ and a decoder matrix $V \in \mathbb{R}^{l \times m}$, such that the reconstruction error is minimized:

$$W, V = \arg\min_{W,V} \; ||X - XWV||_F^2 \tag{2.6}$$

with F denoting the Frobenius norm [HTF09a].
Note that as in this case maximizing the variance in the latent space is equivalent to minimizing this reconstruction error, constraining a linear autoencoder to orthogonal matrices effectively turns it into PCA.
As Linear Autoencoders are usually trained via optimization (see 2.2.4), they are very flexible and can be easily adjusted according to additional requirements (e.g. orthogonality).

Explained Variance Ratio

The previous methods all have in common that they use different goals to learn linear transformation matrices that project input data into a lower-dimensional space. When working with high-dimensional data, it is helpful to compute the amount of variance in the original data that can be explained when only taking the latent variables into account. This is achieved by the explained variance ratio metric:

$$\text{Explained Variance Ratio} = 1 - \frac{Var(X - \hat{X})}{Var(X)} \tag{2.7}$$

where $\hat{X}$ denotes reconstruction. With $l << n$, as is generally the case, the reduction matrix W is not full rank and therefore its inverse W^{-1} undefined. For this reason, an estimator for W^{-1} must be found that reconstructs the data with minimum error, embedding its lower-dimensional representation back into the original data space. Each technique can have its own way of computing its reconstruction matrix. E.g. in PCA, it has shown to be the Moore-Penrose inverse of W and in a Linear Autoencoder that already minimizes the reconstruction error, it corresponds to the simultaneously learned decoder matrix V.

2.2.2. Linear Regression

In the context of machine learning, linear regression is arguably the most basic form of modelling. However, it has countless extensions and serves as a powerful baseline to find

linear connections between datasets. The idea is to represent a set of n multidimensional targets $Y \in \mathbb{R}^{n \times m}$ as a linear combination of another set of predictors $X \in \mathbb{R}^{n \times b}$, by learning a weight matrix $W \in \mathbb{R}^{b \times m}$, such that the error between the predicted and actual targets is minimized:

$$W = \arg\min_{W} ||Y - XW||_F^2 \tag{2.8}$$

Linear regression falls into the category of supervised learning, as it needs a set of external targets to learn W.

Reduced Rank Regression

Reduced Rank Regression (RRR) works similar to linear regression, as it also linearly represents targets as a linear combination from predictors, minimizing the same error. However, instead of learning a single linear transformation matrix, two matrices $W \in \mathbb{R}^{b \times r}$ and $V \in \mathbb{R}^{r \times m}$ are fitted.

$$W, V = \arg\min_{W,V} ||Y - XWV||_F^2 \tag{2.9}$$

$r < \min(b, m)$ is called the reduced rank. Instead of a direct connection between predictors and targets, RRR first projects the predictors into a lower-dimensional space $Z = XW$ and then predicts the targets from these latent representations as $\hat{Y} = ZV$. W and V can also be merged into a single matrix with a rank $\leq r$.
Enforcing a projection into a lower-dimensional space before target prediction can give insights into the intrinsic dimensionality of the data. In this sense, RRR is also a dimensionality reduction technique. In fact, it is strikingly similar to a Linear Autoencoder, only differing in their supervised vs. unsupervised prediction goal.

Coefficient of determination

The coefficient of determination, denoted as R^2, is a measure of prediction goodness, often used in the context of linear regression. Is is defined per feature (i.e. dimension) k, as the sum of the squared differences between each target y_{ik} and its respective prediction $\hat{y}_{ik}$, with respect to the sum of the squared differences between each target y_{ik} and the target mean $\bar{y}_{ik}$.

$$R_k^2 = 1 - \frac{\sum_{i=1}^{n}(y_{ik} - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_{ik} - \bar{y}_k)^2} \tag{2.10}$$

In a broader interpretation, the target mean $\bar{y}_{ik}$ is the prediction of a model that can only predict a flat line, essentially just learning a bias. Therefore, the R^2 score can be understood as the performance of a model with respect to the performance of a flat line model.
In the case of multidimensional targets, the total R^2 across features is defined as the average of all per-feature R^2's:

$$R^2 = \frac{1}{m}\sum_{k=1}^{m} R_k^2 \tag{2.11}$$

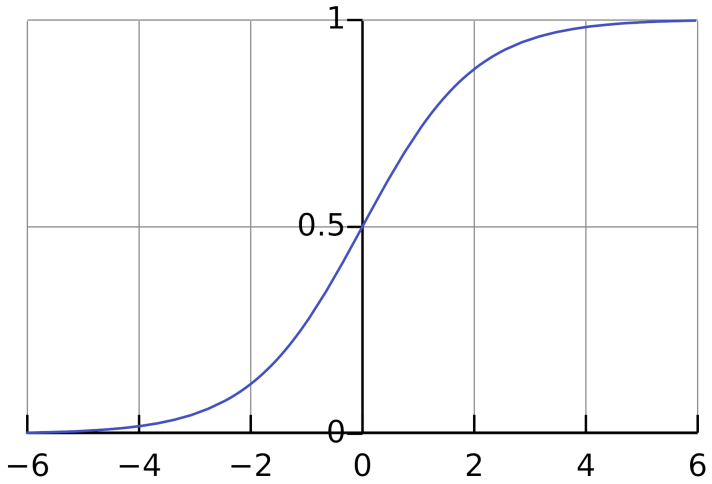


Figure 2.6.: Sigmoid Function $\sigma(x)$.

2.2.3. Multi-Layered Perceptron

A Multi-layered Perceptron (MLP) is the most basic version of a neural network. It simply consists of one or more consecutive linear regression models, where each output prediction is the input to the next regression. In this context, such a set of regression weights is called a layer. Additionally, each prediction is post-processed element-wise before the next layer by a so-called Activation Function, typically the sigmoid function. This activation function adds nonlinearity to the MLP and prevents it from collapsing into a single linear operation.

In summary, a MLP is a nonlinear unsupervised learning method and therefore the most complex technique introduced so far.

Sigmoid Function

The sigmoid function is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2.12}$$

It has two main features: It acts as a normalizer, mapping the interval of $[-\infty, \infty]$ to $[0, 1]$ and due to its nonlinear structure, it is able to emphasize changes closer to 0.

2.2.4. Optimization

All previously established linear machine learning methods have a closed-form or algorithmic solution. That means that their optimal solution can be solved for explicitly, or found within a finite number of steps. In that context, this optimal solution can be

viewed as the minimum (or maximum) of a function that takes their model parameters (e.g. weights W) as input and returns the metric they optimize (e.g. $||X - \hat{X}||_F^2$) as output. Such a function is called a loss function. For each machine learning model, a loss function can be formulated such that the model parameters that correspond to the best achievable model performance return the loss function's minimum when used as input.

Gradient Descent

There are many relevant, often nonlinear models (i.e. MLPs) that have no clear closed-form or algorithmic solution. In this case, they have to be trained computationally on their loss function. Such training is done by approximating the minimum of a function by iteratively moving along its gradient. The most basic technique to do so is called Gradient Descent:

$$W^{(t+1)} = W^{(t)} - \eta \nabla \mathcal{L}(W^{(t)}) \tag{2.13}$$

Here, $\mathcal{L}$ denotes the loss function and ∇ the gradient. In each iteration, the new model parameters $W^{(t+1)}$ are obtained by moving along the negative gradient of the loss function with respect of the previous parameters $W^{(t)}$, in steps of size η. η is called the Learning Rate.
Most modern machine learning models use more complex gradient descent based optimizers like Adam ([KB15]), that take gradient momentum into account.

Autograd

Gradient Descent requires finding the loss function's gradient. For increasingly complex loss functions, the formal mathematical definition of their derivative can easily explode in number of terms. Instead of defining the gradient formally using the chain and product rule on the full function, Autograd is a technique to parallely compute gradients efficiently during numerical computations, by using the chain rule locally at each elementary step in order to trace the gradient through the whole function. It is used by all classic machine learning libraries and proved itself to be the most powerful approach.

2.2.5. Training

Cross-Validation

The more complex a model is, the more powerful it becomes in its ability to model complex data. However, this is not always an advantage, as it can lead to modeling noise in the dataset it is trained on, an effect called overfitting ([HTF09b]). When a model is overfit, it will perform particularly well on the specific dataset it is trained on, while performing badly on new datasets. In order to assess how well a model generalizes, it needs to be tested on held-out data. Therefore, a common strategy is to split the training data into a training and a validation set, typically with a ratio of 80/20. In order to get a reliable performance estimate, this is usually done multiple times with different non-overlapping subsets used for validation each time, until all data was used once. Such a split into training and validation set is called a fold.

Regularization

Regularization is a machine learning technique to counteract overfitting by restricting the model parameters in various ways. Usually, large weights are penalized by an additional loss term (more on that in 5.4.1). It aims to limit the model's sensitivity by preventing small variations in the training data to cause large variations in the model predictions. In that way, it effectively stabilizes the model and reduces its complexity. Regularization strength can be tuned with a parameter called λ.
Different to cross-validation, its a way to prevent overfitting, rather than assessing it.

Hyperparameter Sweeping

The parameters of a model that are learned, such as weights W, are simply referred to as model parameters. However, when designing a model, there are additional parameters that define how the model learns, such as the learning rate (2.2.4) or the regularization λ (2.2.5). Therefore, these parameters are called hyperparameters. The choice of hyperparameters is crucial in the context of tuning a model.
Hyperparameter sweeping refers to the process of repeatedly training a model using different combinations of hyperparameter candidates and tracking the model performance for each configuration in order to find the set of hyperparameters that gives the best validation set predictions. There are different strategies for sweeping. The strategy applied for this work's models is further discussed in the methods.

2.2.6. Significance Assessment

Shuffle Control

In order to determine the predictive power of input data with respect to targets, the predictions of a model trained on this input data can be compared to the predictions of a model that is trained on a control group of input data. This control group is usually obtained by modifying the original input with the goal of decoupling it from the targets, e.g. by shuffling them. By doing this, the underlying structural connection between input and output is erased and the model learns a baseline by-chance prediction. Running multiple training processes on differently shuffled input data generates a baseline distribution of model scores, against which the actual prediction can be statistically compared to assess significance. More on the shuffle strategy in 5.5.2.

Sample p-value

The classical right-sided p-value of a sample is defined with respect to the distribution it is drawn from. In that context, it denotes the probability of drawing another sample that is larger. The right-sided sample p-value of a sample is defined with respect to a set of samples. In that context, it is defined as the relative amount of samples in the set that are larger. In other words, it is defined as $1 - p$ with p being the percentile of the sample.

3. Related Work

3.1. State-dependent encoding in *C. elegans*

As previously established, the animal exhibits more granular behavior patterns that are nested within global motor commands. More precisely, a behavioral hierarchy based on time-scales can be observed (Figure 3.1). On the highest, slowest level, *C. elegans* exhibits an action sequence of motor commands, where the switch between e.g. forward and backward crawling happens on a time scale of approximately 0.05Hz. During crawling, it exhibits body undulations that alternate between ventral and dorsal bends, as the worm is laying on its side, propagating from front to back (or back to front during reversals) at a time scale of approximately 0.5Hz. Additionally, nested within these undulations, even faster head-casts can be observed at about 1Hz, that are only exhibited during forward crawling ([KTKZ20]).
A corresponding nested representation of dynamics at different time-scales was also found on the neuronal level. However, the dynamics of context-dependency on the encoding level and its connection to the neuronal manifold still remained unclear.

However, significant progress regarding this questions has been made in [FLR+25]. Fieseler et al. showed that when subtracting out the part of a neuron's activity that corresponds to the global manifold signal, in some neurons (e.g. "VB02", "DB01" and "DD01") the residual activity after subtraction shows oscillations strongly correlated with the animal's body undulations during crawling. A triggered average of the residual activity during forward crawling, relative to the time of maximum ventral curvature, shows a consistent maximum (or minimum) around this time point (Figure 3.2).
However, during backward crawling, the amplitude of said oscillations significantly diminish in some cases (e.g. "VB02" and "DB01"). Therefore, their work hypothesizes that the neuronal manifold signal that encodes high-level behavioral states including forward- and backward-crawling, also gates the amplitude of the residual activity that encodes body undulations. They formalize this idea by introducing a mathematical model to predict gated residual activity. In the next section, we describe it briefly in the context of other approaches to model neuronal activity.

3.2. Modelling neuronal activity

In neuroscience, there is a large landscape of models for neuronal activity and its connection to behavioral quantifications, sensory information or internal states. In this section, we will give an overview of a few prominent approaches and categorize them by

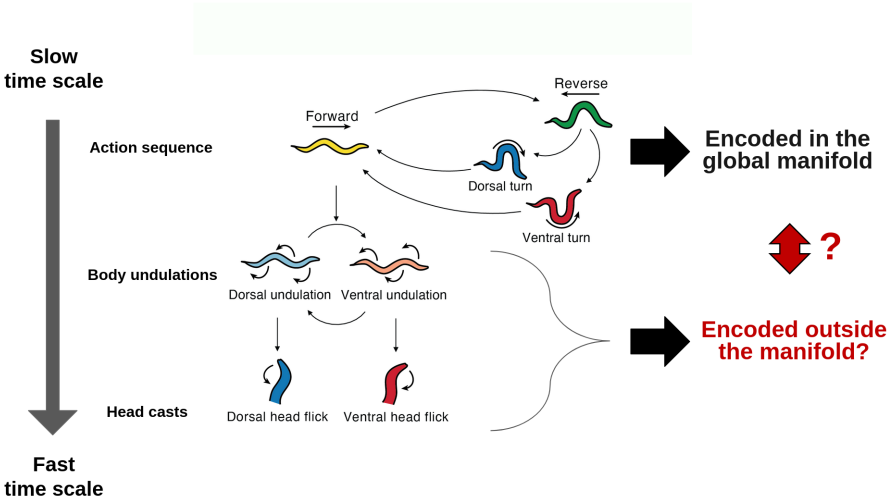


Figure 3.1.: **Behavioral time-scale hierarchy of *C. elegans*.** The global manifold signal is known to encode the slow motor command action sequence. At a faster time scale, the worm exhibits ventral and dorsal body undulations during crawling movement. Nested within these undulation movements, even faster head-casts are observed. How does neuronal activity represent these granular behavior patterns whose encoding is not found in the manifold signal? Furthermore, does the hierarchical interaction between the levels extend to the encoding level and if yes, how?

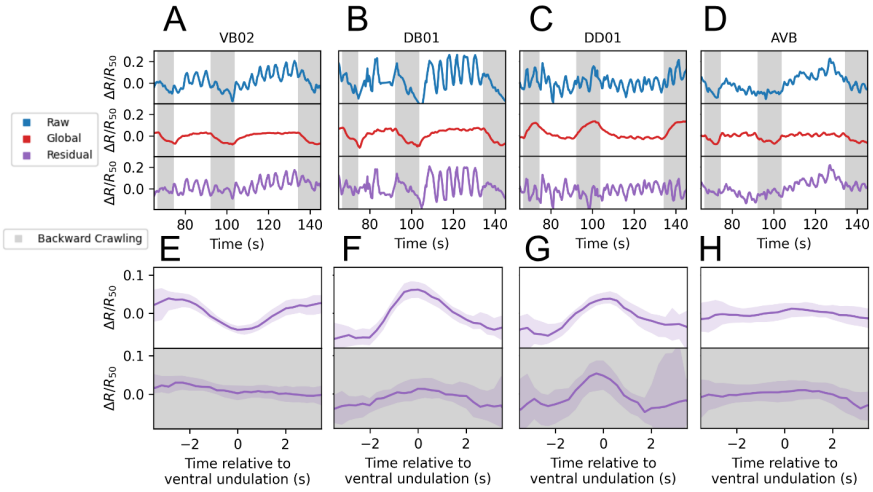


Figure 3.2.: **The global manifold signal gates residual activity.** Example traces of neurons of interest are shown, decomposed into a global (red) manifold part and the residual (purple). Triggered averages with respect to body undulation are shown below. The examples highlight the different cases of no encoding (D & H), linear encoding (C & G) and nonlinear encoding depending on the behavioral state encoded in the manifold (A & E, B & F). Adapted from [FLR+25].

model complexity in a table, with focus on modulation dynamics (Table 3.1). A second category axis describes the biological features the respective model is designed to capture.

	Stimulus, Behavior, Internal State	**Stimulus, Internal State**	**Behavior**	**Behavior (no multiplexing)**	**Internal State**
Continuous modulation			Hierarchy Model [FLR+25], This Thesis		
Discrete modulation	Internal State Conditioned Neural Encoding Model [ALS+25]	Modulated Poisson Model [GMS14]			SLDS [LNB+19]
No modulation					HMM, GPFA [YCS+09], RLVM [WB17]
No modulation, strictly linear			Linear Encoding Model [MKJ+19]	CCA, PLS, RRR [SPS+19], Linear Regression [SMW+23]	PCA [KKS+15], Raw data

Table 3.1.: **Model landscape**. Vertical axis categorizes by modulation complexity, horizontal axis categorizes by captured biological features. The displayed models cover a mixture of different approaches, some modeling time dynamics while others treat time points as independent samples. In addition, black entries indicate modelling activity from latent variables (i.e. making use of manifolds) while gray entries do not. Behavior is modeled in a multiplexed manner unless stated otherwise.

In order to get an idea of how the work of this thesis can be categorized in the context of modelling neuronal activity, we will briefly go over each entry and summarize its idea.

3.2.1. Strictly Linear Models

In functional imaging, neuronal activity is measured over time, e.g. from calcium as a proxy (see 2.1). After post-processing, the result usually contains one activity time-series per captured neuron. The time-dependent nature of such data motivates the use of dynamical models. However, it's a powerful data-driven approach to treat each time point as an independent sample and accordingly, a multivariate timeseries as a point cloud of multidimensional features. The "No modulation, strictly linear" category of Table 3.1 classifies such models that do not take time dynamics into account. All other rows contain nonlinearity, possibly modelling time, and with varying degrees of modulation.

A simple, yet powerful analysis in this context and the basis of the methodology in

[KKS+15] is PCA. When used on timeseries data, PCA treats every timepoint as an independent sample, capturing the most prominent spatiotemporal structures. Since PCA reduces dimensionality, it essentially models the data in a linear manner from a low-dimensional latent time-series ("temporal modes") and doesn't need any external quantifications.
On the other hand, CCA and PLS also reduce dimensionality and capture latent structure in the data, but in contrast to PCA, they do this with the goal to find correlations to an external set of target variables. In neuroscience, they are often used together with timeseries of behavioral quantifications to find latent patterns shared between neuronal activity and behavior, which can be interpreted as given behaviors to be encoded in the neuronal activity.
Another simple approach to find a behavioral encoding is to model neuronal activity via linear regression, using behavior as predictors, as done in [SMW+23]. It allows depicting activity from behavior directly, without enforcing a low-dimensional structure. It's a less constrained method but its interpretation is weaker.
Lastly, both ideas are combined in Reduced Rank Regression. The main difference to CCA/PLS in this context is that instead of identifying latent structures in the actual neuronal activity that are correlated to behavior, it computes a depiction of neuronal activity that arises from latent structures in the actual behavior. Both are useful when assuming intrinsic low-dimensionality and RRR has been done in [SPS+19].

An important distinction is to be made between direct and multiplexed behavioral encoding. As mentioned multiple times so far, behavior can occur on many levels, e.g. time-scales. Classically, a linear regression model uses one type of behavioral variables as predictors for neuronal activity. However, an interesting approach is to use multiple types of behavior simultaneously as input to a linear regression model and measure the unique contribution to predictive power by each category, giving an idea of how strongly the data encodes different types of behavior. This approach has been applied in [MKJ+19] and referred to as a "Linear Encoding Model". Due to the nature of linear combinations, such a model allows for single neurons to encode different types (e.g. levels) of behavior in a multiplexed manner.

3.2.2. Nonlinear Models

When leaving the space of linearity, most models that predict activity without external input such as behavior, model time dynamics.
The simplest is arguably the Hidden Markov Model (HMM), that discretely models a latent state z at a given time point t via a transition matrix A (with $A_{ij} = \Pr[z_t = j | z_{t-1} = i]$). It then uses an emission model, commonly a probabilistic model with parameters θ, to infer an observed state from the latent state at each timepoint (via $\Pr[x_t | z_t = k] = p(x_t | \theta_k)$).
A more complex and continuous way to model observed activity from a latent timeseries is called Gaussian Process Factor Analysis (GPFA) ([YCS+09]). The idea is to model a latent timeseries via a Gaussian Process ([RW05]), and model the actually observed neuronal activity from this latent timeseries via Factor Analysis ([KW78]), a procedure

that focusses on shared covariance. This represents a mixture of linear modelling to predict individual neuron activity, and time dynamics to track the underlying population signal.
Nonlinear models do not necessarily incorporate time. Similar to the GPFA, the Rectified Latent Variable Model (RLVM) ([WB17]) models activity as linear combinations (including bias) of latent variables z, but it doesn't model z as a timeseries. It merely constrains them to be strictly positive, arguing that on a biological level, these latent factors represent internal drives and activations and in this context, negativity makes no sense. In that sense, this part of the RLVM is comparable to PCA, as it learns a set of latent variables that best predict the observed data. However, nonlinearity is added by modeling the observed activity y as $\hat{y} = f(Wz + b)$. The wrapping function f is designed to be able to capture (possibly non-linear) imaging properties. (In [WB17], the authors simply define $f(x) = x$ for calcium imaging, essentially working with linear special case of the model).

3.2.3. Discrete Modulation Models

All models in the previous category aimed to predict neuronal activity from an underlying internal signal. Although nonlinear, the nature of how this signal evolves or translates into measured activity remained constant over time. A step up over this approach is accomplished by Switching Linear Dynamical Systems (SLDS) [LNB+19], which are an extension of Linear Dynamical Systems (LDS). A LDS can be seen as the continuous version of an HMM. It models transitions in a continuous manner via linear transformations (with $x_t = Ax_{t-1} + \epsilon_t$, where ϵ_t depicts noise), effectively turning discrete switching states into a continuous state signal x_t. Analogous to a HMM, an observed state signal y_t of the LDS is obtained from that latent state signal ($y_t = Cx_t + \eta_t$, η_t being noise again. Note that x and y are continuous multidimensional features (vectors), rather than discrete states).
The idea of a Switching LDS now combines this with an actual HMM. Additional to x_t, a discrete state z_t evolves in parallel via a Markov Process as described above. An LDS is now coupled to this state by allocating each value a state z_t can assume to its own transformation matrix (A_{z_t}). The SLDS latent state therefore evolves depending on the discrete state ($x_t = A_{z_t}x_{t-1} + \epsilon_t$).

This mechanism is able to model state-dependency: According to a discrete state whose dynamic is modeled simultaneously, the way the system itself transforms is modulated, whereas this modulation itself is modeled in a discrete manner.

Models that include discrete modulation are used specifically in neuroscience in order to model neuronal activity by making use of various other quantifications measured during experiments. The Internal State Conditioned Linear Encoding Model by Akella et al. [ALS+25] and the Modulated Poisson Model by Goris et al. [GMS14] can serve as examples.

The former work first fits a HMM to observed Local Field Potentials obtained by neuropixel probes, identifying three discrete latent states: SH (synchronized high), SM (synchronized intermediate) and SL (synchronized low). Then, they train three separate linear regression models, one per state, to predict spiking activity, using stimulus, behavior and internal brain activity (same-area LFPs and different area population activity) as regressors. Everything is eventually combined into one model that predicts neural activity by identifying the latent state and then selecting the appropriate linear regression to fit activity.
This model captures modulation by discretely training three different sets of model weights to be applied under different internal circumstances in the brain.

Last but not least, the Modulated Poisson Model is an extension of a simple Poisson Model. Poisson Models are often used to model spiking activity, as they are appropriate in cases of modelling the rate of discrete events within certain time windows, such as action potential induced spikes in neuronal activity. The standard Poisson process for spike counts models the number of occurring spikes N per neuron in a given time window Δt with a Poisson distribution, based on a measured average rate. ($N \sim \text{Poisson}(\mu \Delta t)$. μ is the average rate, also often named λ).
In the modulated poisson model, the average rate μ is not estimated from measured spiking data, but factorized as $\mu = f(S)G$. $f(S)$ is the mean response ("tuning curve") of the modeled neuron with respect to the active stimulus and is passed to the model as input. G is the modulation term. It can be seen as a per-neuron modulation that changes discretely between trials. It is modeled by a Gamma distribution with a mean of 1 and variance fit on the observed data.
Accordingly, the model is able to probabilistically estimate the activity of a neuron from a stimulus, modulated by a discrete internal state.

3.2.4. Continuous Modulation Models

We finally arrive at the model announced in 3.1: The Hierarchy Model described in [FLR+25]. It is a bayesian model that nonlinearly combines linear regression from low-level behavior with a low-dimensional neuronal manifold encoding high-level behavior and therefore predicts neuronal activity from behavior in a multiplexed manner.
The motivation behind the model is the question of how encodings are modulated by the higher level brain state. Therefore, the model works in two steps:

1. Find a global neuronal manifold

2. Use low-level behavior to predict residual activity and the manifold to determine gating of such

A neuron's residual trace y_i is modeled by a StudentT distribution.

$$y_i = \text{StudentT}(\mu_{ij}, \sigma, \nu) \tag{3.1}$$

The distribution parameters are in turn probabilistically modeled using so-called "hyper-priors" that are allowed to differ across datasets j. While the full bayesian model definition, including the other parameters σ and ν, contains multiple probabilistic designs, we will focus only on the idea the model, that is, how μ_j is mathematically designed. We have:

$$\mu_j = B_j \cdot \sigma(M_j) \quad \text{with} \quad B_j = A_j E_j \quad \text{and} \quad M_j = \Gamma_j P_j \tag{3.2}$$

E_j denotes the multivariate timeseries of eigenworm projections (see 2.1.1) and P_j denotes the neuronal manifold. More precisely, it's the temporal PCA modes of the raw neuronal activity (see 3.2.1). A_j and Γ_j are weight matrices fit on the data.
The combined model predicts a neuronal residual trace as a linear combination of eigenworm projections, multiplied with a sigmoid of another linear combination of the neuronal manifold.

With the eigenworm projections as a quantification of instantaneous body posture, a residual trace is hereby modeled by a simple linear regression from behavior, with the exception that it is at every time point gated by multiplying it with a term between 0 and 1. This gating term arises from the neuronal manifold, normalized by the sigmoid function. Therefore, consistent with the observations in Figure 3.2, the model allows a multiplicative gating of these eigenworm-related oscillation predictions. Its mathematical structure also enables modelling general modulation, including up-modulation, by increasing the linear weights overall and using the gating term to reduce them back to the baseline during non-modulated periods.
Since the manifold itself is a continuous timeseries, this model allows a fully continuous modulation of the behavioral encoding.

In the next section, we will describe the architecture of a novel model that extends and refines this idea, together with an additional adaptation that makes use of behavioral subspaces. Together with initial exploratory analysis of the introduced datasets and the results obtained from applying the new model, the following section constitutes the main work of this thesis.
It aims to revisit the ideas of [FLR+25] by designing and implementing a novel unified continuous modulation model that is able to flexibly identify multiplicative hierarchical modulation organizations in neuronal activity. The goal is to accomplish a more generalizable and cross-organism methodology, confirm the previous results in *C. elegans* and establish a foundation for related future research regarding *Drosophila melanogaster* and mice.

4. Results

4.1. Model Architecture

In this section, we will motivate the designs of the a nonlinear gating model as well as a linear baseline model, using toy data that is based on real *C. elegans* observations and build towards the full architecture step by step.

4.1.1. Manifold-Concrete Model

The underlying assumption of the model is the ability to decompose each trace into three core components:

- The **Manifold** component, or "**M**".
- The Residual
  - The residual component encoding the low-level, "concrete" behavior. We call it the **Concrete** component, or "**C**".
  - The residual of the residual, or "**O**". It contains all **Other** information, such as e.g. unrelated sensory input or encoding of other specified behavior.

For conceptualization, we assembled a dataset of toy traces by manually generating the three components for each trace and adding them up. The concrete and manifold components were generated using eigenworm projection timeseries and reversal states of an actual *C. elegans* dataset (see methods). The decomposition of such a toy trace is visualized in Figure 4.1

The manual steps to decompose a trace this way consists of applying a linear dimensionality reduction method, e.g. PCA, to the full dataset, in order to define a manifold signal. This signal is used to reconstruct the globally shared manifold component of each trace and subtracting it leaves its residual. Then, these residual traces act as the targets of a linear regression from low-level behavior (in this case instantaneous body posture). The concrete component of a trace is defined as the prediction of its residual by this regression. The absolute error of this prediction, i.e. the raw difference between the concrete and residual component, constitutes the "other"-component.
This two-step procedure was utilized in [FLR$^+$25]. Here, we modify it such that everything happens simultaneously.

Now, the goal of the linear baseline model is to learn parameters that define such

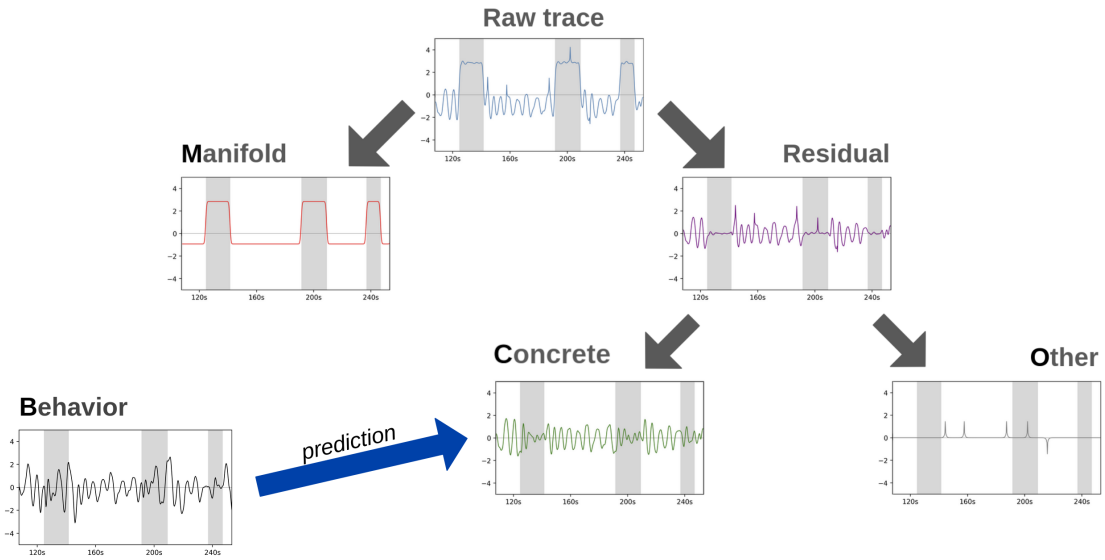


Figure 4.1.: **MCO-decomposition.** A raw trace (blue) is decomposed into a global shared part, the manifold (red), that potentially encodes high-level behavioral states and leaves the residual part (purple) when subtracted. The residual can in turn be decomposed again into a part (green) that encodes low-level, "concrete" behavior, as well as the rest of the trace that contains all other activity (gray). Periods during which reversals happen are colored in gray.

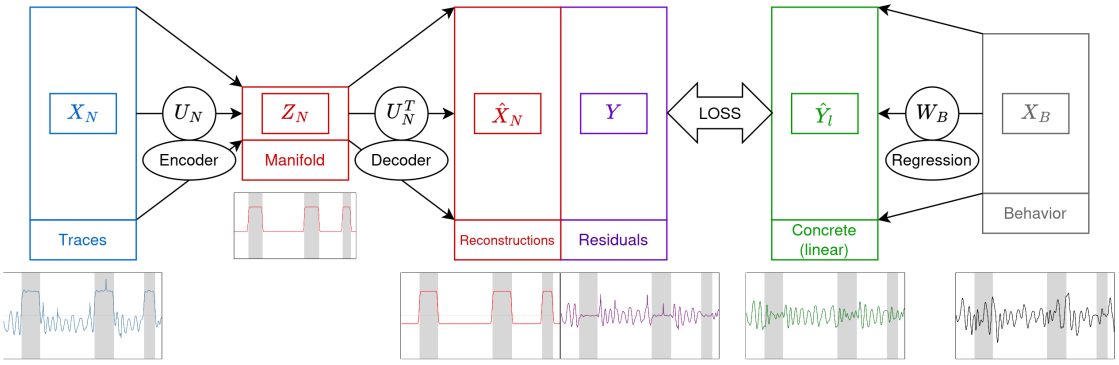


Figure 4.2.: **MCM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The model is solely fit on the error of the residual prediction.

a decomposition in one step. We call it the Manifold-Concrete model, as it learns and predicts the two components simultaneously. Its architecture is visualized in Figure 4.2.

Instead of PCA, the MCM uses a Linear Autoencoder U_N to learn the manifold signal Z_N from the raw traces X_N and reconstruct the manifold component for each trace. We constrain the Autoencoder to be orthogonal, defining the reconstruction (decoder) matrix as U_N^T, as for an orthogonal matrix A it holds that $A^{-1} = A^T$. We do this in order to enforce distinct manifold modes, analogous to the result of PCA. Subtracting the reconstructed manifold components $\hat{X}_N$ from the raw traces leaves the residuals Y. We predict these residuals from a set of low-level behavior quantifications X_B, via linear regression W_B, to come up with a set of linearly predicted residuals $\hat{Y}_l$.
The full model parameters consist of the two matrices U_N and W_B and are fitted solely on the Mean Absolute Error (see methods) between the residuals and the predicted residuals, optimizing the Autoencoder and the Regression parts simultaneously.

The idea is that by training only in the residual error, the model will push as much shared variance as possible into the manifold and as much variance that is predictable from the behavior as possible into the residuals, successfully predicting it from the regression. Only variance that is both not shared across neurons and not predictable from the behavior will cause an increase in the model loss.
Even though we technically do not train the Autoencoder on the reconstruction error, we still implicitly do. When passing random noise (or 0's) as behavior, the residual prediction will converge to a flat line at 0 (as we do not allow a bias), in which case the Mean Absolute Error is equivalent to the standard deviation of the residual traces. In such a case, the model will collapse to its Autoencoder part. Due to the orthogonalization constraint, this Autoencoder is itself equivalent to PCA (trained on the Mean Absolute Error instead of the Mean Squared Error). This means that the worse the neuronal input data encodes low-level behavior, the closer the learned manifold will be to (MAE-based) PCA.

We call this baseline model the Manifold-Concrete Model (MCM). It will find a linear decomposition of each trace such that their global components arise from the low-dimensional signal shared across neurons that leaves the best possible residuals with respect to linear predictability from low-level behavior, learning such a prediction simultaneously.

4.1.2. Hierarchical Gating Model

The main model of this thesis is based on the idea of modelling gating via multiplication, as described earlier in 3.2.4. It extends the MCO-decomposition by modelling a multiplicative interaction between the manifold and the concrete component (Figure 4.3). This nonlinear gating extension of the MCM is what we call the Hierarchical Gating Model (HGM). It uses the same linear autoencoder U_N to compute manifold and residuals. But instead of directly predicting the residuals merely via linear regression from behavior,

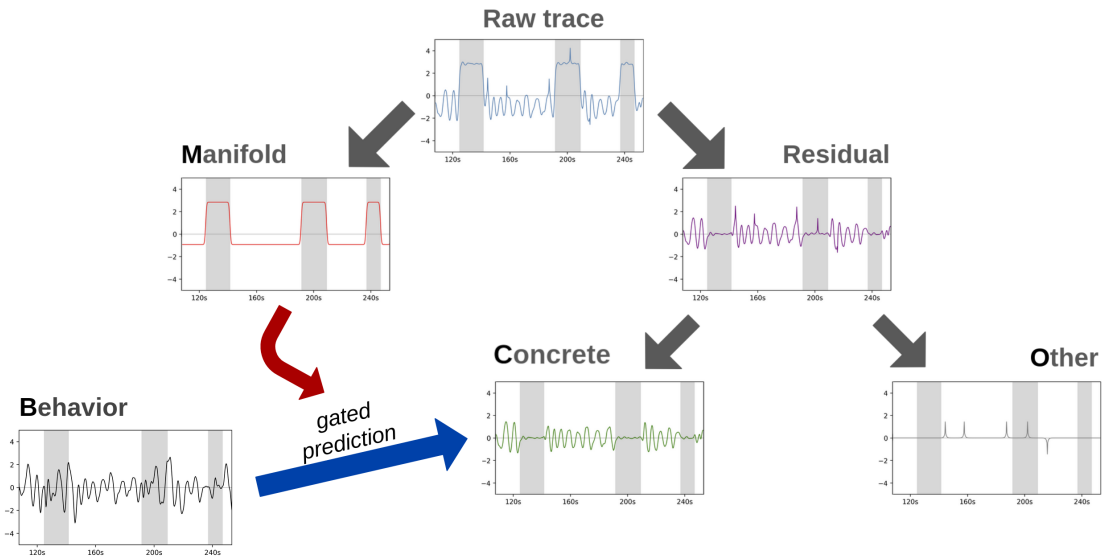


Figure 4.3.: **Hierarchical organization of behavioral encoding.** Based on the observations in 3.2, the amplitude of the concrete component is significantly reduced during periods of reversals (gray). In order to model this feature, the decomposition is combined with a nonlinear gating interaction between the manifold and concrete component.

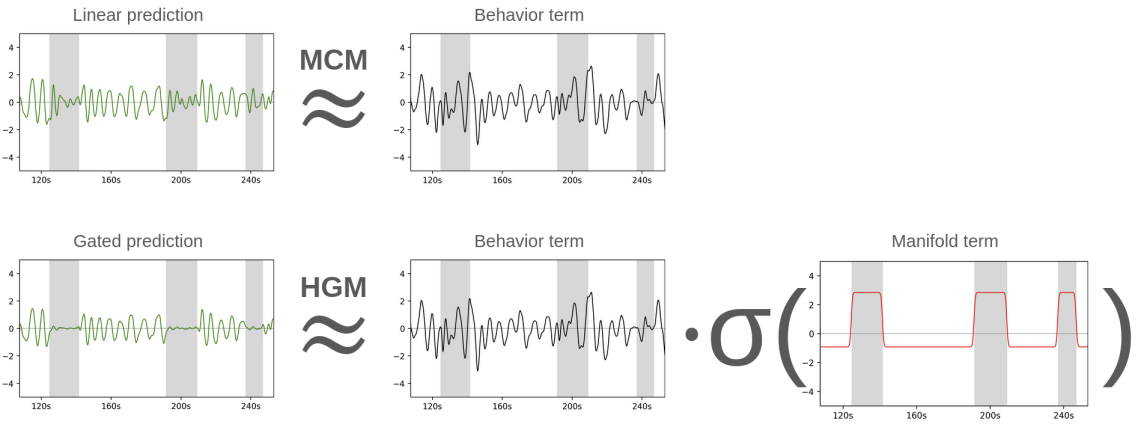


Figure 4.4.: **Gating via multiplication with the manifold.** The MCM (top) predicts the concrete component linearly from low-level behavior. The HGM (bottom) extends this prediction by multiplying it with a sigmoid of the manifold term. This is a visual depiction of the idea formulated in Equation 3.2 (we omitted the weight matrices in this figure).

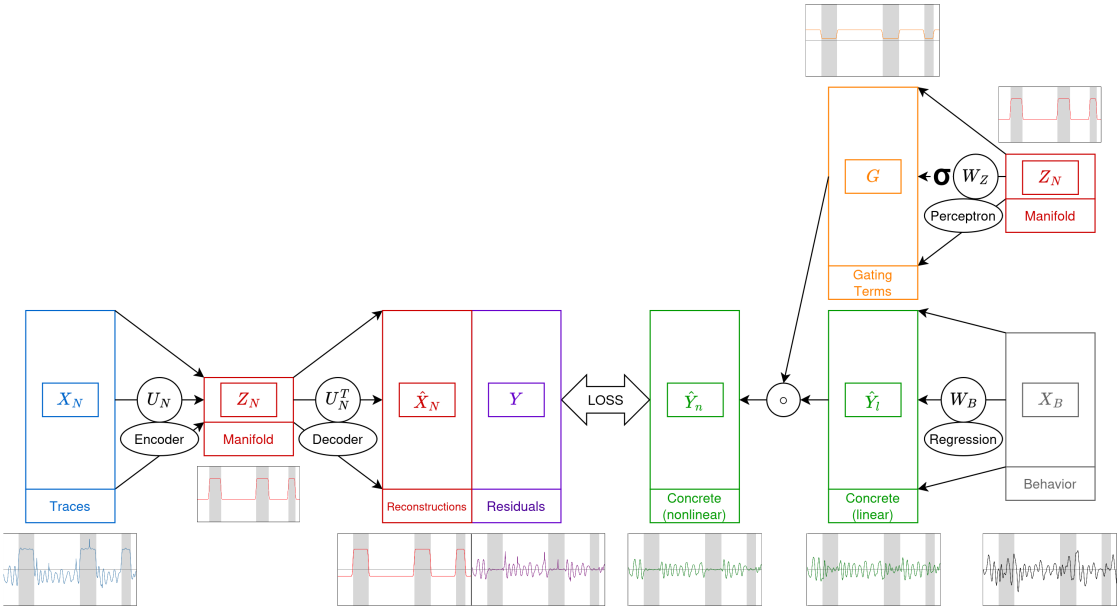


Figure 4.5.: **HGM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The circle symbol indicates element-wise multiplication. The model is solely fit on the error of the residual prediction.

we instead train an additional one-layer MLP with the manifold signal as input. The result of this MLP is multiplied element-wise with the result of the linear regression from behavior. This essentially means that we train two linear regressions simultaneously, one from behavior (W_B) and one from the manifold (W_Z), and apply the sigmoid activation function to the second. Then, we multiply both results element-wise (Figure 4.4). This part of the model design is mathematically similar to the structure of the Hierarchy Model in [FLR+25] as described in Equation 3.2.

Multiplying the residual prediction with the result of a sigmoid allows gating dynamics to take place. The MLP allows a more flexible connection between the manifold signal and the gating dynamics, e.g. by an inverse relationship, causing large manifold values to result in small multiplication factors and therefore high gating. Like this, the model is able to capture the feature of a global manifold signal gating the residual activity that encodes information such as low-level behavior.

The full model architecture is pictured in Figure 4.5. Its parameters comprise the three matrices U_N, W_B and W_Z.

The HGM will find a nonlinear decomposition of each trace such that their global components arise from the low-dimensional manifold signal shared across neurons that leaves the best possible residuals with respect to nonlinear predictability from low-level behavior modulated by that simultaneously learned manifold signal. Therefore, it well predicts traces which show such an multiplicative modulation interaction between the components. Although designed with the idea of gating in mind, mathematically, this

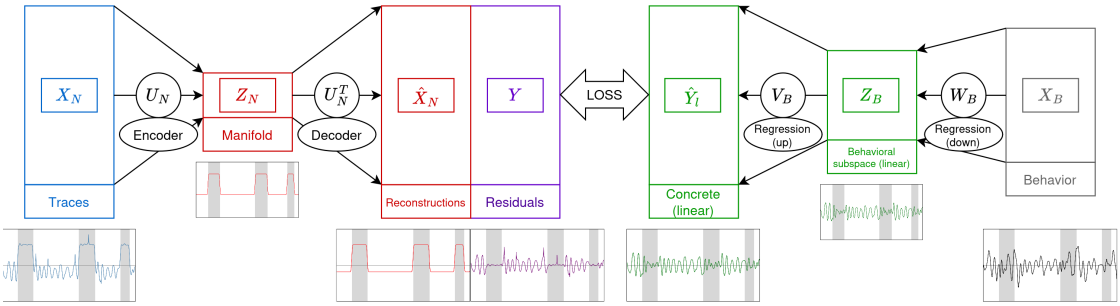


Figure 4.6.: **SS-MCM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The model is solely fit on the error of the residual prediction.

model can capture general modulation, including up- and down-modulation, similar to the Hierarchy Model in [FLR+25]. It is able to achieve up-modulation by a general upscaling of the regression weights and a compensating down-gating via the gating term to counteract this scaling during unmodulated periods.

4.1.3. Residual Subspace Extension

The residual subspace extension of the model is based on the idea that a modulated signal encoding the low-level behavior could also be low-dimensional and integrated across neurons. In order to model such a behavioral subspace Z_B, we replace the linear regression part of the model with reduced rank regression (Figure 4.6). In this extended model, W_B now refers to the reduction matrix of the RRR, projecting the behavioral input X_B into the lower dimensional behavioral subspace Z_B. The additional matrix V_B regresses back up to single-neuron space. This way, the residual prediction $\hat{Y}_l$ will be intrinsically low-dimensional.
The Subspace Manifold-Concrete Model (SS-MCM) parameters consist of three matrices in total: U_N, W_B and V_B. We modify the classical RRR by constraining W_B to be orthogonal. We do this in order to prevent instability in the subspace and also to ensure distinct behavioral modes, allowing for a clearer interpretation.

Finally, the Subspace Hierarchical Gating Model (SS-HGM) combines this concept with the modulated prediction from earlier Figure 4.7. At this point, main reason of this subspace-based model design shows: In the SS-HGM, we apply the gating term to the behavioral subspace instead of the single-neuron space. That means that the model finds a low-dimensional signal encoding low-level behavior gating. This signal is then integrated on a subspace-level and shared across neurons, by projecting the gated behavioral subspace up into single-neuron space via V_B.
We can furthermore trace the isolated gating signal through each vector space of the model by applying the respective matrix transformations to the gating term only, in order to get further insight into the data regarding the actual gating subspace (see 4.3.2

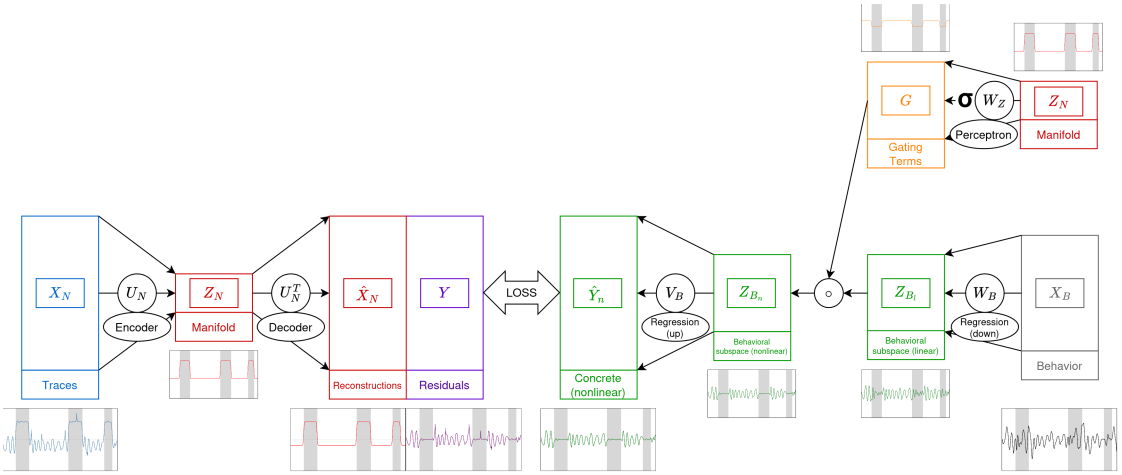


Figure 4.7.: **SS-HGM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The circle symbol indicates element-wise multiplication. The model is solely fit on the error of the residual prediction.

and 4.5.2). This final model's parameters comprise four matrices: U_N, W_B, W_Z and V_B. As before, we orthogonalize W_B to enforce distinct behavioral modes, each potentially modulated differently.

All model variations have in common that they require a multivariate timeseries of neuronal traces as well as behavioral quantifications in order to find neurons that encode this behavior (with varying degrees of complexity).

4.2. Literature Review

The goal of this thesis is applying the model with its different variations to real-world experimental datasets of various model organisms and analyse the results, possibly finding a modulated encoding of information related to the quantifications of behavior in the data. Therefore, the datasets were researched with the aim to be able to classify the quantified behavior as high- and low-level behavior. The low-level behavior will serve as the information we aim to find a (nonlinear) neuronal representation of, whereas the high-level states serve to gain further insight into the neuronal data by exploratory analysis and help with the interpretation of the results.
Suitable datasets need to fulfil the following requirements:

- Neuronal imaging timeseries, preferably using calcium imaging
- Timeseries of more granular information assumed to be encoded in the neuronal data, such as low-level behavior, e.g. locomotor activity
- Additional high-level state quantifications are desirable, but not necessary

We've conducted a research of such publicly available data in the context of this thesis and settled for the three datasets described in the preliminaries (2.1). The *C. elegans* data from [FLR$^+$25] has been acquired by our lab. The full compiled table of results with suitability assessment can be viewed in the appendix (Table A.1).

In the following sections, we will analyse these data with the decomposition introduced in Figure 4.1 in mind. We search for a global manifold signal and a possible connection to high-level behavioral states, as well as low-level behavioral encodings and possible modulation dynamics.

4.3. *C. elegans* Analysis

Since the encoding feature in focus has in parts already been found in [FLR$^+$25], the *C. elegans* data serves as a baseline to verify and tune the model, since it shows significant results. A strong advantage of *C. elegans* as a model organism is its canonical connectome and its stereotypic cell lineage, which allows for neuron identification across datasets. Therefore, we can focus on specific identified neurons of interested also considered in [FLR$^+$25]. That is:

- VB02 & DB01: These neurons have been shown to encode body undulation in the gated manner we aim to identify with the model.
- DD01: This neuron strongly encodes the body undulation, but does not seem to be gated.
- AVA, AVB: These neurons are known to drive high level motor commands like forward crawling or reversals.
- ALA, AWB: These are examples of neurons about which relatively little is known.

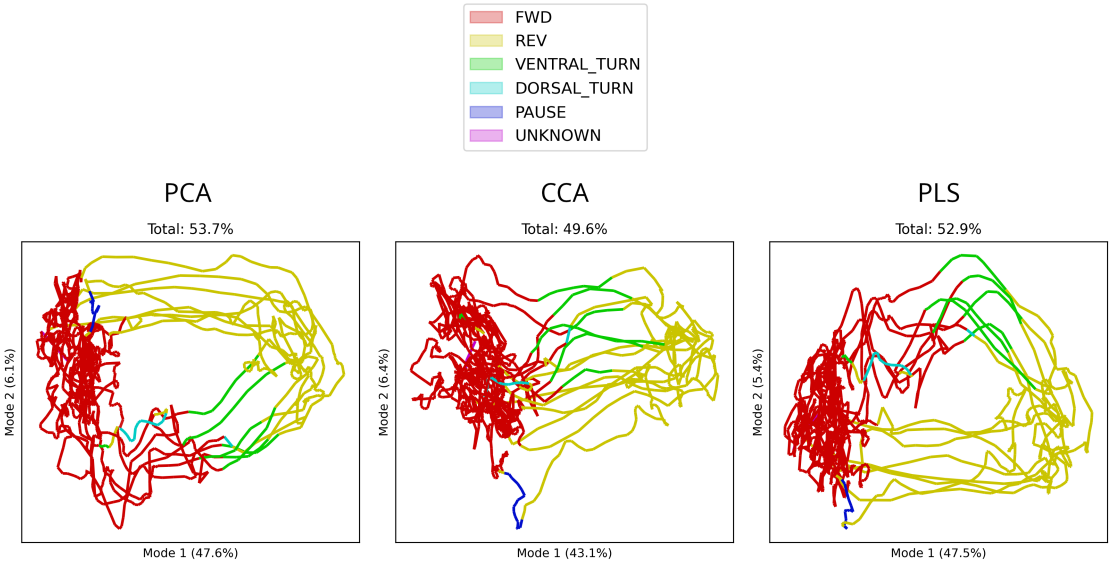


Figure 4.8.: ***C. elegans* manifolds.** The axis show the relative variance the respective mode explains. The total explained variance is given at the top. Latent dimension is set to 2. The plots are colored by the high level state quantifications to visualize their connection to the signal.

4.3.1. Exploratory Analysis

In order to get an estimate of how the different components are represented in the data before applying the full model, we start by manually performing dimensionality reduction on the neuronal traces to identify a globally shared widespread manifold signal. We apply PCA, CCA and PLS with 2 latent dimensions, the latter two with the discrete high-level states as targets, since the manifold is known to encode the global action sequence in *C. elegans* (Figure 4.8).
The PCA manifold neatly shows the first mode encoding the Forward-Reversal axis and the second mode encoding turns, revealing a circular structure. Even though CCA does not aim to maximize variance, its manifold still explains almost 50% of the total variance. PLS covers almost as much as PCA but additionally disentangles the pause state from the others, similar to CCA. Since PCA offers the best combination of simplicity and state encoding and is the easiest to interpret, it will be the focus of later analyses.
Sorting the neuronal traces by their PCA loadings, i.e. alignment with the first PC, reveals the manifold related structure in the neuronal heatmap (Figure 4.9). In line with previous results, the reversal driving neurons AVAL and AVAR appear at the top. The forward locomotion driving neuron AVBR appears at the very bottom, being inversely correlated to reversals.

In order to analyse activity beyond the manifold, we subtract the global component from each neuronal trace. The concrete component is interpreted as the component of the

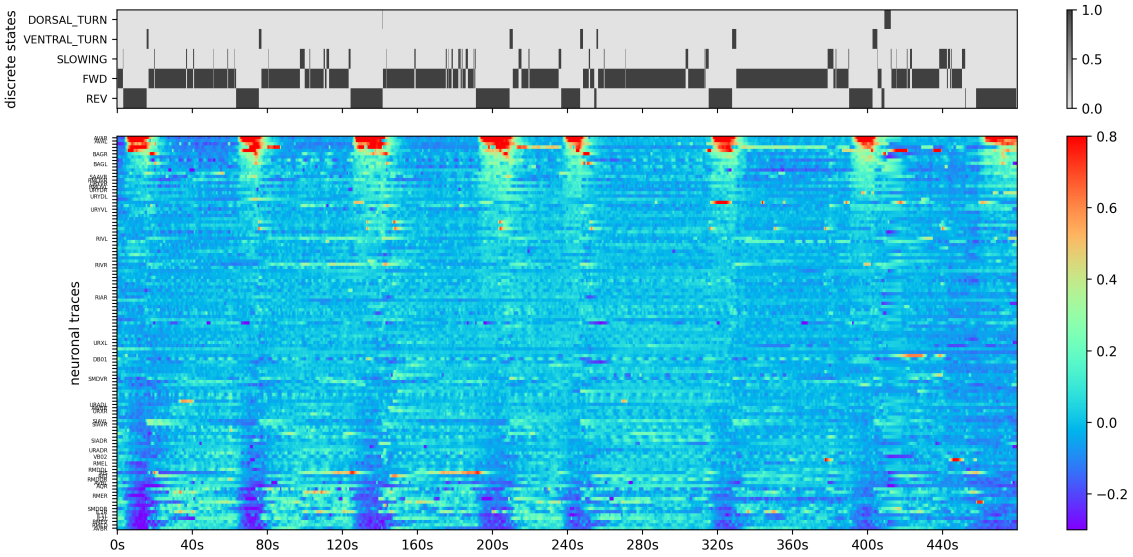


Figure 4.9.: ***C. elegans* trace heatmap.** Neurons are sorted by PCA loadings. Discrete states are sorted manually.

residuals that encodes low-level behavior quantifications, i.e. instantaneous body posture. Therefore, we aim to extract the part of the residual activity that shares spatiotemporal structure with the eigenworm projection timeseries, by applying CCA to the residual traces, using the eigenworm projection traces as targets. We set the dimensionality of the latent CCA space to be the same as the dimension of the behavior we use, i.e. the number of eigenworms. This means that we effectively allow CCA to find activity related to any eigenworm, without forcing it to search for some latent signal that encodes information of multiple eigenworms combined, as eigenworms are already a latent representation of behavior, making up fundamental building blocks.
Diverging from the model design by making use of CCA instead of linear regression allows us to sort the residual traces by their alignment with the general encoding of body posture, similar to as we did for the general manifold alignment with PCA earlier (Figure 4.10).
The residual activity reveals oscillatory dynamics on a similar timescale as the eigenworm projection timeseries. Eigenworm 1 and 2, representing simple body undulations during forward and backward crawling, show the strongest encoding overall. VB02 appears at the very top and DB01 at the very bottom, which means that they exhibit strong correlations to body undulation, inverted relative to each other, which is in line with Figure 3.2.

Further subtracting residual activity encoding body posture, i.e. the concrete activity, leaves activity that is neither connected to the manifold nor the behavior. This leftover activity can contain encodings of everything unrelated to the other two components.
This step-wise decomposition of the data is a good estimator for the M, C and O compon-

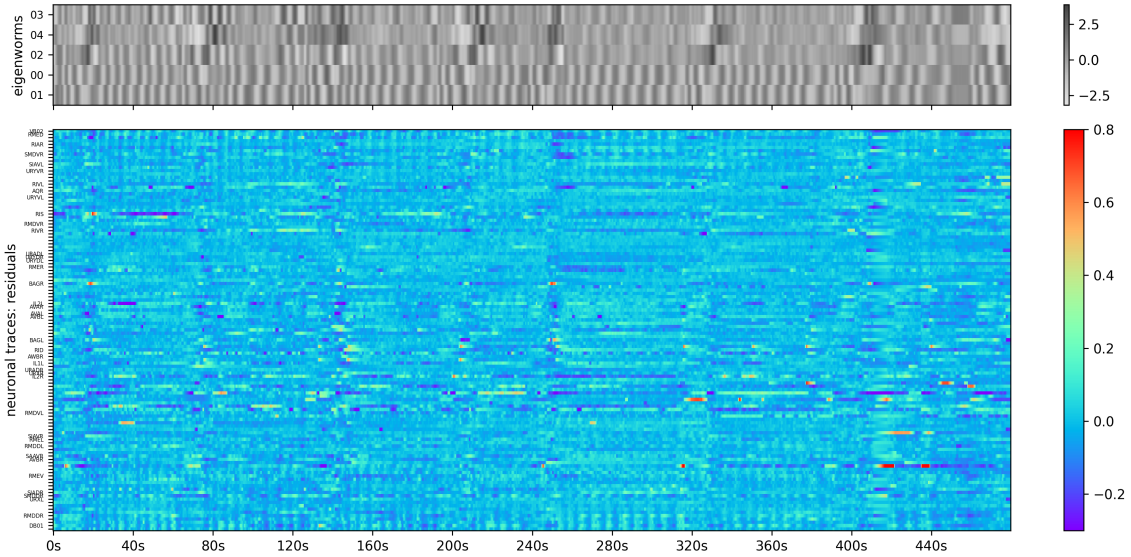


Figure 4.10.: ***C. elegans* residual heatmap.** Neurons are sorted by CCA loadings. Eigenworms are inversely sorted by CCA target loadings.

ents of the MCO-decomposition introduced earlier. We apply this analysis to all datasets and compute the relative variance explained by each of the three components across all identified neurons to get and estimate of what each known neurons activity pattern is composed of. Figure 4.11 gives an overview of this multiplexed encoding structure.
Overall, we see that the manifold related activity constitutes a large portion of total activity, with the unrelated activity coming in second. The concrete-component is rather small in comparison, but given the fact that CCA maximizes correlation and not explained variance, this is not surprising. Still, a few distinct neurons even show uniquely high values of relative variance explained. As expected, DD01 and VB02 consistently show up as body posture encoding neurons. The gated neuron DB01 is next in line, as well as multiple other motor neurons like SMDD and RMDD. AVAR and AVAL on the other hand shows almost no connection to the body posture and have most of their activity explained by the manifold. Lastly, the two neurons with the least explained variance by either component are ALA and AWBR & AWBL.

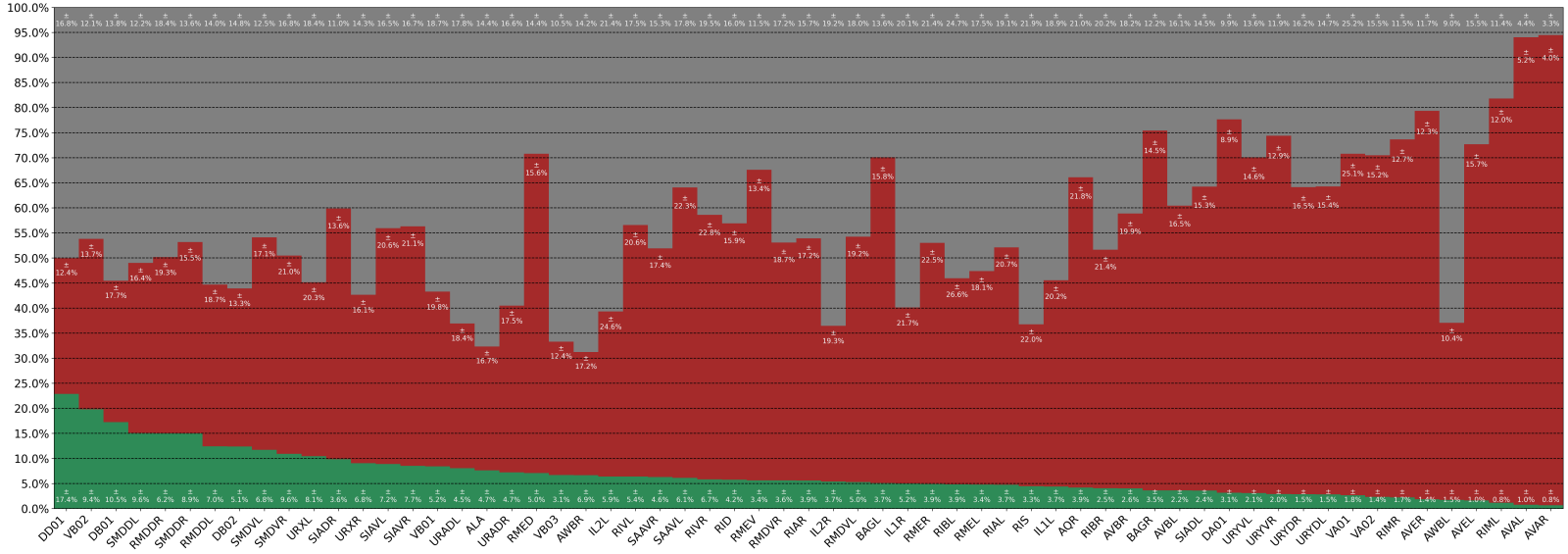


Figure 4.11.: ***C. elegans* MCO ratios.** x-axis shows neurons, y-axis shows the relative variance explained by each component of the given neuron, averaged across all datasets. M component in red, C in green, O in gray. Neurons are sorted by C. The per-neuron standard deviations of the relative variance explained for each component are overlaid on their bar segments.

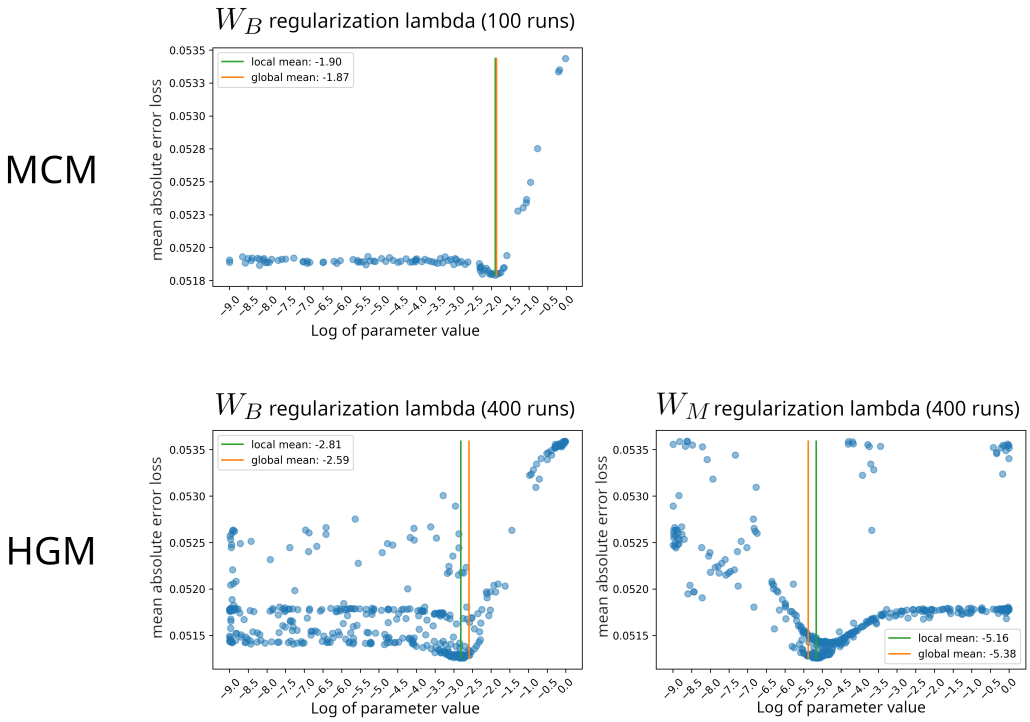


Figure 4.12.: **Hyperparameter sweeps.** The validation loss (y-axis) shows clear minima for all hyperparameters in both model types. The green lines indicate the mean parameter value of the 5% best runs for the depicted dataset (green) and its average across all swept datasets (orange).

4.3.2. Model Results

In *C. elegans* uniquely, there is the option to pool activity over all datasets into one timeseries for each cross-identified neuron, similar to [FLR+25]. However, in cases where neuron IDs are not available, this isn't possible. Therefore, in order to stay consistent in the methodology with the other organisms in mind, we train the models separately for each dataset. We analyse the results across datasets wherever possible.
But before training the models, we first run hyperparameter sweeps for all weight matrices that we regularize, that being W_B (or V_B for the subspace model version) and W_M. We sweep their regularization hyperparameters with respect to the unregularized residual validation loss. More details on regularization and sweeping strategy can be found in the methods. To stay consistent with the goal to design a model that is able to learn a decomposition for any dataset, we average the swept configurations across datasets to define one final hyperparameter configuration to use in in training for all datasets. The sweeps reveal configurations that result in clear validation loss minima in all cases (Figure 4.12). This indicates that the models were able to find the desired structure and generalize it to unseen data in all datasets.

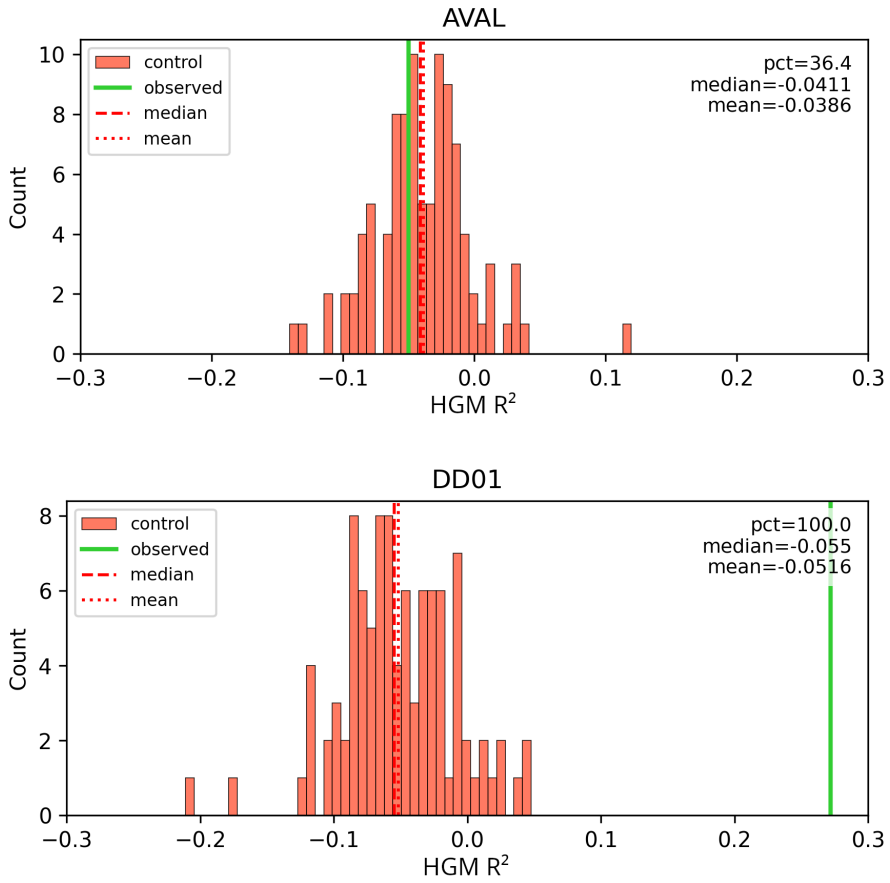


Figure 4.13.: **HGM R^2 control distribution examples.** The histograms show the distribution of control scores and the observed score in comparison for a significant (bottom) and a non-significant (top) example neuron.

We use the 5% best runs to obtain the hyperparameters for the final model designs and train them on all datasets. Additionally, we use the same hyperparameters to train 100 shuffles per dataset (see 2.2.6), building the by-chance distribution of model scores in order to assess significance. This additional control is necessary, specifically later in mice, as we deal with such a large number of neurons that we expect good R^2 scores to occasionally happen simply by chance, even with Cross-Validation.

Residual Prediction

With the models being solely trained on the residual prediction loss, the R^2 of this prediction is the main metric by which to evaluate model goodness. Whenever we henceforth

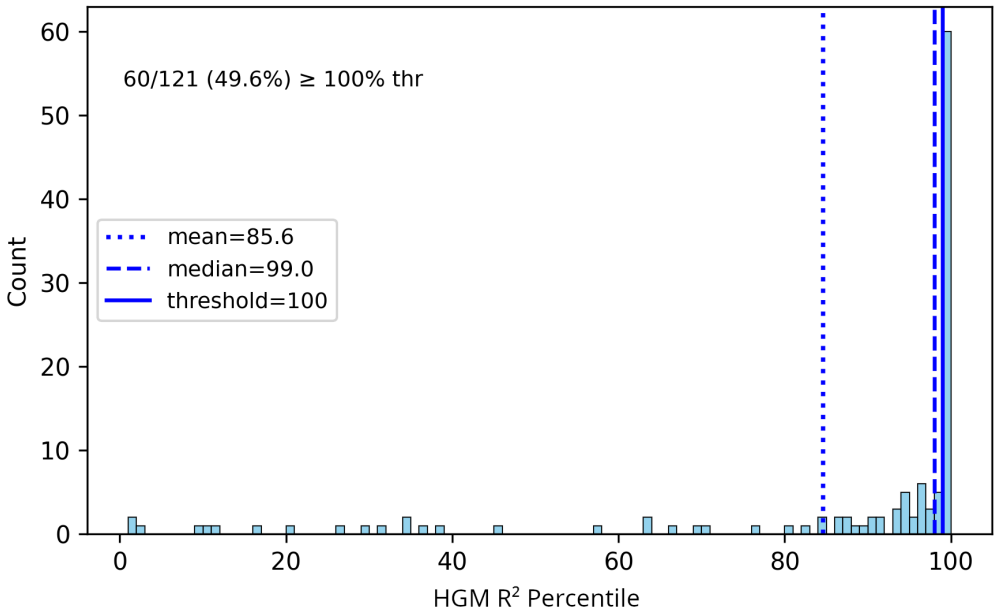


Figure 4.14.: **HGM R^2 control percentile distribution over all neurons.** The histogram shows the distribution of percentiles in which a neuron's HGM R^2 falls, with respect to its shuffle control distribution, over all neurons of dataset 19. In this dataset, 60 out of 121 neurons land in the 100th percentile.

mention the model R^2, we refer to the R^2 of the model's residual prediction.
Since we use multivariate regression to predict each target neuron, in order to assess significance of a prediction, we test the R_k^2 of a neuron k against the distribution of its shuffle control R_k^2's (see 2.2.6). We henceforth deem a neuron k significant if R_k^2 is greater 0 for both models and if it additionally lands in the 100th percentile of controls for the HGM, meaning for that neuron, the HGM performs better than for all its controls. Figure 4.13 shows two example histograms of such control distributions, for a significant and a non-significant neuron. AVAL has most of its variance explained by the manifold component and its small residual is not reliably predicted by body posture. DD01 on the other hand is mostly describable by the concrete component, which means it has residual activity that is well predicted by body posture.
This procedure assigns a percentile regarding its R_k^2 to each individual neuron k. Figure 4.14 shows the distribution of these percentiles over all neurons of a dataset. In the presented case, almost half of all neurons R^2's live in the 100th percentile of HGM control R^2's.
Across datasets, we find that the number of significant neurons average at 32.9%, with a maximum of 49.6% and a minimum of 10.7% (see Figure 4.15).

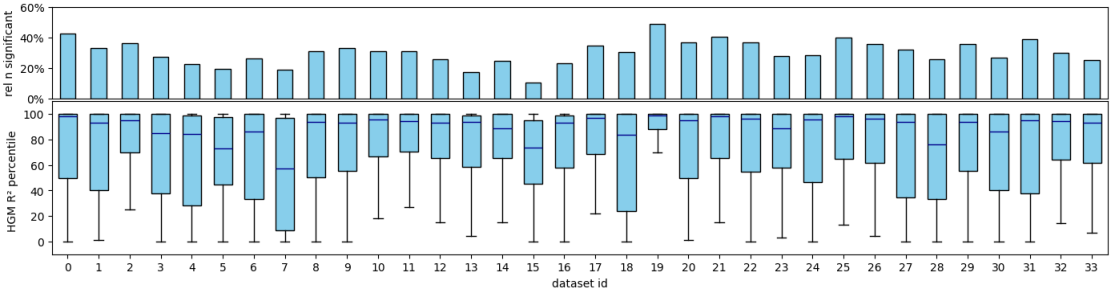


Figure 4.15.: **HGM R^2 control percentile distributions for all datasets.** The boxplots show the distribution of HGM R^2 control percentiles as in 4.14 for all datasets. The upper bars show the respective relative share of significant neurons.

To get an understanding of how the model decomposes real data, we examine the predicted components of the two neurons presented in Figure 4.13, AVAL and DD01. Figure 4.16 shows their manifold components as predicted by the HGM. Figure 4.17 shows how it predicts the residuals to come up with the concrete component. AVAL is a reversal driving neuron and shows activity strongly correlated with backward crawling. The manifold component captures this activity, leaving residuals close to 0. DD01 exhibits faster oscillations which are poorly explained by the manifold component and mostly land in the residual. The model is able to reliably predict these residual oscillations from body posture. In AVAL however, the residual does not appear to show such correlation, having the prediction resort almost to a flat line.

A significant HGM R^2 for a neuron confirms an encoding of instantaneous body posture in its residual activity. However, it does not contain a statement about the nature of modulation within this encoding. The residual of a neuron like DD01 can be equally well predicted by the MCM, in other words, by a simple non-modulated linear regression from body posture. Therefore, in order to identify neurons that actually benefit from the nonlinearity of the HGM, we next focus on the actual improvement that a neuron's HGM R^2 shows over its MCM R^2.
Figure 4.18 shows the two R^2's plotted against each other for each neuron. We find that in this dataset, the neurons which exhibit the highest score overall are consistent with the observations of neurons with high concrete component related variance in Figure 4.11. These neurons, namely VB02, DB01, both RMDDL/R and SMDDR are well predicted by both the MCM and HGM. SMDDL and DD01 are notably missing, but this is due to them not being identified in this specific dataset.
The mentioned neurons also show a rather large distance from the diagonal line, indicating a relevant improvement of prediction by the HGM over the MCM.
We plot the HGM and MCM R^2 distributions over significant neurons next to each other for all datasets in Figure 4.19 in order to get a qualitative estimate of general model improvement. We see that in general, the HGM R^2 distributions generally show a higher median and usually reach higher up. We also see considerable amounts of single neuron

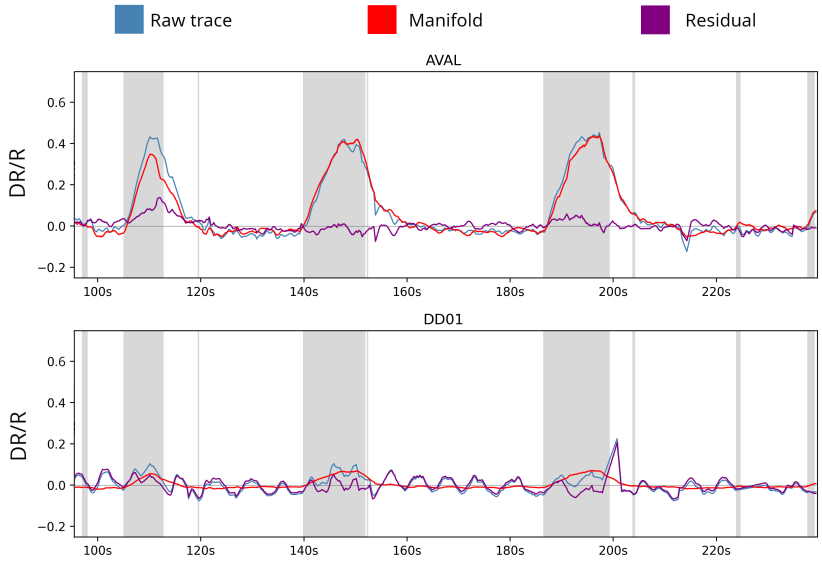


Figure 4.16.: **Single-neuron manifold prediction examples.** Excerpts of the neuronal traces of AVAL and DD01 are shown in blue. The respective manifold component as predicted by the HGM is overlaid in red, the resulting residual in purple. Gray bars indicate periods of reversals.

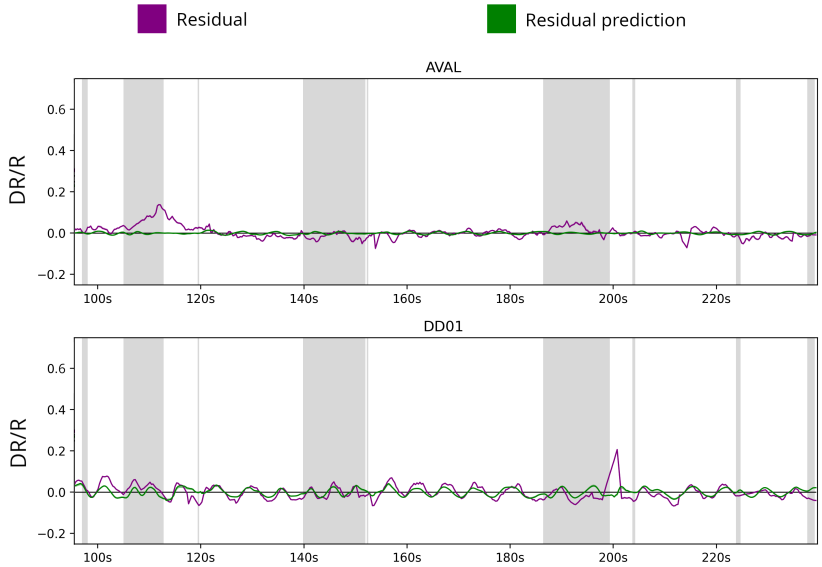


Figure 4.17.: **Single-neuron residual prediction examples.** Similar to Figure 4.16, the residual activity of AVAL and DD01 as predicted by the HGM is shown in purple. The respective predictions of such residual activity by the HGM are overlaid in green. Gray bars indicate periods of reversals.

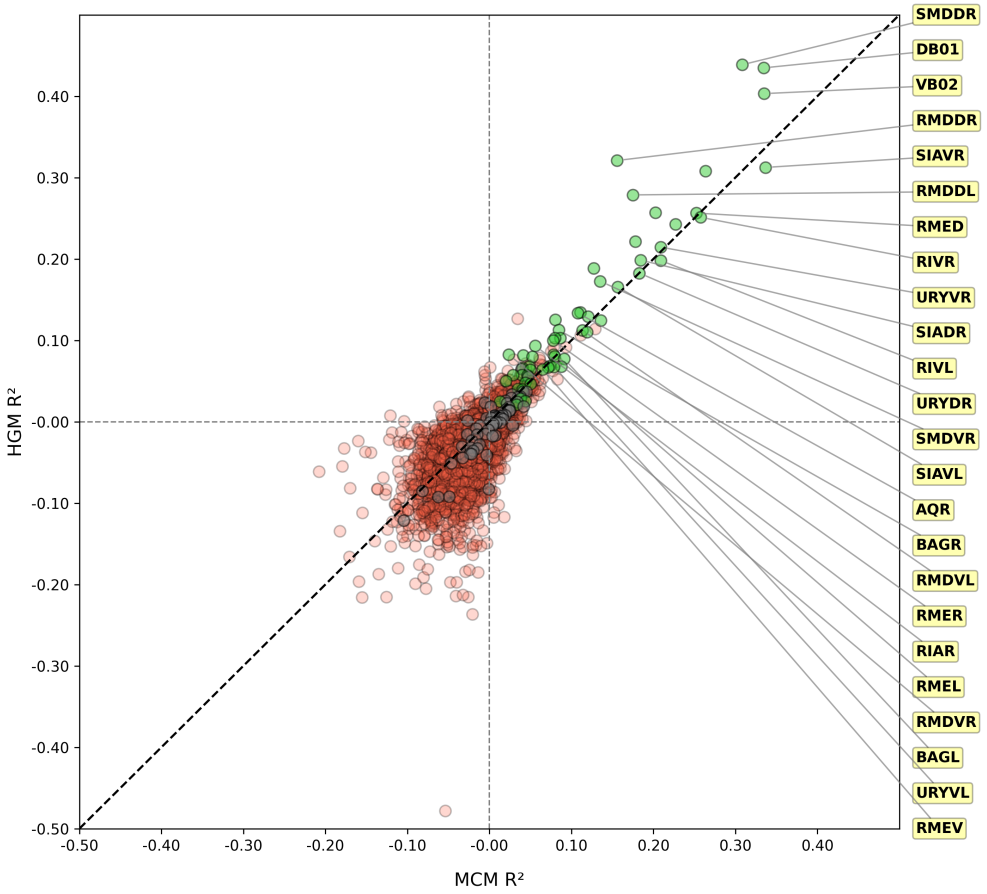


Figure 4.18.: **HGM vs. MCM R^2 for all neurons with controls.** The plot shows MCM and HGM R^2's for all neurons, including all shuffle controls (pooled) for dataset 19. Significant neurons are colored green, non significant neurons gray, controls red. Significant neurons which are identified are labeled.

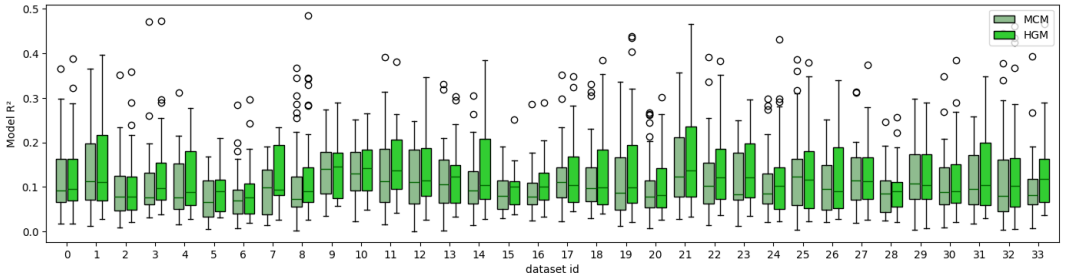


Figure 4.19.: **HGM and MCM R^2 distributions for all datasets**. Distributions only take significant neurons into account.

outliers in both cases.

To quantify this improvement, we use the shuffle controls again to compute the improvement distributions for each significant neuron and use them to calculate the respective sample p-value of the actual observed improvement for that neuron. We assemble the results for dataset 19 in a volcano plot in Figure 4.20. Neurons that have the smallest sample p-value out of all are defined to have significant improvement.
This way of further filtering significant neurons by additionally assessing the significance of their improvement leaves 8 neurons that show a significant improvement by their modulated prediction in dataset 19:
VB01, DB01, RMDDL/R, SMDDR, SMDVR, RMDVR, AQR.
Finally, Figure 4.21 shows the distribution of R^2 improvements for significant neurons with significant improvement across all datasets. We see improvements reaching up to almost 20% in some cases. The fact that these values are mostly reached by the whiskers of the box plots or single outliers while the IQR's usually stay smaller show that its mostly specific neurons that are improved, rather than large groups thereof.

Last but not least, in the special case of *C. elegans*, the possibility to identify neurons across worms enables us to additionally assess significance of identified neurons across datasets, using the medians of their score distributions. Furthermore, we use the additional GFP traces of the *C. elegans* data for control. Since the GFP traces do not capture neuron-related activity, we control by comparing a neuron's actual score with the distribution of all GFP scores, rather than only GFP scores of the same neuron. To be more precise, we compute the median of each identified neuron's HGM R^2 distribution. We then assemble the distribution of these medians. Then, we do the same for the GFP data. The two obtained distributions of medians are depicted in Figure 4.22.
Since we test all neurons against all other control neurons, this methods poses a more conservative control than the per-neuron shuffle assessment. It reveals a few distinct outliers that consistently pass the 100th percentile threshold:
DD01, VB02, DD01, RMDDR, SMDDL/R, SIADR.

As before, we further compute the distributions of their R^2 improvements. We plot them as boxplots against the medians of their MCM R^2 distributions in Figure 4.23 and additionally compute the 100% threshold of the corresponding GFP control R^2 improvement median distribution. The neurons that show a consistent significant improvement are:

- SMDDL
- DB01
- VB02
- RMDDR
- SMDDR
- SIADR

This improvement control reveals DD01 as a neuron with a consistently high R^2, but

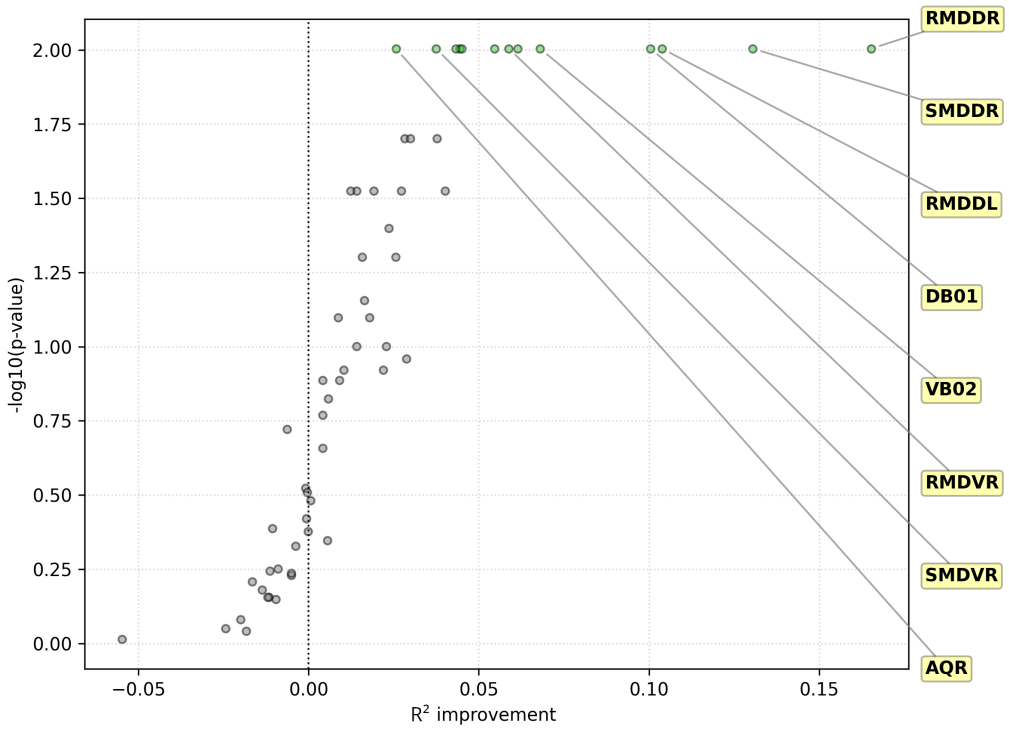


Figure 4.20.: R^2 **improvement against shuffle control R^2 improvement distribution sample p-value.** Showing dataset 19, and only significant neurons. Neurons whose improvement is significant are colored green, others gray. Neurons with significant improvement are labeled if they are identified.

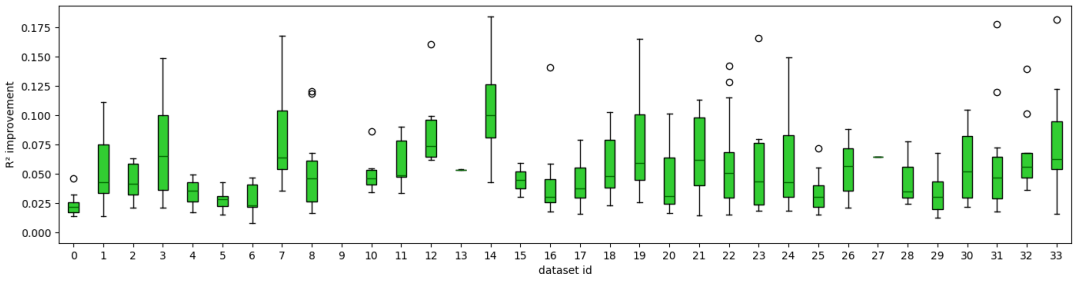


Figure 4.21.: R^2 **improvement distributions for all datasets**. The distributions only take significant neurons with significant improvement into account. (In dataset 9, all neurons happened to be filtered out by at least one of the two significance assessments.)

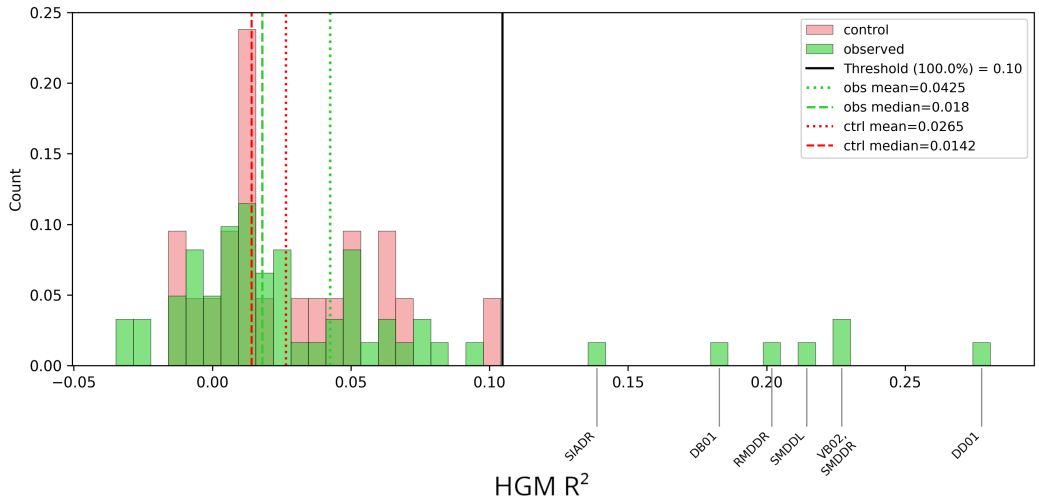


Figure 4.22.: **Distribution of HGM R^2 medians for identified neurons, with GFP control, across datasets.** GFP control distribution in red, GCaMP in green. The black line indicates the 100 percentile threshold of the control distribution. Neurons that pass this threshold are labeled.

without a consistent R^2 improvement.
This result means that body posture is encoded in the activity of these neurons in a modulated manner, with the manifold signal as the modulator. DD01 however encodes them linearly, without any influence from the manifold. We visualize this modulation, showing the model's manifold and residual prediction as well as the predicted modulation term for the neurons VB02 and DB01, in Figure 4.24 and Figure 4.25. Similar to DD01, DB01 has most of its activity in its residual. VB02 is an interesting case of a neuron which shows a mixed encoding of the manifold and concrete component related activity. The residual prediction terms visualize how the model implements the modulation in the concrete component. Both VB02 and DB01 exhibit body posture related oscillations. The predictions of these oscillations by the HGM and MCM coincide, except during periods of reversals. During such backward crawling, the oscillation amplitude of the concrete component as predicted HGM only is significantly reduced. The black modulation term shows how the manifold multiplicatively gates these oscillations.

In summary, the HGM is able to improve the prediction of a mere linear regression from body posture to residual activity by learning a manifold that multiplicatively modulates the residual prediction. In the cases of VB02 and DB01, it achieves this by a gating of the oscillatory activity.

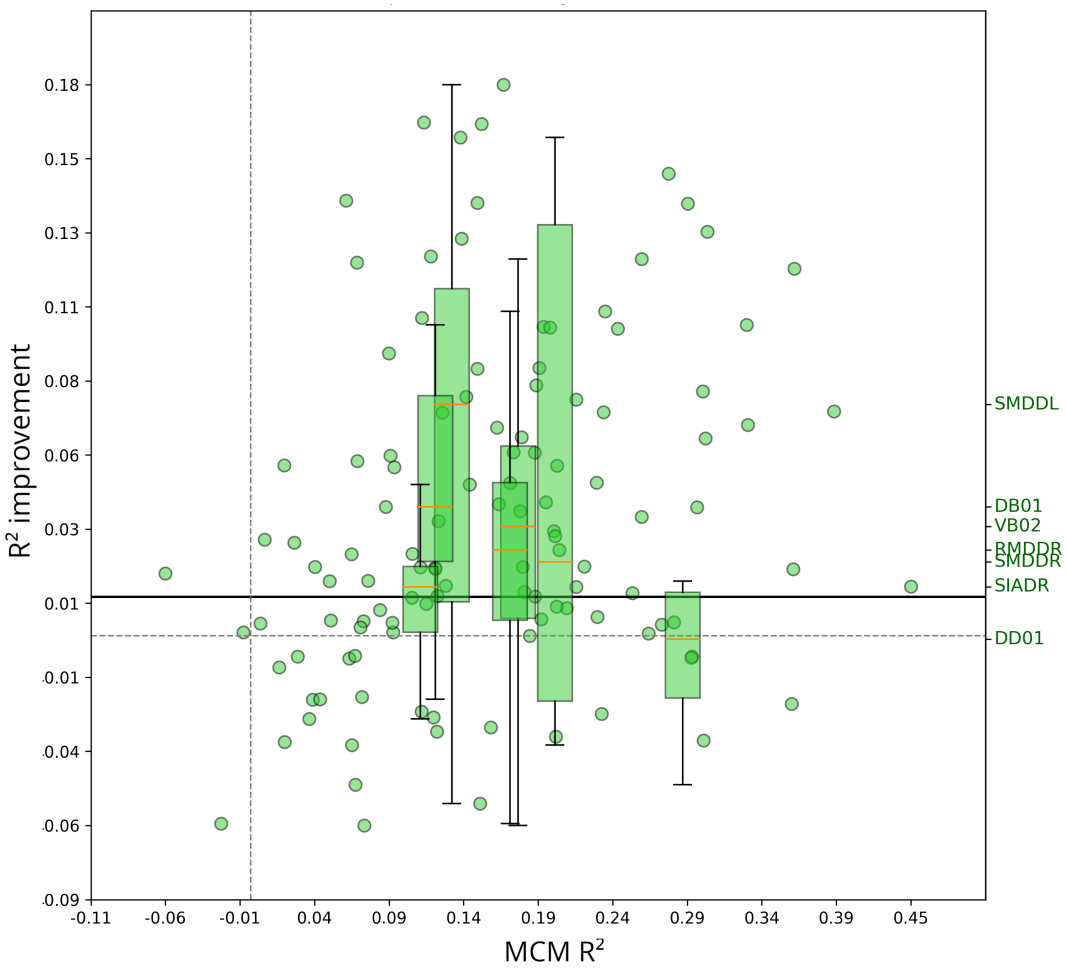


Figure 4.23.: **Distribution of HGM R^2 improvements for identified neurons, across datasets.** Showing only significant neurons with respect to GFP control. Boxplot are placed on the x-axis at the median of their corresponding MCM R^2 distribution. Individual neuron R^2's are indicated by dots. The black line indicates the 100th percentile threshold of distribution of GFP control improvement medians.

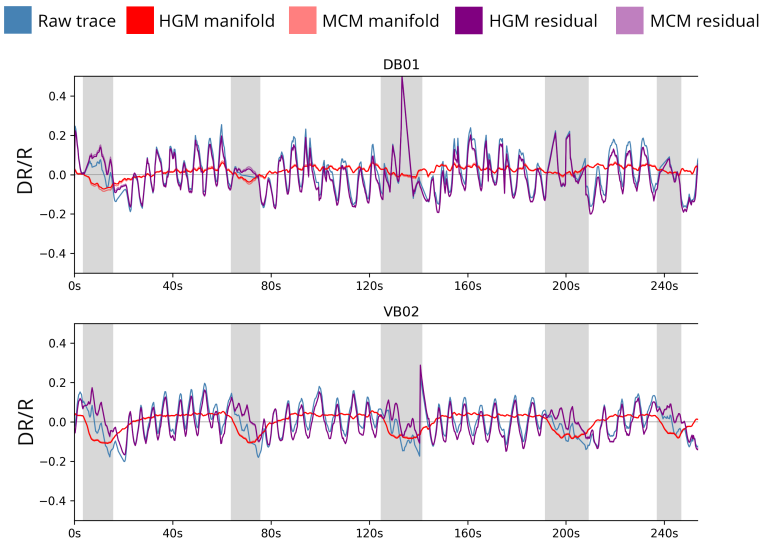


Figure 4.24.: **Manifold prediction examples for modulated neurons.** Excerpts of the neuronal traces of DB01 and VB02 are shown in blue. The respective manifold component as predicted by the HGM is overlaid in red, the resulting residual in purple. Transparent red/purple denote the respective traces as resulting from the MCM. Gray bars indicate periods of reversals.

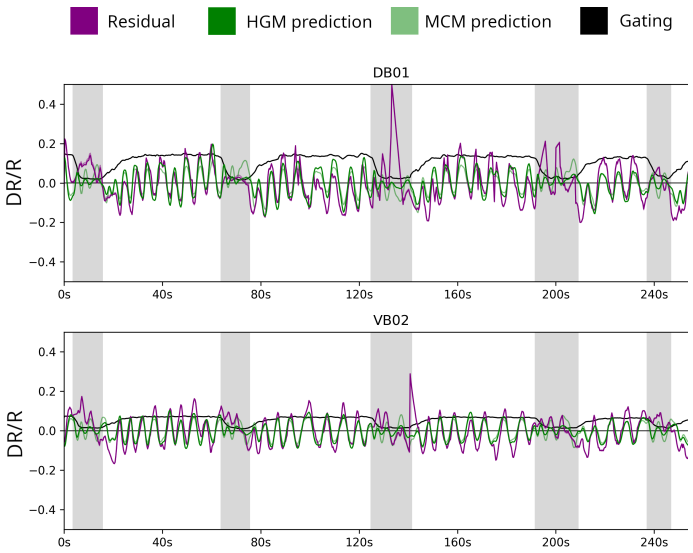


Figure 4.25.: **Residual prediction examples for modulated neurons.** Similar to Figure 4.24, the residual activity of DB01 and VB02 is shown in purple. The respective predictions of such residual activity by the HGM is overlaid in green, the respective gating term prediction in black. Transparent green denotes the residual prediction as resulting from the MCM. Gray bars indicate periods of reversals. A corresponding plot for the (unmodulated) neuron DD01 can be found in the appendix (Figure A.5).

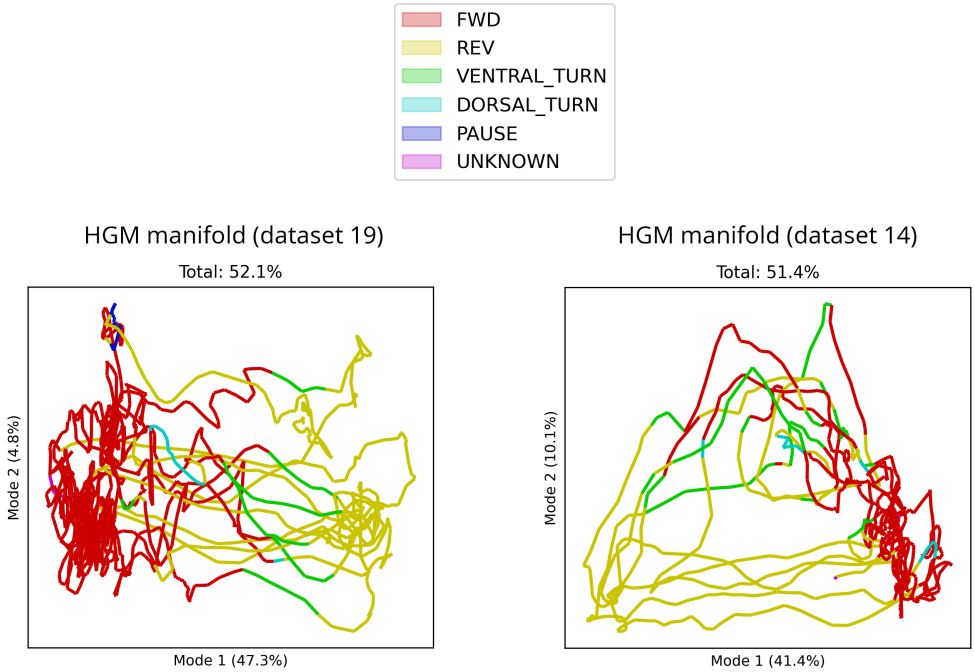


Figure 4.26.: **HGM manifolds.** Color coding and labelling is the same as in Figure 4.8. Latent dimension is also set to 2.

Manifolds

Apart from the multiplicative design of modulation for the nonlinear residual prediction, the second unique property of the HGM is that the low-dimensional manifold signal that determines this modulation is simultaneously learned. The model then reconstructs the neuronal traces from this signal, generating the global manifold component and in turn the residuals to predict downstream by learning gating terms based on that manifold. We introduced some results of this reconstruction briefly in the trace prediction examples in Figure 4.24 by showing the respective per-neuron manifold components.
In Figure 4.26, we depict this manifold as learned by the HGM for two datasets in a phase plot. In dataset 19 (also shown in Figure 4.8), the manifold now exhibits a clear separation at the Forward-Reversal axis and in dataset 14, the second mode additionally seems to capture Ventral Turns.
The manifold determines modulation of body posture encodings in the HGM by translating it into a one-dimensional gating term for each neuron. This means that each neuron can be gated in a unique way and the model is learning a manifold that contains all information necessary to build these gating dynamics, in other words, all active dimensions of gating. In Figure 4.27, we visualize gating for a few interesting neurons by coloring the manifold by the respective neuron's gating term, giving an idea how the manifold translates into the actual gating dynamics of a neuron.

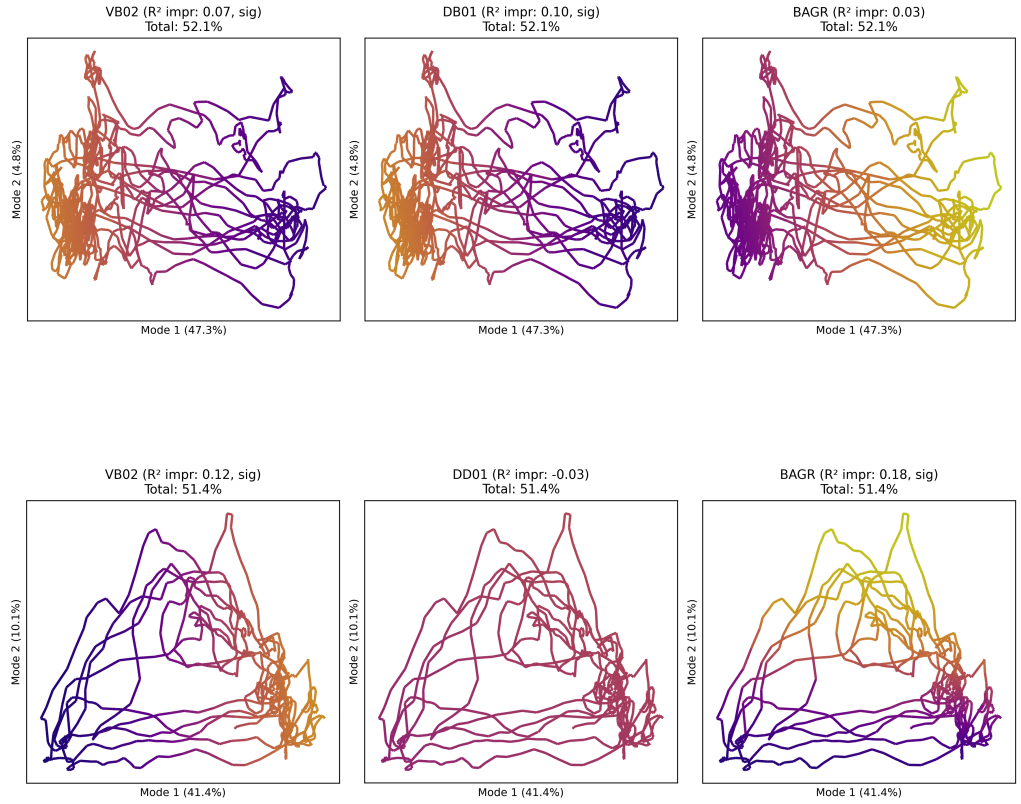


Figure 4.27.: **HGM manifolds by gating of different neurons.** Brighter colors indicate a larger sigmoid term and therefore less gating. Plots are titled by the neuron that is modulated in the depicted way. The respective improvement of their R^2 is given in the brackets, followed by 'sig' if the improvement was significant with respect to the shuffle controls. We show the same neurons for both datasets except for the middle column, where we picked DD01 instead of DB01 for dataset 14.

In dataset 19, gating happens exclusively along the Forward-Reversal axis that is encoded in manifold mode one. VB02 and DB01 are both gated down during reversals whereas the gating dynamic of BAGR seems to have an inverse relationship with them, but BAGR also did not show a significant improvement by the HGM. In dataset 14 however, it shows a strong improvement based on gating along the second mode that encodes Ventral Turns. Additionally, VB02 is still gated based on mode one in this dataset. This indicates that gating happens in two dimensions in dataset 14. Lastly, DD01 serves as an example where no relevant gating was found, in line with previous results.

In general, the axis of gating are strongly aligned with the manifold axis. Also, these manifolds as learned by the HGM are usually very similar to the result of PCA. The visible difference between the two in dataset 19 mainly arises from using the MAE rather than MSE (see methods). The fact that we continuously find a similar manifold with different methods like PCA, CCA and the HGM, suggests that in *C. elegans*, there is a low-dimensional signal that has multiple features:

- It explains a lot of the brain activity ([KKS+15] also indicates that it's widespread)
- It encodes the global action sequence
- It determines modulation of instantaneous body posture encodings

Subspaces

Lastly, we ask how the information that encodes body posture (in a modulated or linear manner) is distributed across the brain. Is there a low-dimensional signal that is spread across neurons and encodes body posture? Furthermore, does modulation happen on a single-neuron level, or does it apply to such a latent signal? To investigate these questions, we sweep and train the SS-HGM on the data. Similar to before, the sweeps find configurations that minimize the losses.
We set the dimensionality of the behavioral subspace to be the same as the number of eigenworms used for body posture quantification, allowing the model to learn a unique gating dynamic for each eigenworm projection timeseries, if necessary.
In order to get an idea of the general gating dynamics in play, we focus on the isolated gating signal as learned by the subspace model. By fitting PCA on the final per-neuron gating terms predicted by the model, we can obtain the gating subspace, with PCs sorted by gating activity. Furthermore, we can also project the actual per-neuron residuals and their predictions into this same gating subspace by reusing the same PCA. Figure 4.28 shows the resulting temporal modes for the gating, predicted residual and actual residual time-series of dataset 19.
Even though we allowed the model to learn a unique gating dynamic for each eigenworm, we see gating to happen almost exclusively along PC 1. Strikingly, this mode also appears to encode constant oscillatory activity with reduced amplitudes during reversals, suggesting it encoding the Forward-Reversal axis. In PC 2, very tiny changes in the gating term can still be spotted, sorting it in second place, as any further PC shows a

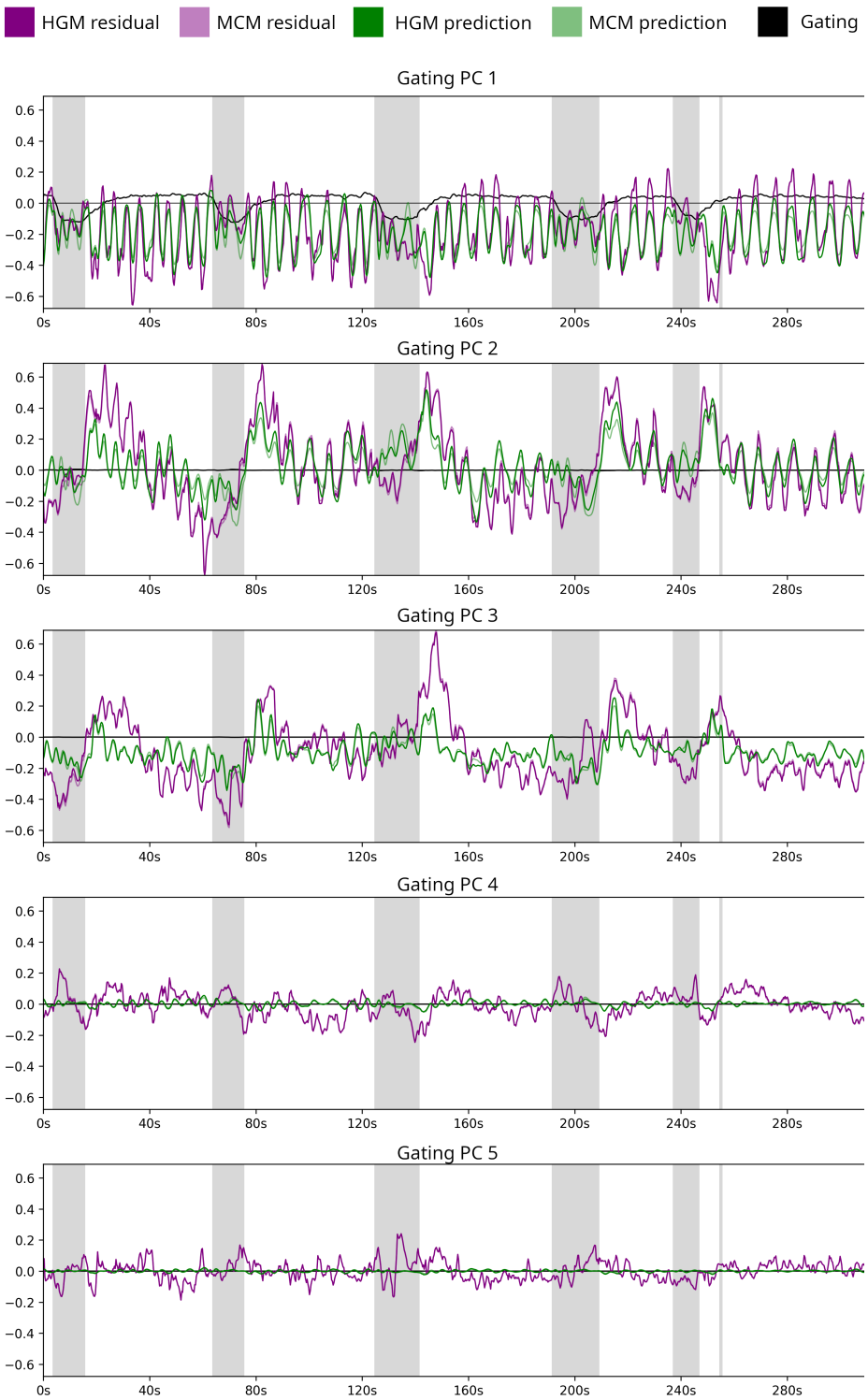


Figure 4.28.: **Gating subspace.** The black traces depict the modes of the gating subspace itself. Projections of the residual predictions in green, projections of the actual residuals in purple. Transparent lines denote the SS-MCM prediction, SS-HGM in opaque. As always, gray bars indicate periods of reversals. The oscillations appearing centered below 0 is merely a side effect of transforming data with PCA fitted on other data.

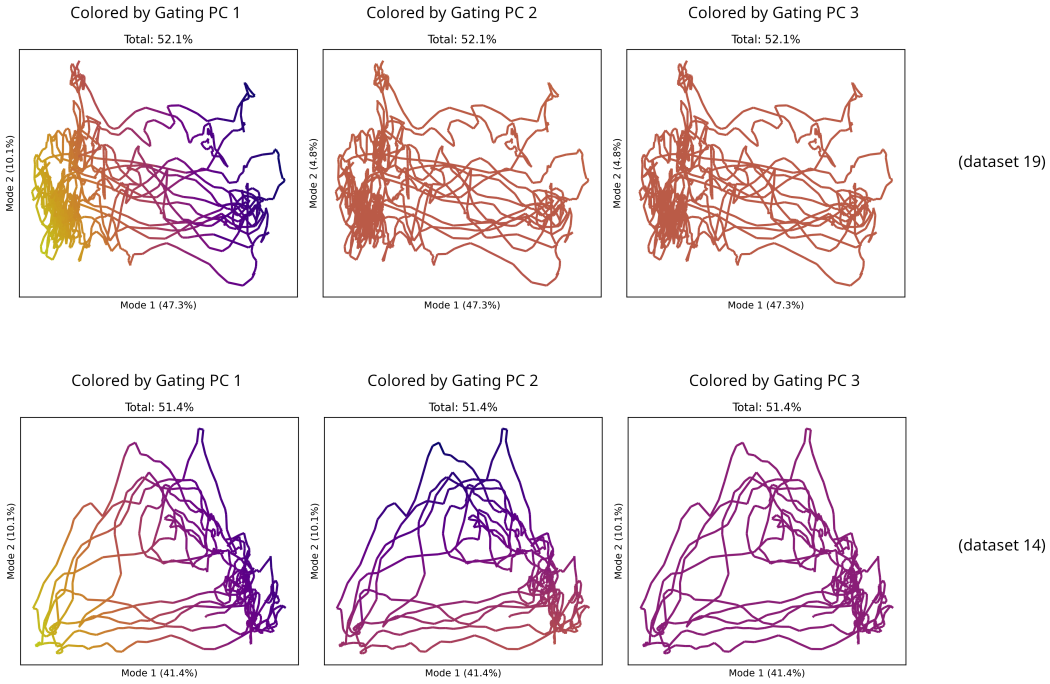


Figure 4.29.: **HGM manifolds by different gating modes.** Only the first three PC's are shown, as the others behave like PC 3, capturing no relevant gating activity.

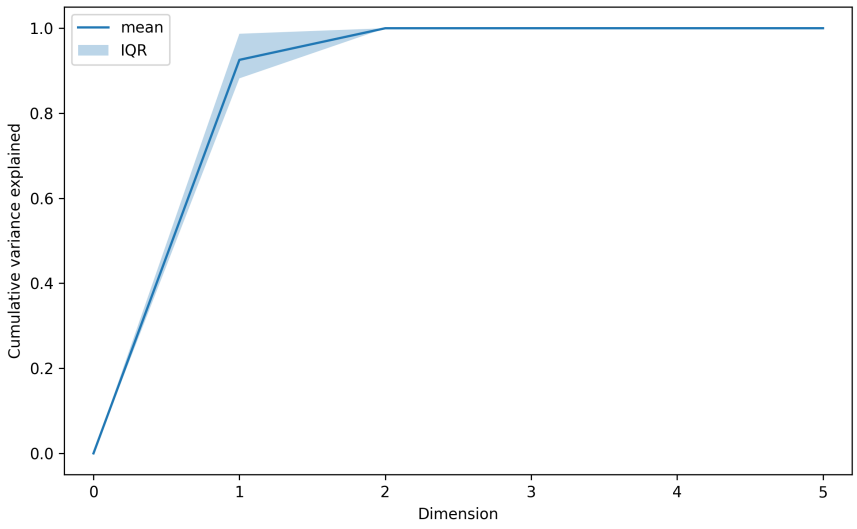


Figure 4.30.: **Gating dimensionality across datasets.** Average cumulative variance explained maxes out at 2.

clear 0-line. The second PC also exhibits repeating spikes in activity right after reversals, indicating an encoding of Turns.
This result confirms the gating dimensionality observed in the example neurons for dataset 19 Figure 4.27. For dataset 14, this same gating subspace analysis (accessible in the appendix, Figure A.10) also qualitatively confirms the additional Turn-related gating PC. By using the gating terms as in these gating PC's to color the learned manifold, similar to as done for single neurons in Figure 4.27, we can get a better intuition for the one-dimensional gating dynamic along to the Forward-Reversal axis in dataset 19 and the two-dimensional gating dynamic with an additional term along the Turning axis in dataset 14 (Figure 4.29).

Across datasets, we find that this intrinsic gating dimensionality does not exceed 2, with most gating activity living in the first PC (Figure 4.30). This result is not trivial, since while restricting the manifold to two dimensions, that latent gating signal arises from transforming the manifold in a nonlinear manner and therefore, linearly reducing it again does not necessary result in the same intrinsic dimension as the manifold. It suggests that the gating dynamics primarily depend on the manifold and not on the behavior whose encoding is modulated. Analogous analysis on the per-neuron gating time-series of the standard HGM reveals similar dimensionalities.

4.4. *Drosophila* Analysis

With the methodology clear, we present the results from analyzing the *Drosophila* data from [SMW+23] in this section. With 100.000 neurons, the brain of adult *Drosophila* poses a much bigger challenge. Furthermore, the lack of neuron ID's renders us unable to apply some of the cross-dataset analysis the way we did for *C. elegans*. It should also be kept in mind that the present data only covers the dorsal third of the central brain, possibly missing crucial pieces of information. However, with the literature of context-dependency in flies (Figure 1.3) in mind, this data is still of specific interest.

4.4.1. Exploratory Analysis

The initial qualitative look that the trace heatmaps shows a significant drift over time. In order to filter these dynamics that live in time-scales of recording lengths, we apply a high-pass filter to the traces of all datasets before further analysis. We will focus on the dataset with ID 1 for this analysis, as it showed the least drift. The raw heatmaps for all datasets with the filter can be viewed in the appendix (Figure A.11).

We start again by applying dimensionality reduction to the raw traces, as we did for *C. elegans*. Different to the worms, we do not see a circular structure in any of the cases. In dataset 1, PCA and CCA show a separation between states of running ("walk") and quiescence ("still"), with CCA also slightly separating Abdomen Moving. However, the variance explained from CCA is considerably smaller, suggesting a less clear encoding of high level states by a global manifold. This is further indicated by the phase plot of PLS appearing more strongly entangled. Other datasets show clearer separation in PCA of specifically the quiescent state, however, this separation only occurs in datasets where the fly also was quiescent most of the time during the recording. The relative shares of state durations and the PCA phase plots of the other datasets can be viewed in the appendix (Figure A.12, Figure A.13). In total, only six datasets show a distribution of states that is not dominated by quiescence, and out of those six, dataset 1 shows the best qualitative separation between states, with all the others appearing more entangled.
Whether there is a manifold encoding a global action sequence in *Drosophila* as well is an open question in neuroscience. Therefore, we run a quick analysis in order to find possible reliable state transitions across datasets. We calculate a state transition matrix showing the sample probabilities of transitioning from one state to another (leaving out reflexive transitions) and average the probabilities across all datasets (Figure 4.32).
The most prominent transition feature that emerges is the clean 0-probability to change from Back Grooming directly to front Grooming and vice-versa. Whether this is based on actual behavior or is merely a side-effect of uncertainty in the classification model that possibly classifies brief periods during such a behavioral transition as "Background" needs to be further investigated.
Apart from this feature, there are no significant systematics to be recognized in the matrix, except an almost 50% probability to transition from Abdomen Move to Running ("walk"), and from Quiescence ("still") to the Background state. These numbers are not

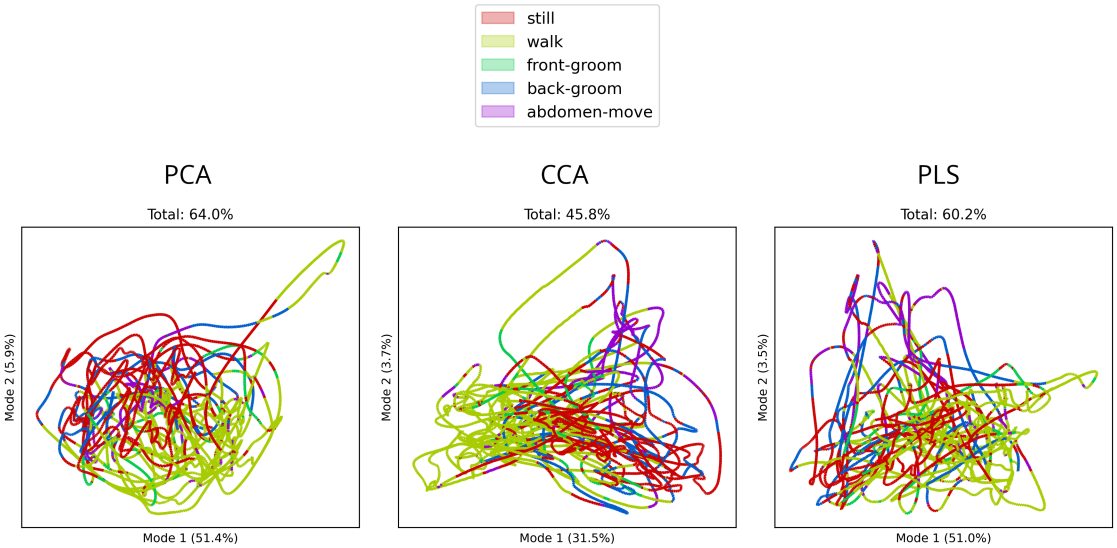


Figure 4.31.: ***Drosophila* manifolds.** Latent dimension is set to 4.

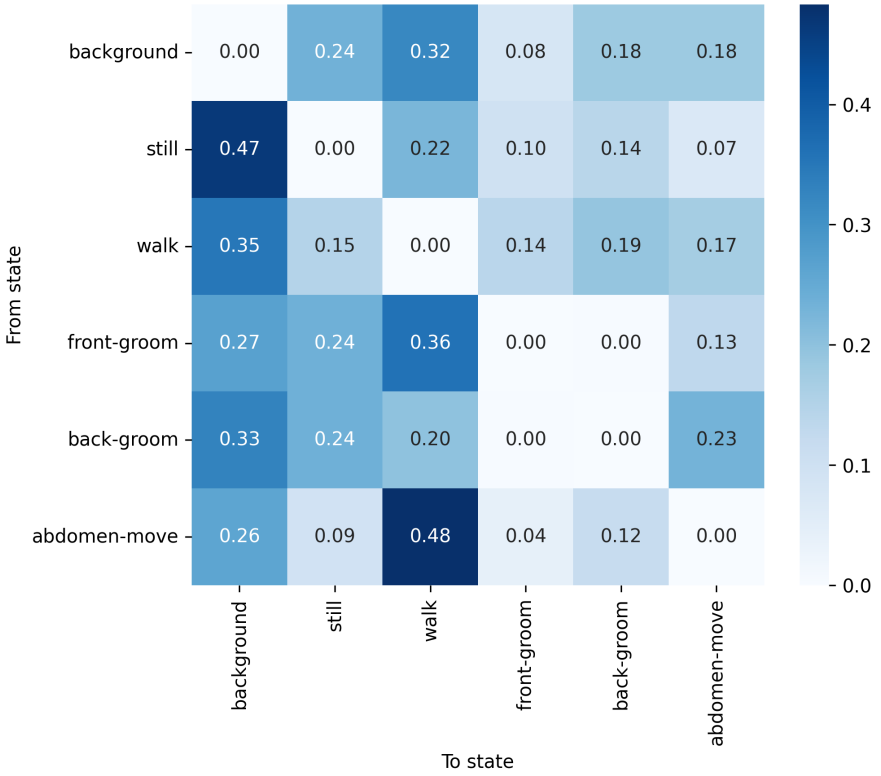


Figure 4.32.: **State transitions.** Transition probabilities are averaged across datasets. Reflexive transitions are excluded.

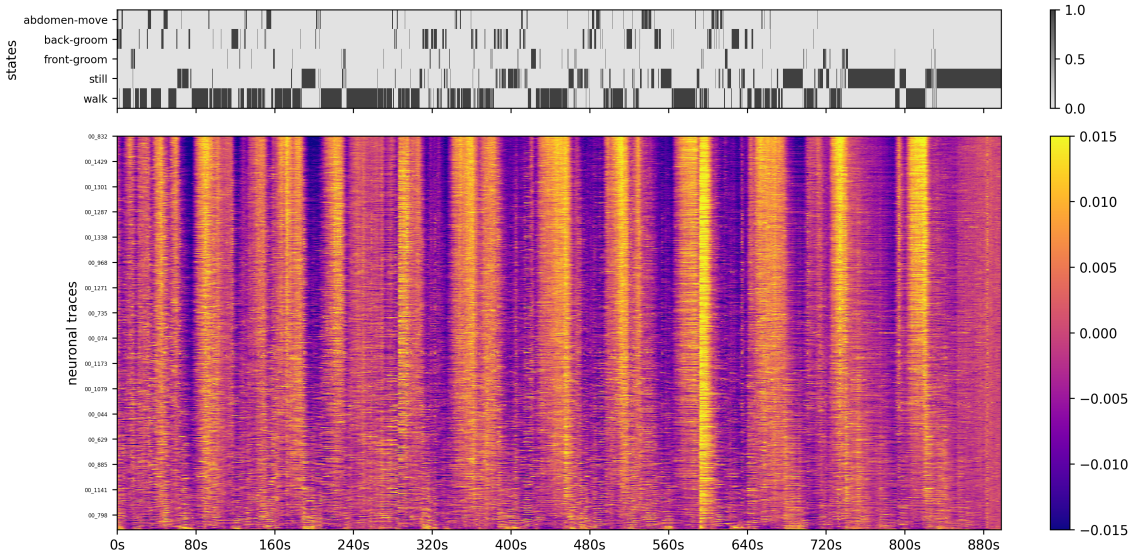


Figure 4.33.: ***Drosophila* trace heatmap.** Neurons are sorted by PCA loadings. Discrete states are sorted manually. 'still' refers to quiescence, 'walk' refers to running.

considerably high, and a transition into the background state holds no clear meaning, considering this state denoting model uncertainty.
Overall, a reliable global action sequence and its connection to manifold trajectories are much less apparent in flies, at least in the light of this brief analysis. We however did qualitatively find a connection between neuronal activity and the Running-Quiescence axis via PCA. This can also be further observed in the trace heatmap when sorting neurons by their PCA loadings (Figure 4.33). Groups of neurons with correlated activity emerge, similar to *C. elegans*, that seem to be connected to running. During quiescence, these groups appear largely silent.

The low-level behavior present for flies consists of a total 8 two-dimensional markers on the flies legs and abdomen. We apply CCA to the residual traces with the raw (z-scored) markers as targets, keeping the dimensionality the same as the markers and sort both groups by their respective CCA loadings to generate the residual heatmaps for the fly data in the same way that had revealed body posture related oscillations in *C. elegans* (Figure 4.34). The residuals contain much less visible structure and a connection to the marker traces is also not as visually evident.
Lastly, we use these results to compute the shares of variances explained by the M, C and O components for the data as well. Since there are no neuron ID's present, this can only be done per dataset. Figure 4.36 shows the resulting bar plot for dataset 1. The concrete component is much less represented compared to the result in worms. On average, only 2.7% of a neuron's variance is explained by its concrete component, with the maximum at 14.8%.

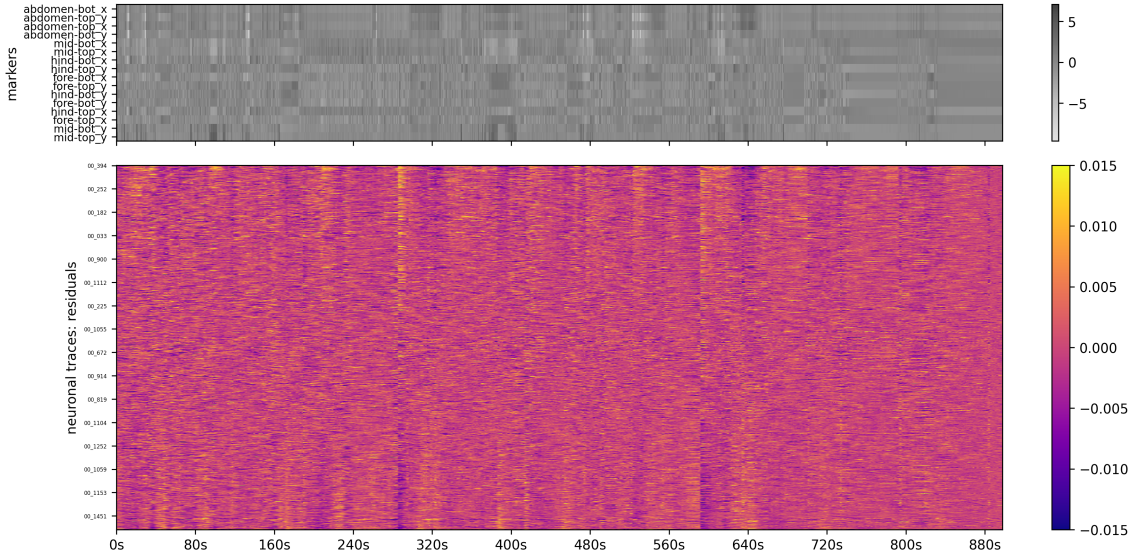


Figure 4.34.: ***Drosophila* residual heatmap.** Neurons are sorted by CCA loadings. Markers are inversely sorted by CCA target loadings.

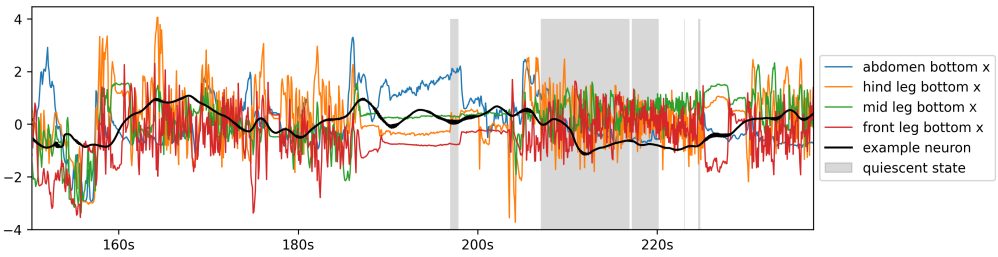


Figure 4.35.: **Marker vs. Neuron timescale.** The example neuron's time-scale is representative of the general neuronal residual dynamics.

Despite these low numbers, it remains possible that CCA found canonical variates of relevant shared correlations between neuronal activity and markers and that these variates do not happen to explain a lot of variance. Therefore, we additionally fit a direct linear regression model from markers to residual activity, explicitly maximizing the variance explained and keeping the methodology close the the actual MCM. The resulting exploratory per-neuron R^2's max out at 17.4%, with a full model R_2 of only 4%. For both these cases, the results were not promising. Based on the idea that neuronal activity likely encodes changes rather than absolute values, we also re-analyzed the data using marker gradients (see methods) instead of the absolute marker coordinates. However, this resulted in even weaker scores, with R^2's close to 0.

A qualitative look at the marker time-series reveals a difference in time-scale of marker

dynamics compared to the time-scale of neuronal residual dynamics (Figure 4.35), illustrating a likely reason for this lack of an identifiable relationship. Preprocessing the markers by using PCA with down to as little as 4 dimensions did not get rid of these fast dynamics (reduced trace accessible in the appendix, Figure A.14), indicating it is more than just noise.

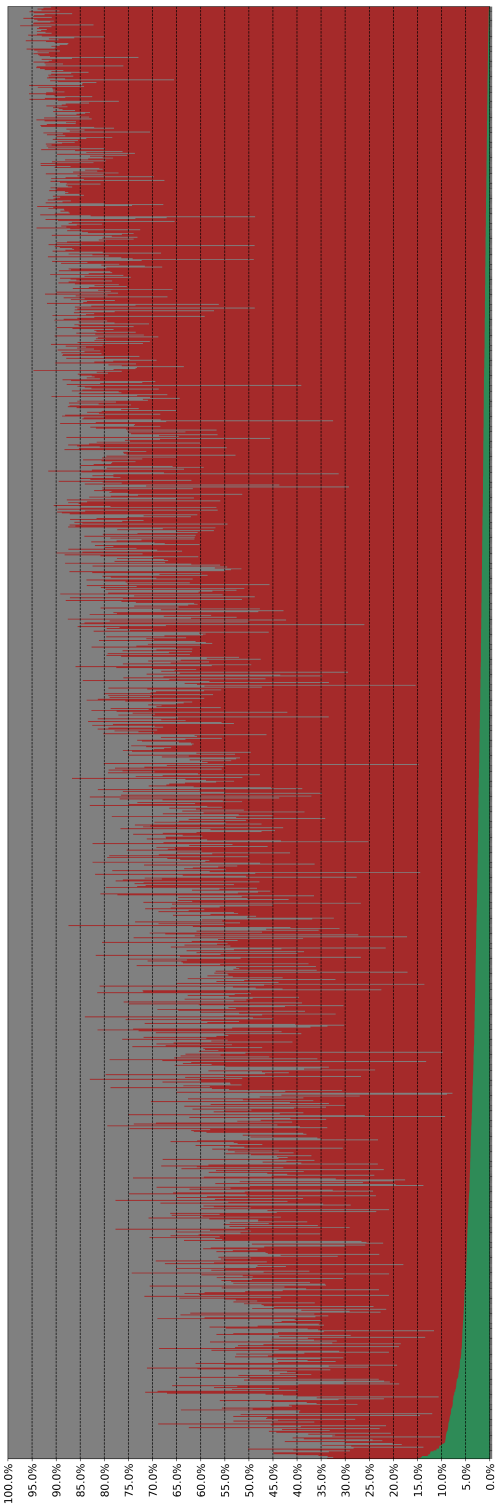
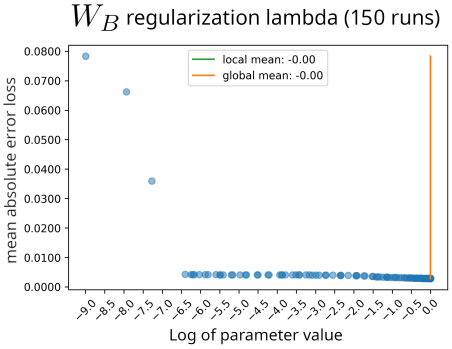


Figure 4.36.: ***Drosophila* MCO ratios.** Color coding is the same as in Figure 4.11. Plot only shows data for dataset 1, as missing neuron ID's do not allow cross-dataset mean and standard deviation analysis.

MCM



HGM

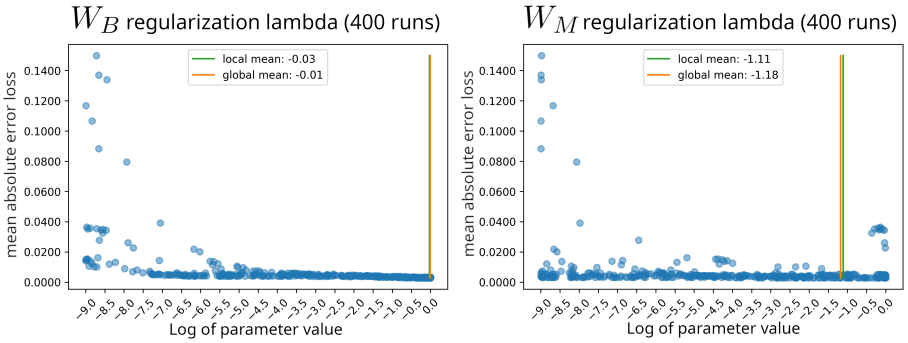


Figure 4.37.: **Hyperparameter sweeps for *Drosophila*.** Labeling is the same as in Figure 4.12. The validation loss (y-axis) is consistently minimized at values of 1 for W_B. This indicates that the model did not find the desired structure in the data and performs best by forcing the weights to be 0, simply predicting a flat line average. This also renders the result of the W_Z sweep rather meaningless.

4.4.2. Model Results

Since the model is based on predicting residual dynamics using low-level behavior, in this case the markers, the analytical linear regression from markers to residuals giving bad R^2's is already an indicator for a bad model performance. Indeed, sweeping for regularization parameters reveals the unfitness (Figure 4.37):
In both cases, the sweeps converge to the maximum allowed hyperparameter for the regression weights. This results in the model's regularization term to strongly punish any increase in weights, causing it to simply predict the residuals as a flat line at 0. The fact that the sweeps converge to this result means that this is the best way the model can consistently predict the validation set. In other words, no generalizable connection between markers and the residuals could be found. It also indicates that the earlier results from CCA and linear regression were mainly caused by overfitting. This problem was consistent across datasets.

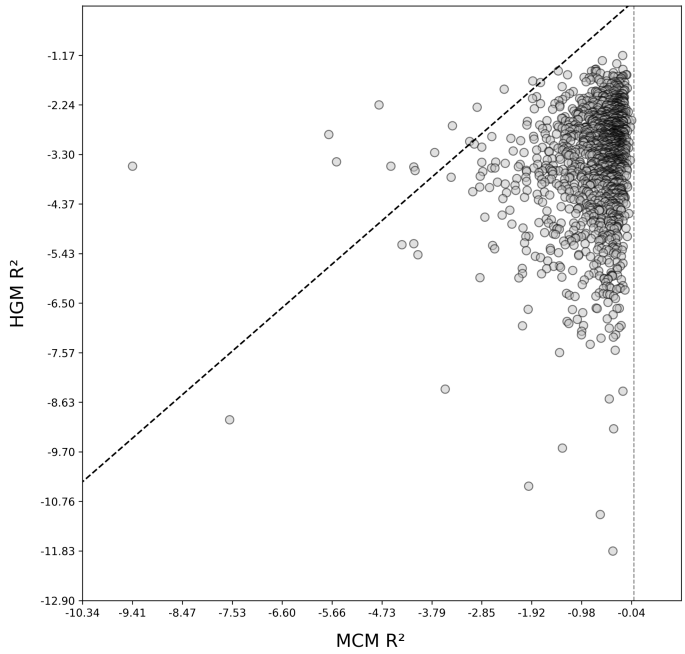


Figure 4.38.: ***Drosophila* HGM vs. MCM R^2.** Not a single neuron is significant.

In hopes of being able to increase the model score, we retrained it with additional marker time-series shifted forwards in time (see methods). This would allow a delayed relationship between activity and behavior to be identified by the model. Additionally, we also increased the dimensionality of the manifold. In no scenario we were able to obtain a relevant result. The hyperparameter sweeps shown in Figure 4.37 were obtained using these additional shifted marker traces and with a manifold dimensionality of 8. The subspace model versions (see appendix, Figure A.16) behaved similarly bad.
In order to get more insight into the problem, we manually chose a configuration of hyperparameters to train the model with nonetheless. Figure 4.38 shows the residual

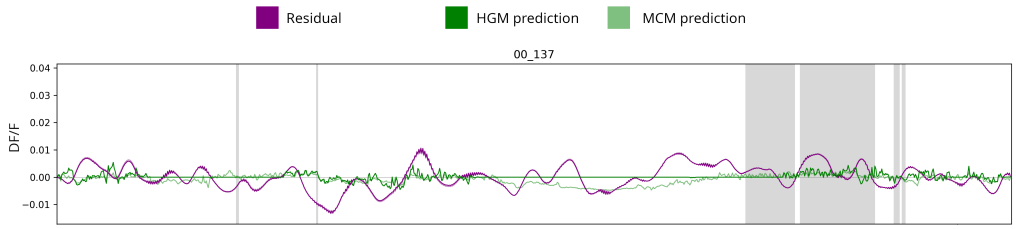


Figure 4.39.: **Residual prediction example.** As always, the residual activity is shown in purple. The residual prediction by the HGM is overlaid in green, MCM in transparent green. Gray bars indicate periods of quiescence.

R^2's (as done in Figure 4.18) for the resulting model. Not a single neuron happened to have a significant score. The more complex model giving worse scores is in line with the improper regularization from manually estimating parameters.
Lastly, we investigated the residual traces as predicted by this model. Figure 4.39 shows the best-fit neuronal residual in comparison to its prediction. Both models are not able to deal with the data, generating concrete components close to 0.
With the residual prediction not being able to capture variance, the manifold prediction does not hold a meaningful difference to PCA.

In total, we were not able to find an encoding of low-level behavior in the residual activity. The most immediate problem appears to be a difference in time-scales regarding the neuronal- and behavioral timeseries. It should be mentioned at this point that we had to contact the authors of [SMW+23] for access regarding the raw marker data, as it was merely used to classify high-level behavioral states and therefore not part of the publicly available dataset. The original acquisition rates were 8-12 Hertz for the calcium imaging and 70 FPS for the behavioral video, differing by a factor of $\sim$7 and the markers extracted from the video had to be downsampled to match the slower neuronal traces, possibly losing relevant information.
Furthermore, the calcium dynamics in the fly data also show a qualitative difference in time-scale compared to worms (see Figure 4.17 & Figure 4.35). In order to investigate this difference, the respective time constants of exponential fits to autocorrelation functions of the two types of calcium traces could be computed and compared, quantifying the effect. As of now, the reason for this discrepancy remains unclear (more on it in the discussion) and it is possible that further processing or different quantifications of behavior would result in a better model performance.

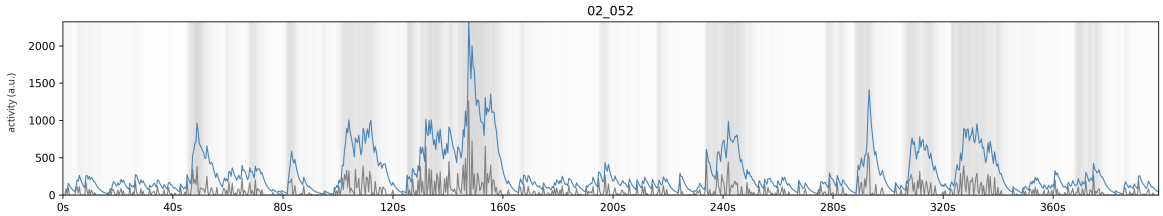


Figure 4.40.: **Exponential filter to "reconvolve" spikes.** The gray trace is the GCaMP data as deconvolved in [SPS+19]. Blue is the result of the exponential filter, transforming it back into continuous calcium dynamics. The continuous gray bars indicate pupil area.

4.5. Mouse Analysis

The third organism we chose for analysis is mice. In [SPS+19], work has been done showing a connection between the primary visual cortex (V1) activity and high-level arousal variables. Therefore, this data does not contain discrete high-level behavioral states but instead the three continuous global arousal measures of Running Speed, Pupil Diameter and Whisking Motion Energy. Furthermore, the GCaMP traces are already preprocessed to represent the underlying spiking activity typical for all-or-none responses of mammalian neurons.
In the following analysis, we will focus on dataset 2, as is it the one that is used in most of the plots of [SPS+19], allowing proper comparison.

4.5.1. Exploratory Analysis

In order to keep the methodologies as comparable as possible across organisms, we transform the estimated spike trains back into continuous calcium dynamics, by applying a one-sided exponential filter with a decay of 2 seconds (see methods). Figure 4.40 shows the result of this preprocessing step in comparison to the deconvolved spiking data.
Additional to the global arousal measurements, the data also contains 1000 Face Motion PCs, quantifying more granular facial movements. In [SPS+19], these have been used as predictors in reduced rank regression, with the 128 most reliable dimensions of neuronal activity as targets, finding that up to 16 behavioral dimensions are linearly encoded in this reliable V1 activity. Additionally, the first face motion PC has been claimed to encode overall arousal, similar to the three global arousal measurements. These global variables, as well as the first face motion PC, are all correlated with one another.

However, while it has been shown that when predicting reliable V1 activity, the predictive power of these 16 behavioral dimensions decreases the less reliable the prediction target dimension is claimed to be, with no relevant predictability beyond dimension 128, no statement has been made regarding connections between neuronal dimensions after 128 and behavioral dimensions after 16. It only has been shown that the reliability of V1 activity decreases when moving into these later dimensions. However, reliability in this

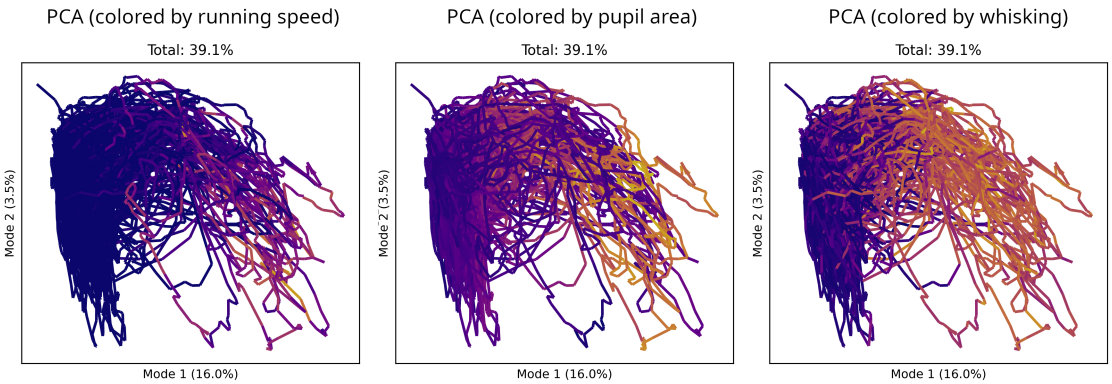


Figure 4.41.: **Mouse manifold.** Latent dimensionality is set to 16. Manifolds are colored by the arousal variables, with brighter colors corresponding to larger values.

context was defined based on spatial cross-validation. That means that activity is only considered reliable if it is consistent across two spatially segregated partitions of V1 neurons. Therefore, it could still be that activity of more localized groups of neurons encode more concrete behavior, even linearly. Furthermore, a nonlinear encoding of behavior at any dimension could still be present.

The approach we choose for our analysis is to keep the constraints loose; instead of restricting the manifold to a few dimensions only in hopes to have it encode high-level global arousal, we allow it to span 16 dimensions, giving it the theoretical ability to capture all 16 dimensions of linearly encoded behavior in V1 if needed, enabling the model to explore a larger range of possible modulation signals in the data, where face motion encodings can act as either a source or target of modulation. This allows for a wider space of potential positive results that can be further analysed and interpreted afterwards, possibly leading to reasonable hard constraints to retrain the model with in future work.

Just as before, we start with an initial PCA. Figure 4.41 shows the resulting phase plots colored by each of the global variables. The plots illustrate the connection between neuronal activity and the arousal variables. Due to their correlation, they are not fit to be used as targets for dimensionality reduction methods that are sensitive to ill-conditioned data like CCA or PLS. Nevertheless, global arousal appears to be encoded in the first neuronal mode. Interestingly, out of the total explained variance, a little less than half lives in this first mode, with the second mode covering only about a tenth.
The trace heatmap of Figure 4.42 shows the first 1000 neurons after sorting by PCA loadings. The continuous arousal variables are plotted above for comparison, sorted manually. We see groups of correlated neuronal activity in sync with them, specifically with running speed and whisking.
As before, we continue with CCA between residual traces and low-level behavior, i.e. the

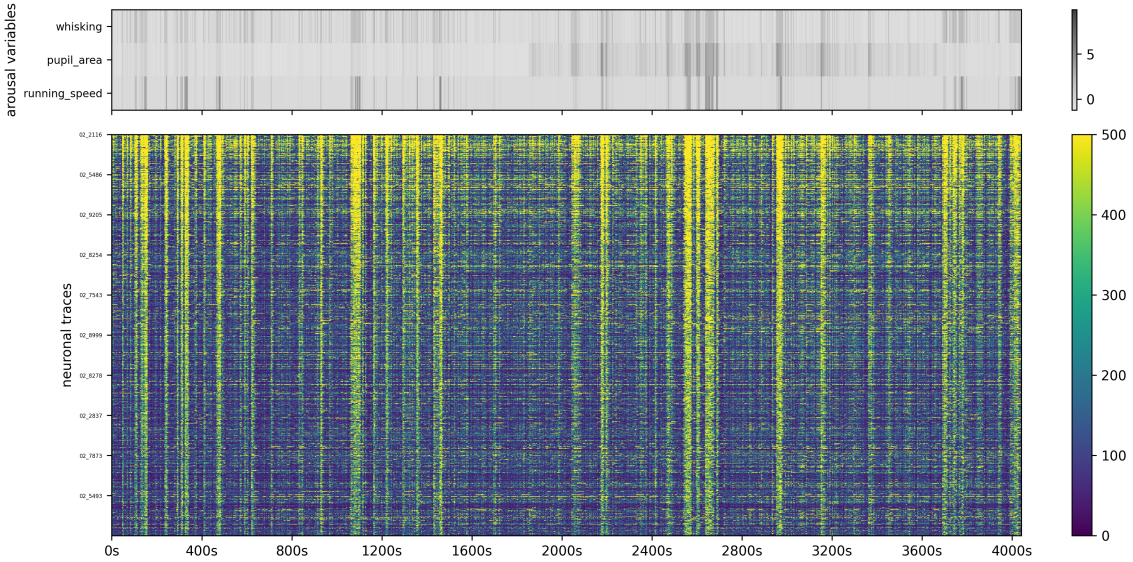


Figure 4.42.: **Mouse trace heatmap.** The first 1000 traces are shown, as sorted by PCA loadings. The three general arousal variables are sorted manually.

1000 face motion PCs. Different to worms and flies, the behavioral data for mice is itself very high-dimensional. It would be suboptimal and computationally expensive to set the latent dimensionality of CCA to be the same as this behavior, i.e. 1000, as we did for the other two organisms. Since we know 16 dimensions of behavior to be encoded in V1, we therefore constrain the number of canonical variates to 16 as well. For the subspace version of the full model, we will later also utilize this dimensionality of 16 to define the behavioral subspace. Again, this allows the model to find shared activity, possibly encoding arousal, that modulates up to all 16 dimensions of behavioral encodings found in V1, or use behavioral encoding signals to modulate other behavioral encoding signals (possibly even past dimension 16), as well as a mixture of both.
Regarding CCA, this could also find other, more correlated behavioral dimensions with less variance, that were not found in the reliable V1 activity.
Figure 4.43 shows the residuals sorted by CCA loadings, with the 10 best-fit face motion PCs sorted by CCA target loadings. This residual heatmap looks rather chaotic, with little structure visually recognizable.
The variances explained by the three components estimated as described are shown in Figure 4.44 as before. The concrete component barely explains any variance, maxing out around 2.9%. Again, this CCA-based concrete component estimation did not provide sufficient insight into the relationship between residuals and face motion.
Therefore, we follow up by fitting a direct linear regression model from behavior (i.e. face motion PCs) to residual activity, as we did for flies. Here, we find an average R^2 of 34.4%, with the maximum at over 70%. As stated earlier, Stringer et al. were able to find low-dimensional encodings of face motion in the full neuronal data, at up

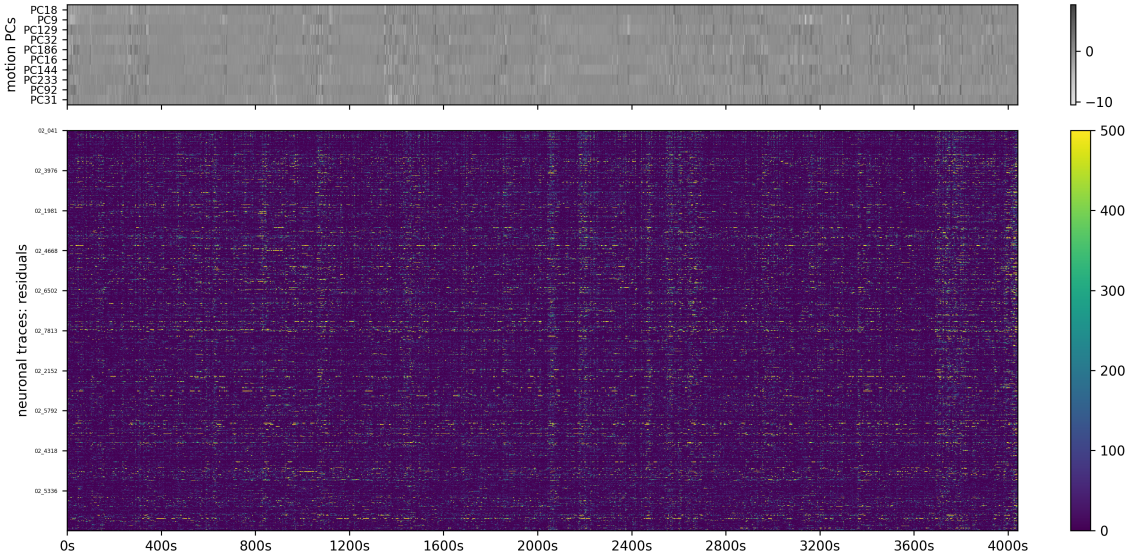
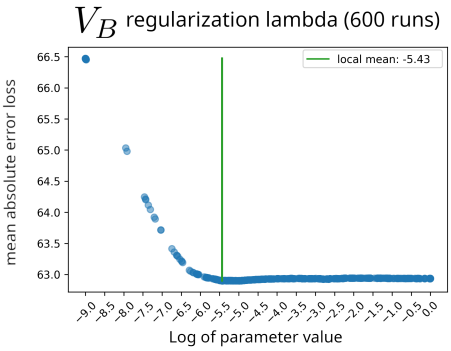


Figure 4.43.: **Mouse residual heatmap** The first 1000 traces are shown, as sorted by CCA loadings. The 10 best-fit face motion PCs are shown, sorted by CCA target loadings.

to 16 reliable dimensions ([SPS+19]), by employing RRR. Our exploratory regression indicates behavioral encodings to be present past the 16 dimensions that we defined as the manifold.

Figure 4.44.: **Mouse MCO ratios.** Color coding is the same as in Figure 4.36. Plot only shows data for dataset 2. The latent dimension of CCA between residuals and face motion PCs is set to 16.

SS-MCM



SS-HGM

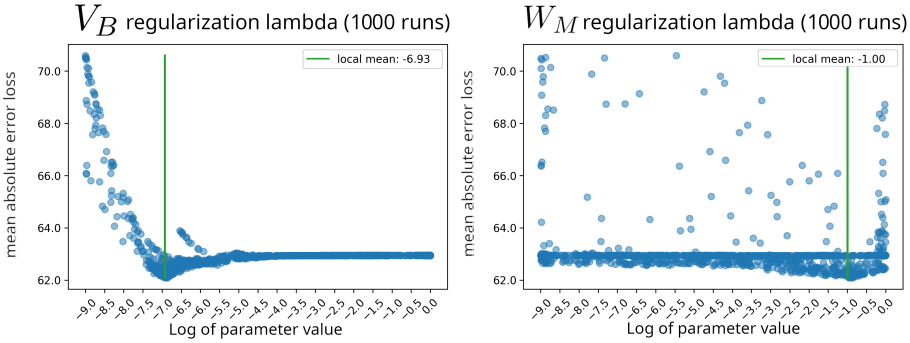


Figure 4.45.: **SS-Model sweeps.** Both models show validation loss minima for all hyperparameters. Due to computational limitations, only one dataset was used for sweeping.

4.5.2. Model Results

We sweep for the best regularization hyperparameters again, for all four model designs (i.e. MCM, HGM, SS-MCM and SS-HGM). Although not strictly converging to the maximum possible value like in flies, the single-neuron versions unfortunately still exhibit sweeping dynamics that show no clear minima for the validation loss (plots accessible in the appendix, Figure A.17).
However, the Subspace versions intriguingly do find prominent minima for all hyperparameters in both models (Figure 4.45). The fact that the Subspace versions appear to have found a structure the single neuron versions were not able to identify is unexpected, considering the subspace versions to be strictly less powerful (see methods). A possible reason for such behavior could be the fact that models with fewer parameters are intrinsically differently regularized. Alternatively, it could also be attributed to the properties of the training process (see discussion). Nonetheless, they were able to generalize their findings and consistently reduce the validation loss, specifically in the case of the SS-HGM.

Residual Predictions

We repeat the significance analysis from before, assessing the results of the SS-HGM and SS-MCM. We again train 100 shuffle controls per dataset and assemble a distribution of sample percentiles across neurons. With around 35%, we find that a rather large amount of neurons in dataset 2 land in the 100th percentile (Figure 4.46). However, the number of actually significant neurons is considerably lower, sitting at about 23%. This means that even though many neurons land in the 100th percentile, a large portion of them have R^2's below 0 for at least one of the models. Across all datasets, this relative share of significant neurons still averages around 35%, ranging from 17% to 44% (Figure 4.47). Interestingly, the dataset used for sweeping did not show the highest number.

When looking at the individual neuron R^2's as predicted by the SS-HGM and SS-MCM plotted against each other, the nonlinear model shows a much larger range compared to the linear model, going about four times as high (see Figure 4.48). In contrast to worms, this difference is not reflected by a few individual outliers but instead appears to be a consistent effect within the distribution of scores.
Across datasets, the SS-MCM continues to perform much worse than the SS-HGM (Figure 4.49). We see SS-HGM R^2's for individual neurons cross 25% in some datasets, with the maximum R^2 for the SS-MCM sitting at 9.5% and in six out of nine datasets, this value doesn't even cross 3%. Interestingly, when considering significant neurons only, many of these high scoring neurons are actually considered outliers.

As before, we quantify the model differences by testing R^2 improvements against shuffle control R^2 improvements for significant neurons. For dataset 2, a grand total of 1799 neurons have the smallest sample p-value, indicating significant improvement (see Figure 4.50).
Across datasets, the improvement is accordingly high (Figure 4.51). Again, within significant neurons with significant improvements, it's mostly outliers that exhibit strong improvements.

Overall, these results indicate a linear encoding of intrinsically low-dimensional face motion information in the residual V1 activity to be vanishingly weak. However, the exploratory analysis indicated a direct, not necessarily low-dimensional linear encoding to be present, which was unexpectedly not picked up by the MCM or HGM (see discussion). But most intriguingly, the Subspace-HGM was able to consistently find a significant nonlinear encoding of such intrinsically low-dimensional face motion information, present in rather large groups of neurons. Distinct behavioral modes seem to be present in the residual activity, but Manifold-related information appears to be of essential relevance regarding the way they are encoded.
How does the manifold improve the prediction in the SS-HGM? In worms, we have confirmed this to be happening mostly via a gating effect. However, Figure 4.52 and Figure 4.53 show a few example traces with high R^2 and paint a different picture for mice:

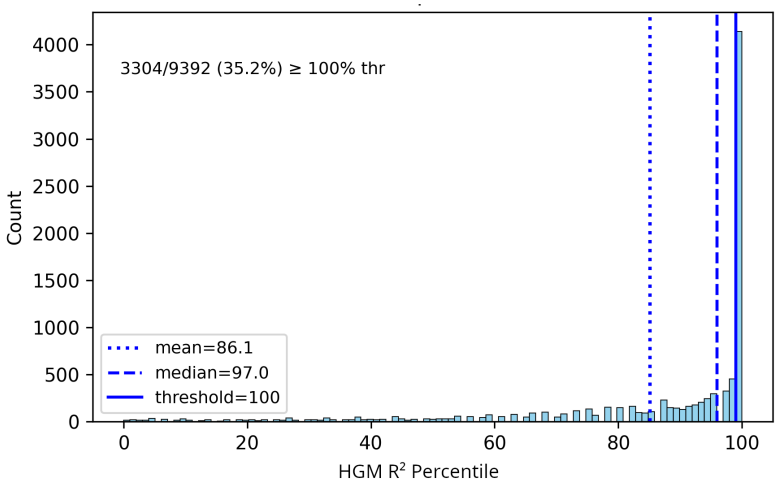


Figure 4.46.: **SS-HGM R^2 control percentile distribution over all neurons.** In this dataset (ID 2), about a third of all neurons land in the 100th percentile.

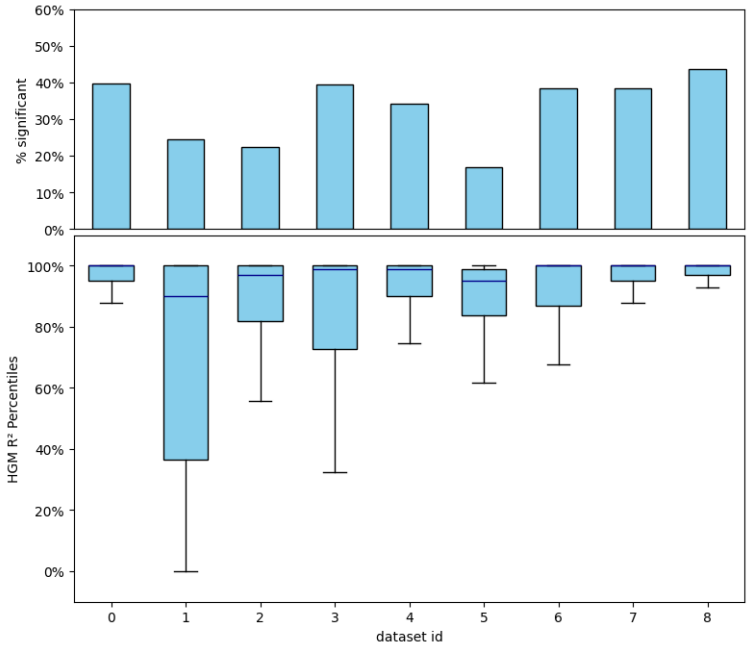


Figure 4.47.: **SS-HGM R^2 control percentile distributions for all datasets.** The boxplots show the distribution of HGM R^2 control percentiles for all datasets. The upper bars show the respective relative share of significant neurons, i.e. neurons that land in the 100th percentile of HGM controls and having an R^2 above zero for both models.

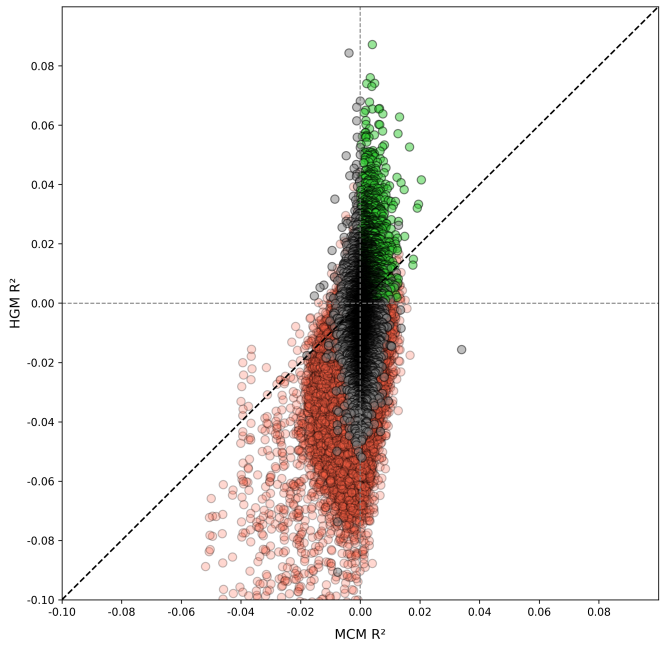


Figure 4.48.: **SS-HGM vs. SS-MCM R^2 for all neurons with controls.** Showing all neurons, including all shuffle controls (pooled) for dataset 2. Significant neurons are colored green, non significant neurons gray, controls red.

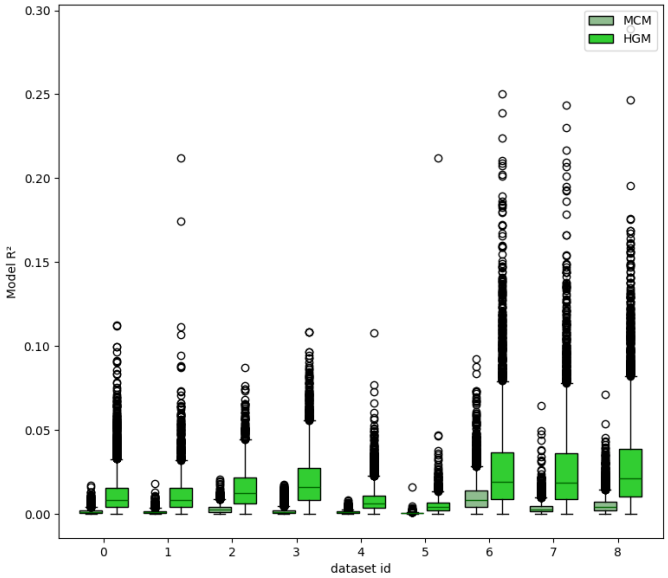


Figure 4.49.: **SS-HGM and SS-MCM R^2 distributions for all datasets.** Distributions only take significant neurons into account.

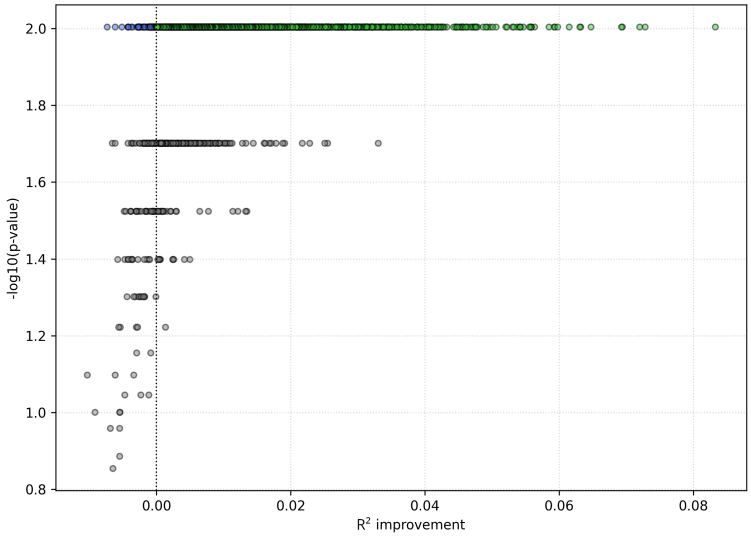


Figure 4.50.: R^2 **improvement against shuffle control R^2 improvement distribution sample p-value.** The plot shows only significant neurons. Neurons whose improvement/non-improvement is significant are colored green/blue, others gray.

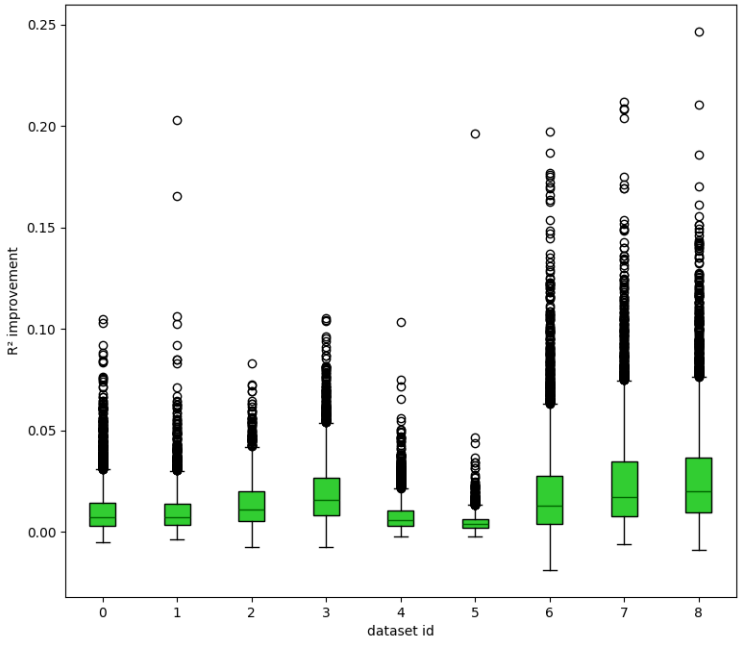


Figure 4.51.: R^2 **improvement distributions for all datasets.** The distributions only take significant neurons with significant improvement into account.

Foremost, we can see residual activity to occur more often during periods of manifold activity even though by definition of the model, the residuals are not linearly related with what we define as the manifold and must correspond to other signals.
These other residual signals are apparently not well predictable in a linear manner by face motion information only. In the example traces, this prediction consistently stays close to the zero line. However, the nonlinear model is able to represent them fairly well by making use of the manifold signal to up-modulate the linear predictions.
It is also imminent that the residuals as defined by the linear and nonlinear model differ significantly more than in the case of worms. Other than improving a linear prediction baseline of the same residuals, the model seems to only be able to even find predictable residuals when allowing the nonlinear manifold integration. A possible biological interpretation of these results requires further parameter analysis and feature attribution.
In any case, the HGM was able to consistently predict these residual signals both during periods of high and low manifold activity (i.e. arousal in this case) by modulating a low-dimensional combination of face motion PC's.
In total, these results are certainly of particular interest and motivate further analysis.

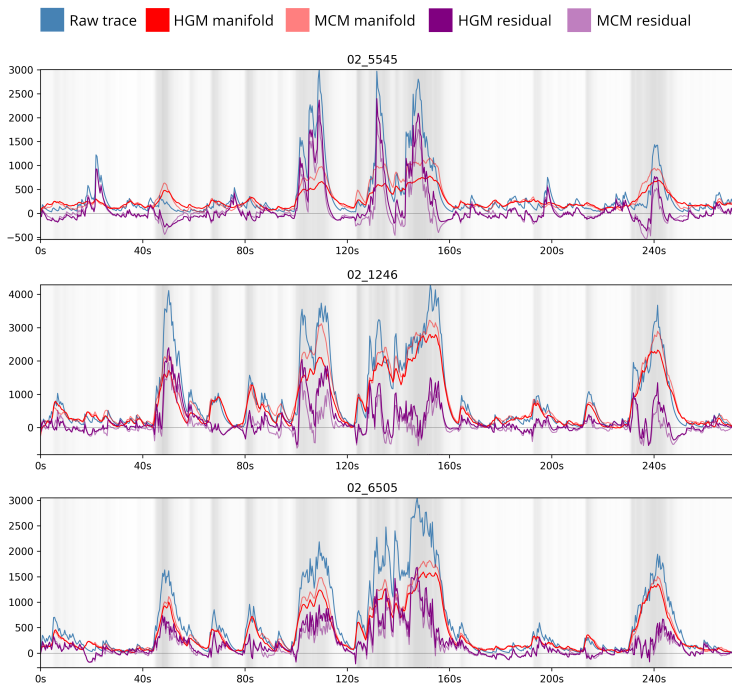


Figure 4.52.: **SS-HGM Manifold prediction examples.** Showing a few examples of high R^2 neurons. Original traces in blue, manifold components in red, residuals in purple. Transparent lines denote the SS-MCM prediction, SS-HGM in opaque. Gray bars represent pupil area.

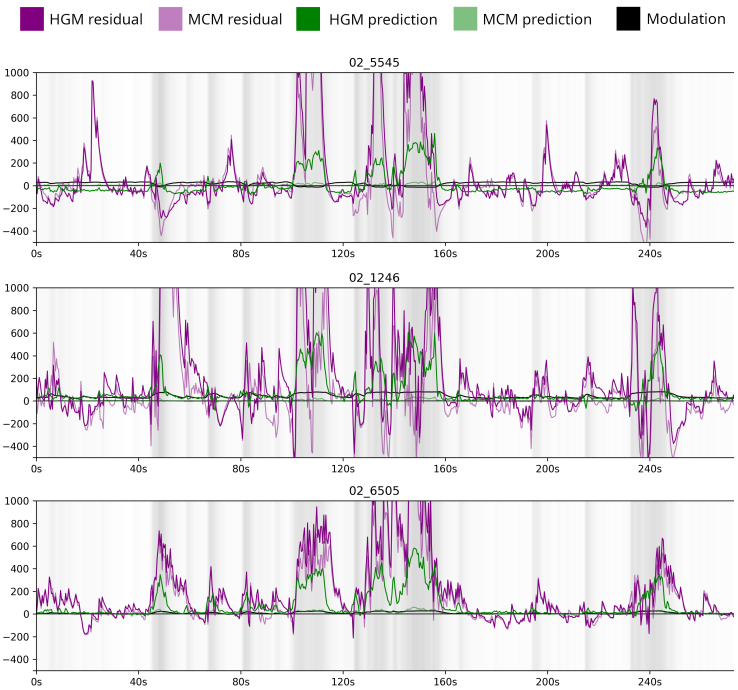


Figure 4.53.: **SS-HGM Residual predictions examples.** Showing a few examples of high R^2 neurons. Residuals in purple, residual predictions in green. Transparent lines denote the SS-MCM prediction, SS-HGM in opaque. The subspace gating term when projected into the single-neuron space is indicated in black. Gray bars represent pupil area.

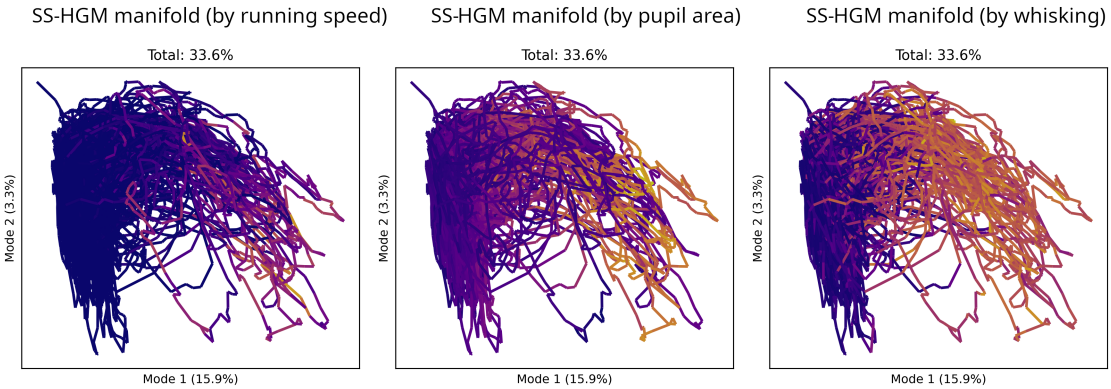


Figure 4.54.: **SS-HGM manifolds.** Manifolds are colored by the arousal variables, with brighter colors corresponding to larger values.

Modulation

How does this modulation arise from the manifold and how is it present in the behavioral subspace?
As before, we visualize the manifold found by the model in a phase plot, colored by the high level behavioral variables (Figure 4.54). This time, we plot each variable individually, as their occurrence is not exclusive. We see a salient similarity to the result of PCA, however covering about 5.5% less variance. This difference lives mostly in later modes and the first mode again covers almost half the total explained variance and appears to be correlated with all three variables to slightly varying degrees. A modulation of residual activity therefore seems to be strongly depending on a signal encoding arousal.
We obtain the modulation subspace the same way we computed the gating subspace in worms, by applying PCA to the isolated single-neuron modulation terms (Figure 4.57). Unfortunately, no visible structure regarding the projections of the (predicted) residual modes into this modulation subspace is apparent in mice either. However, the modulation modes reveal that the first PC captures the strongest effect by far, with the second and third also consistently showing small activity. In some datasets, even the fourth mode captures very slight dynamics thereof.
Coloring the manifold by modulatory activity in these PC's (Figure 4.56) reveals a connection between the modulation and manifold modes: This first and strongest modulation is aligned with the first manifold mode, which is also aligned with arousal. A possible alignment of further modulatory activity that is captured by PC 2 and 3 with the manifold appears rather unclear. A qualitative assessment could be made to connect the second PC to the second manifold mode; however, this relationship seems unsteady and could just as well be attributed to a later mode that is not captured by the two-dimensional phase plot.
In any case, the first PC captures the strongest modulatory effect by far and shows a positive relationship with arousal, meaning it up-modulates residual activity according

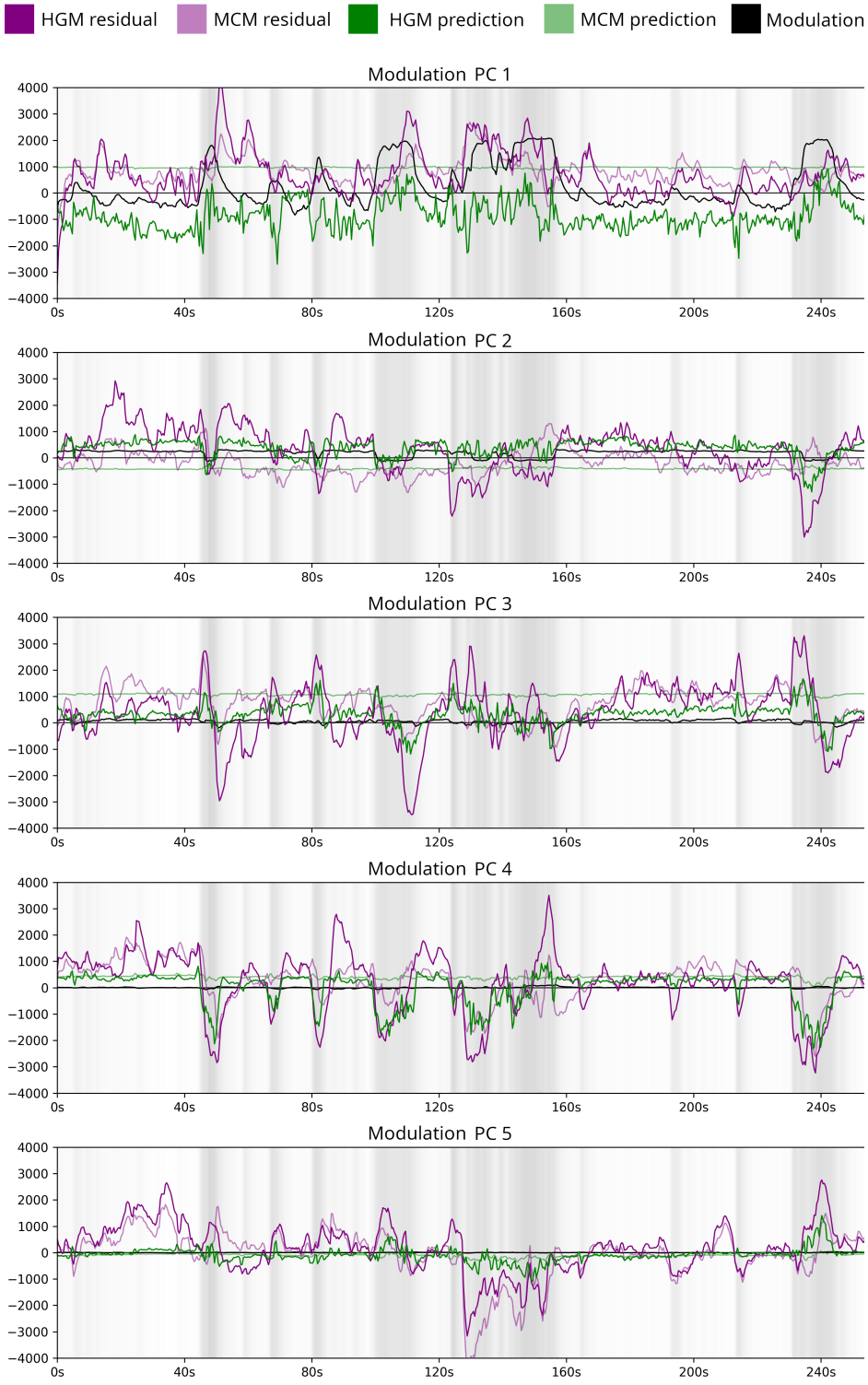


Figure 4.55.: **Gating subspace.** Black traces depict gating modes. Residual prediction modes in green, actual residual modes in purple. Transparent lines denote SS-MCM prediction, SS-HGM in opaque. Gray bars show pupil area. The off-centering of the traces is a side effect of transforming data with PCA fitted on other data.

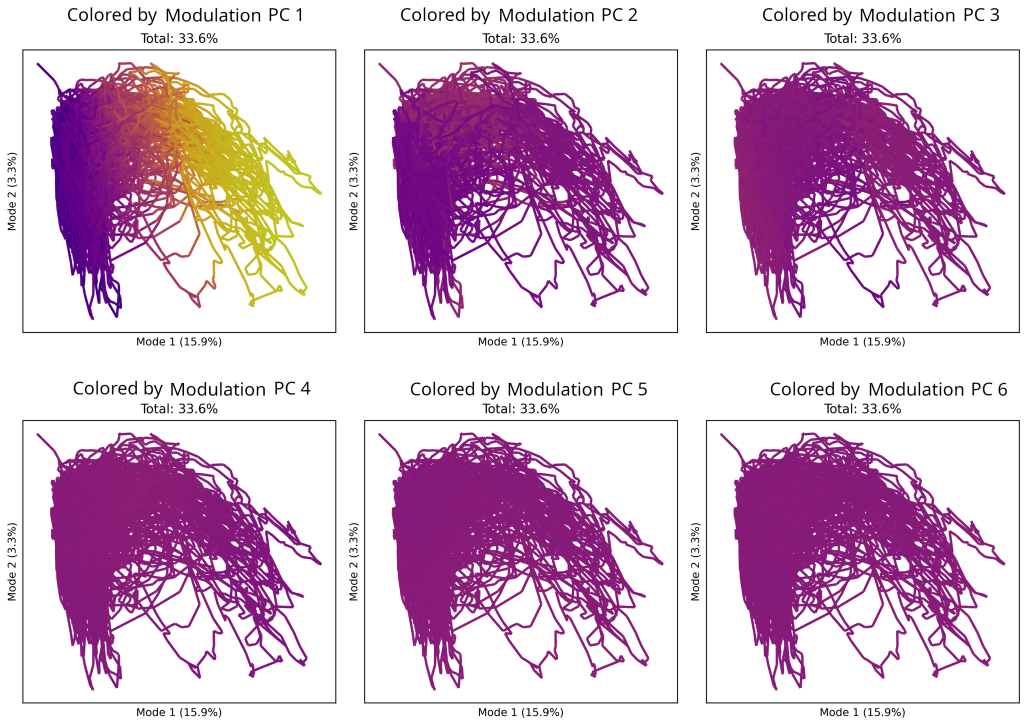


Figure 4.56.: **SS-HGM manifolds by different gating modes.** Manifolds are colored by modulation subspace activity, with brighter colors corresponding to larger values. Only the first 6 out of all 16 PC's are shown, as modulation only happens in the first few.

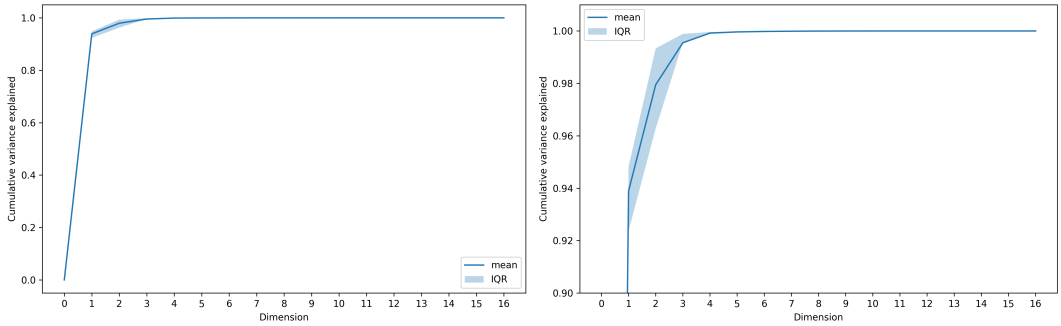


Figure 4.57.: **Gating dimensionalities across datasets.** Zooming in on the top 10% (right) shows the average cumulative variance explained continuing to weakly increase up until dimension 4.

to the general arousal axis, rather than down-gating it.
Lastly, we find that across datasets, the modulation effect strength quickly decreases for later PC's. Most modulation variance is explained by the first PC, with additional PCs still showing relevant but rather small effects (Figure 4.57). The modulation dimensionality seems to be capped at 4, even though the model would allow unique dynamics in up to 16 dimensions, indicating that this number reflects an intrinsic feature.

5. Methods

5.1. Preprocessing

For all three organisms, the neuronal traces as well as behavioral time-series were mean-centered before training.

5.1.1. Fly

Neuronal Traces

For preprocessing the neuronal traces of all *Drosophila* datasets in order to get rid of the observed drift, we applied a 4th order high-pass Butterworth filter ([But30]) to the traces. We set the cutoff frequency to 100, which causes all dynamics that are slower than 100s to be filtered. This was achieved by calling the respective functions `scipy.signal.sosfiltfilt` and `scipy.signal.butter` from the SciPy Python library ([VGO+20]).

Markers

We added additional shifted marker time-series for model training by re-adding each marker trace multiple times, each shifted by one timestep (corresponding to approximately 0.1s) forwards in time, with the maximum shift not exceeding 5 seconds.

In order to regress marker gradients to neuronal residual activity, we processed the marker traces using `numpy.grad` ([HMvdW+20]). It forms gradient approximations by taking two-sided differences for all values in the middle and or one-sided differences at the edges:

$$grad(x_i) = \frac{x_{i+1} - x_{i-1}}{2\Delta} \tag{5.1}$$

$$grad(x_0) = \frac{x_1 - x_0}{\Delta} \qquad \& \qquad grad(x_n) = \frac{x_n - x_{n-1}}{\Delta} \tag{5.2}$$

5.1.2. Mice

Neuronal Traces

The OASIS approach ([FZP17]) used by Stringer et al. interprets the calcium signal as the spike input blurred by an exponential decay model. Therefore, it deconvolves the calcium traces by undoing this exponential decay. Since we want to work with the

original continuous calcium signal, we "reconvolved" it by re-applying an exponential filter with a decay of 2s, as Stringer et al. also used 2s in their application of OASIS.

$$filter(s_t) = x_t = x_{t-1} \cdot e^{-\frac{\Delta t}{2s}} + s_t \tag{5.3}$$

Whisking

The available data for [SPS+19] does not contain a one-dimensional whisking timeseries even though it is used in the paper. It does however contain the whisker motion PC's and whisker motion masks, from which the whisker area motion energy timeseries can be reconstructed to a 1000 dimensions accuracy. We recomputed the one-dimensional whisking time-series ourselves by summing up over the reconstructed whisker motion energy pixels. Due to the reduced dimensionality, this might not exactly represent the whisking variable as used in the paper, but suffices for our analysis.

5.2. Toy Data generation

The toy traces used to visualize the MCO decomposition (Figure 4.1) as well as the hierarchical organization of behavioral encodings (Figure 4.3) were created by generating each of the M, C and & O components individually and adding them up. Mathematically, the generation of the Manifold- and Concrete-components comprises of generating random weight matrices to multiply an artificial manifold signal and eigenworm projections with, respectively. These weight matrices are structurally equivalent to the U_N^T and W_B parameter matrices in the Model.
The artificial manifold signal was generated by smoothing the binary one-dimensional reversal state timeseries of *C. elegans* dataset 19 using a 1D gaussian filter with a σ of 2. This way, we obtain smooth increases and decreases in artificial activity synchronized with reversal timing that serves as a one-dimensional manifold.
To generate the Concrete-component, we use the actual *C. elegans* eigenworm projections of dataset 19. In the case of the nonlinear Concrete-component shown in Figure 4.3, we additionally generate an artificial equivalent of W_Z. We then use the same mathematical structure that we also define in the model to create the gated trace.
All random matrices are generated using a normal distribution with $\mu = 0$ and $\sigma = 1$.
The Other-component is generated by a poisson series with $\lambda = 0.01$ to obtain a time-series of statistical binary events that are then randomly sign-flipped and eventually smoothed with an exponential filter (decay equals 2s).

5.3. Exploratory Methods

For exploratory analysis, we used PCA, CCA, PLS and Linear Regression as implemented in the Python library SciKit-Learn ([PVG+11]).
For dimensionality reduction, it implements the iterative deflation algorithm described in [Weg00] to find the best solution in a finite number of steps.

It order to regress data X onto targets Y, it minimizes the mean squared error as described in 2.2.2 using the least-squares estimator for W:

$$\hat{W} = (X^T X)^+ X^T Y \tag{5.4}$$

where A^+ denotes the Moore-Penrose inverse of A. A computationally efficient and numerically stable way to compute it using QR decomposition utilized by the library is implemented in `numpy.linalg.lstsq`.

5.4. Model Design

5.4.1. Residual Regression

As laid out in 2.2.4, instead of computing the closed-form solution, optimization problems can also be solved numerically using gradient descent. In this interpretation, the loss of the closed-form solution is merely a point on a function of possible losses depending on model parameters, positioned at its minimum. In order to combine multiple such loss functions to design a more complex model that cannot be solved analytically, we have to resort to also training the analytically solvable problems, such as linear or reduced rank regression, using gradient descent.

The MCM uses a direct linear regression from behavior to residuals. When predicting multiple targets in such multivariate linear regression, the model has to tune its parameters such that it achieves the highest overall score for all targets. However, when targets differ significantly, decreasing the prediction error for one target can cause an increase in the prediction error for another target. Assuming a linear relationship between these errors, the choice of the error norm determines how the model deals with such a tradeoff (Figure 5.1). Optimizing the L_2 norm will result in the model making a compromise between both targets. Optimizing the L_1 norm however, causes the model to minimize on the option that is in general easier to satisfy. Since we aim to find specific neurons which show a the desired structure we aim to identify, rather than looking for large sets of neurons that only somewhat suffice, we prefer the L_1 over the L_2 norm, which means minimizing the absolute rather than the squared error.
Furthermore, we regularize the regression weights via Ridge-regularization. It punishes large weights by adding their L_2-norm as an additional term to the model's loss function. For the standard MCM, the loss function optimizes the regression weights $W_B \in \mathbb{R}^{b \times m}$, with m the number of neurons and b the number of low-level behaviors.

$$W_B = \arg\min_{W_B} \|(X_N - \hat{X}_N) - X_B \cdot W_B\|_F^1 \quad + \lambda \|W_B\|_F^2 \tag{5.5}$$

$\hat{X}_N$ denotes the reconstruction of the original data X_N by the linear autoencoder, such that subtracting it from X_N gives the residuals. More on that in the next section. In the case of matrices, the L-norm generalize to the Frobenius norm, denoted by $\| \ \|_F$.

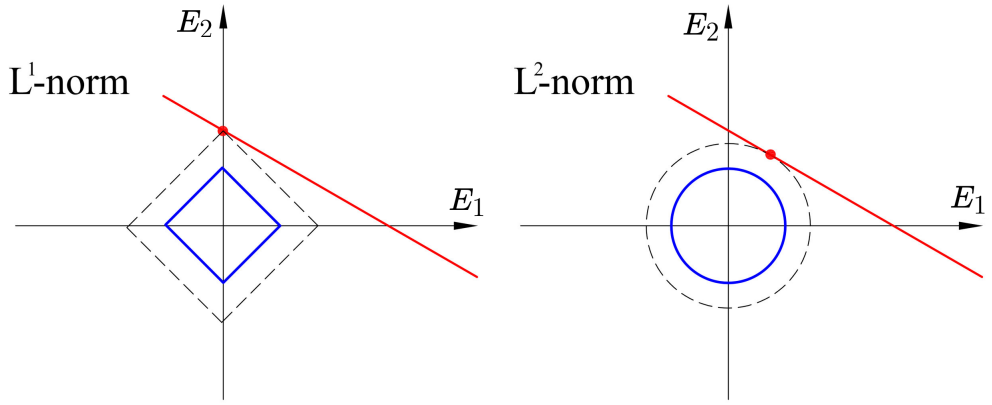


Figure 5.1.: **MAE vs. MSE.** When adding the flat errors for both targets (L_1-norm), the minimum is achieved by concentrating on E_2. When adding the squared losses however (L_2-norm), the optimal parameters fall somewhere between satisfying E_1 and E_2.

The Subspace-MCM uses reduced rank regression instead. This means that the loss function optimizes two weight matrices: The matrix $W_B \in \mathbb{R}^{b \times r}$ that predicts the subspace and the matrix $V_B \in \mathbb{R}^{r \times m}$ that predicts the residuals. r is the dimensionality of the behavioral subspace. As before, we train on the mean absolute error and regularize the weights that predict the residuals, in this case V_B, via Ridge-Regularization. We do not need to regularize W_B as well, since we modify the classical RRR by enforcing orthogonality for W_B (we use the term "orthogonal matrix" to denote matrices with orthonormal columns), which already ensures unit norm of each column. We achieve this by extracting its orthogonal part via QR-decomposition ([GVL96]) in each training step.

$$W_B, V_B = \arg\min_{W_B, V_B} \|(X_N - \hat{X}_N) - X_B \cdot W_B \cdot V_B\|_F^1 \quad + \lambda \|V_B\|_F^2 \quad \text{s.t. } W_B^T \cdot W_B = I \tag{5.6}$$

5.4.2. Linear Autoencoder

The linear autoencoder learns the orthogonal weights matrix $U_N \in \mathbb{R}^{n \times d}$, with n the number of samples and d the dimensionality of the manifold. The matrix is orthogonalized at each learning step via QR-decomposition. The manifold is then defined as $Z_N = X_N \cdot U$. Since we enforce orthogonality, we do not have to train a second reconstruction matrix separately as done for a general linear autoencoder, but instead reconstruct X_N as $\hat{X}_N = Z_N \cdot U_N^T$, since for orthogonal matrices it holds that $U_N^{-1} = U_N^T$.
This autoencoder determines the residuals to be predicted from the low-level behavior. Since the full model is trained on this prediction's loss only, the parameters of the autoencoder also depend on the residual prediction. The trivial way to reduce that loss is to minimize the overall variance of the residuals, reducing the general magnitude of the prediction error. This is equivalent to maximizing the variance of the manifold. Therefore, when assuming no connection between residuals and low-level behavior, the linear autoencoder is trained to maximize the variance of the data in a low-dimensional

space found by an orthogonal linear transformation. This is very similar to PCA, with the only difference being the use of the MAE instead of MSE, which is minimized in classic PCA. The result is a version of PCA that is more tolerant to individual large values.
However, the encoder can further decrease the downstream loss by specifically capturing shared variance that is not predictable by behavior. This way, shared variance is captured by the manifold and variance that is predictable by behavior is captured by the residual prediction, leaving only uncorrelated unshared variance that can increase the loss. Like this, the autoencoder can spread the variance of the data over the Manifold- and Concrete-components such that the Other-component, that defines the loss, is as small as possible.

5.4.3. Nonlinear Gating Extension

The nonlinear HGM modifies the residual regression of the MCM with an additional gating term. This term is defined by another weight matrix $W_Z \in \mathbb{R}^{l \times m}$ that is multiplied with the manifold trace matrix. Nonlinearity is introduced by mapping the result to values between 0 and 1 using an element-wise sigmoid. This way, the gating term is the result of a small MLP with only one layer of size $l \times m$ and a sigmoid as the activation function. We regularize the perceptron weights W_Z via ridge-regularization.
This gating term is then multiplied element-wise with the result of the linear residual regression to obtain the nonlinear prediction of the HGM.
The HGM is strictly more powerful than the MCM as it can always find parameters which result in the same prediction the MCM is capable of, by defining $W_Z = \mathbf{0}$ such that $\sigma(W_{M_{ij}}) = 0.5 \; \forall i \in l, j \in m$ and $W_{B_{HGM}} = 2 \cdot W_{B_{MCM}}$ to undo the generic gating. In fact, any arbitrary gating for a neuron can be achieved while keeping the prediction the same during ungated periods by calculating its HGM weights from its MCM weight and the desired gating $g \in [0, 1]$ as follows:

$$w_{b_{HGM}} = (1 + g) \cdot w_{b_{MCM}} \qquad \text{and} \qquad w_z = -\ln(g) \tag{5.7}$$

The subspace model works analogously, with the difference that the gating is applied to the modes of the behavioral subspace found by RRR instead, making $W_Z \in \mathbb{R}^{l \times r}$.

5.4.4. Loss function

All things considered, the full regularized loss function for the HGM is defined as follows:

$$\begin{aligned} U_N, W_B, W_Z = \arg\min_{U_N, W_B, W_Z} & \|(X_N - (X_N \cdot U_N \cdot U_N^T)) - X_B \cdot W_B \circ \sigma(X_N \cdot U_N \cdot W_Z)\|_F^1 \\ & + \lambda_1 \|W_B\|_F^2 + \lambda_2 \|W_Z\|_F^2 \qquad \text{s.t. } U_N^T \cdot U_N = I \end{aligned} \tag{5.8}$$

Here, $\circ$ denotes element-wise multiplication.
For the subspace case, the residual loss also has to be computed in the single-neuron

space, as defining it in the subspace case would result in the model being able to achieve a perfect loss by simply reducing everything to 0 in this step. Therefore, the SS-HGM loss function only differs by the additional weight matrix V_B.

$$U_N, W_B, V_B, W_Z = \arg\min_{U_N, W_B, V_B, W_Z} \|(X_N - (X_N \cdot U_N \cdot U_N^T)) - X_B \cdot W_B \cdot V_B \circ \sigma(X_N \cdot U_N \cdot W_Z)\|_F^1 + \lambda_1 \|V_B\|_F^2 + \lambda_2 \|W_Z\|_F^2 \quad \text{s.t. } U_N^T \cdot U_N = I, \; W_B^T \cdot W_B = I \tag{5.9}$$

The loss functions for the MCM and SS-MCM are defined analogously, leaving out the sigmoid (and W_Z regularization) terms.

5.4.5. Post-processing

The subspace models find a behavioral subspace as well as single-neuron residual predictions arising from this subspace. We post-process the model results to find the projections of the actual residuals into this behavioral subspace, as well as a modulation subspace and respective residual projections into this modulation subspace. These projections indicate how the latent signals in these spaces would look like when considering the information of the actual residuals, rather than their predictions. In other words, they contain the other-component as well.

Since the linear transformation that predicts the residuals Y from the behavioral subspace Z_B is given by V_B, the behavioral subspace modes could in turn be obtained again from the predicted residuals $\hat{Y}$ by inverting this operation. Since V_B is not square and usually ill-conditioned and therefore not full rank, we cannot use V_B^{-1} for that purpose, so we instead use the least-squares estimator again (see 5.3):

$$\hat{V_B^{\leftarrow}} = (\hat{Y}^T \hat{Y})^+ \hat{Y}^T Z_B \tag{5.10}$$

In the case of the rank of $\hat{Y}$ being larger or equal to the rank of Z_B, which is usually the case since the intrinsic neuronal dimensions typically exceeds the intrinsic behavioral subspace dimension, this estimator is equivalent to V_B^+.
In order to project the actual residuals into this space, obtaining the residual modes Z_Y, we simply use $\hat{V_B^{\leftarrow}}$ to transform Y instead of $\hat{Y}$.

$$Z_B = \hat{Y} \cdot \hat{V_B^{\leftarrow}} \tag{5.11}$$

$$Z_Y = Y \cdot \hat{V_B^{\leftarrow}} \tag{5.12}$$

Unfortunately, for worms and mice, the behavioral subspace modes (accessible in the appendix, Figure A.9, Figure A.20) appeared rather chaotic, with no clear separation connected to the behavioral predictors that was qualitatively identifiable. Also, the modulation signal in this behavioral subspace appeared spread across modes. Therefore, we turned to modulation subspace based analysis (see 4.3.2, 4.5.2).

The modulation subspace is obtained by isolating the modulation terms Y_G for single neuron residuals first:

$$Y_G = \sigma(Z_N \cdot W_Z) \cdot V_B \tag{5.13}$$

This can be interpreted as predicting residuals with the SS-HGM while setting the linear regression predictions $X_B \cdot W_B := \mathbf{1}$. We apply PCA to Y_G to obtain the modulation subspace. Since Y_G is the result of a nonlinear neural network, a linear method like PCA does not necessarily collapse back to the dimensionality of the manifold Z_N. Its maximum dimensionality is however bound by the behavioral subspace dimension.

5.5. Neuron scoring

5.5.1. Out-of-sample R2

The R^2 score is classically defined using the ratio between squared prediction error sum and dataset variance (see 2.2.2). However, in an supervised case where the training data is split into training and validation sets, this would mean using the validation set mean to compute variance. Doing so would give a biased prediction, as information from the validation set would go into the score assessment, artificially decreasing the R^2. Therefore, we use the mean of the training set instead. This is called the out-of-sample R^2 ([CT08]). We use it for all model score assessments.

5.5.2. Shuffle Control

For the shuffle control datasets, we use a binning shuffle strategy: The input predictors are partitioned into bins of a set size of 50, which are then randomly shuffled, creating a new indexing. This decouples the relationship between regressors and targets, causing the training to not find a generalizable set of weights, while maintaining local autocorrelation in the data, which is of relevance when working with time-series where each sample is a timepoint. We retrain each model on 100 shuffles for each dataset for mice and worms, accumulating a distribution of control model performance metrics for each neuron, against which the actual model can be statistically compared. Although more shuffles would be desirable, the computational effort of each additional shuffle poses a limit. Since the control R^2's are assessing the performance of a model that tries to fit mostly noise, these mean R^2's are usually slightly below 0 due to overfitting [CSL$^+$24].
To assess neuron score significance using the control distributions, we use sample-percentiles instead of fitting a distribution and computing the population percentiles. Therefore, we also work with sample p-values. For neuron significance, we always use the right-sided sample p-value as we are interested in neurons with significantly large R^2's.

5.6. Training

We train the models on the residual loss using the Adam ([KB15]) optimizer without batching. For worms, we train for a maximum of 3000 epochs, for flies and mice we

increase this limit to 4000. Learning rate, early stopping and cross-validation is set up as described below.
Since increasing the manifold dimension will strictly decrease the model loss, we define it manually instead of sweeping for an optimal value. For worms, we chose a manifold dimension of 2, based on previous literature. For flies, we initially used 4 but increased it to 8 later and for mice, we allowed 16 dimensions.
The dimension of the behavioral subspace used in the SS-models is set to be the same dimensionality as the behavioral input itself (5 for worms, and 16 for flies), except for mice, where we constrain it to 16 instead of the full 1000, in order to maintain a reasonable runtime.

Regarding regularization parameters, we sweep as described below, except for the subsequent manual training for flies for visualization purposes, where we estimate them as $\lambda_{W_B} = 1\text{E}{-4}$ for the MCM and $\lambda_{W_B} = 1\text{E}{-4}, \lambda_{W_Z} = 1\text{E}{-7}$ for the HGM.

5.6.1. Cosine Decay

We use cosine decay without warmup to dynamically define the learning rate in all models. This strategy continuously decreases the learning rate over the course of the training epochs according to the dynamics of cosine function between 0 and π. We initialize it with 1 and decrease it down to a minimum of $1\text{E}{-4}$.

5.6.2. Early Stopping

Additional to Ridge-Regularization, we also use early stopping to avoid overfitting, which is a type of regularization that works in combination with gradient descent. When overfitting, the train loss continues to decrease while the validation loss increases. This motivates a simple idea: Observe the validation loss during the training process and stop the training if it does not improve for a certain number of training iterations, reloading the last best model parameters. We set this number, the "patience"-hyperparameter, to be $\frac{1}{4}$ of the maximum epochs. Additionally, we define an equally long initial warmup period, in which the patience does not count down, in order to allow initial loss fluctuations.

5.6.3. Stratified k-Fold Cross-Validation

We use the stratified k-fold cross-validation for training. It is useful when a dataset contains samples that each belong to a class. The data is split k times into different folds, such that the relative ratio of classes in each fold reflects the ratio of classes in the whole dataset (Figure 5.2). In our case, we want to have a balanced ratio of samples during different high-level behavioral states for training, specifically those states where we assume a different encoding. For worms, we balance the data with respect to the reversal state, in flies with the quiescent state and in mice with a running state. We define this running state as periods where the running speed is within the right-sided 95'th sample percentile of the running speed distribution.

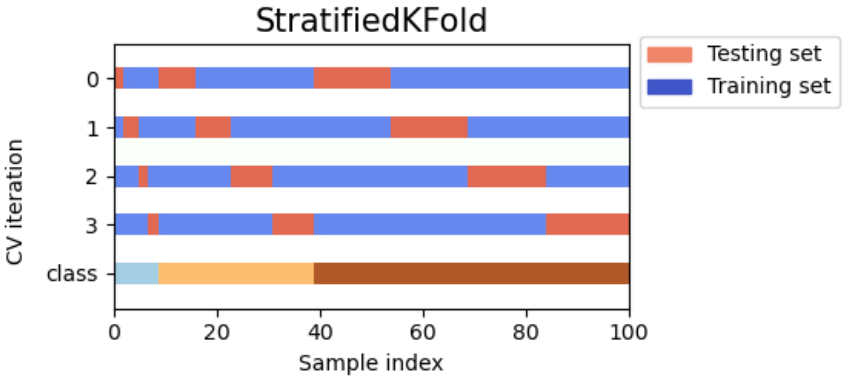


Figure 5.2.: **Stratified k-Fold Cross-Validation.** Adapted from the SciKit-Learn documentation ([PVG+11])

5.7. Sweeping

We optimize the regularization hyperparameters for all models by sweeping for them (see 2.2.5). We use the unregularized residual MAE loss of the validation set, averaged across all splits, to evaluate a sweep. This way, sweeping will search for the set of hyperparameters that causes the model to learn the best possible prediction of neuronal residuals from low-level behavior that is generalizable for held-out data. We use bayesian optimization for sweeping. This strategy iteratively builds a probabilistic model for the function mapping hyperparameter configurations to the resulting evaluation metric, aiming to optimize the next choice in terms of getting close to the functions minimum, by balancing between exploration of new configurations and exploitation of the current model. This strategy has been shown to be in general more efficient than basic grid search.

We search for optimal values in an interval of $[1\text{E}{-9}, 1]$ for all regularization λ's. We set maximum sweep counts based on the models complexity, allowing between 100 and 150 for the (SS-)MCM and 400 for the (SS-)HGM in worms. For flies and mice, we increased to 600 and 1000, respectively.

After sweeping, we build a set of hyperparameters to train the model with by filtering for the the 5% best runs, i.e. runs with the lowest validation MAE, and taking the geometric mean with base 10 across the hyperparameters of these runs.

$$GM_{10}(\vec{x}) = 10^{(\frac{1}{n}\sum_{i=1}^{n}\log(x_i))} \tag{5.14}$$

We chose the geometric mean since we observe a logarithmic relationship between hyperparameter value and loss. Lastly, we take the geometric mean again across all swept datasets for the final configuration we use for training.

The sweeping was implemented and evaluated using the Weights & Biases library ([Bie20]).

5.8. JAX

The model and training code was implemented using JAX ([BFH+18]). We chose this library as it simplifies and speeds up implementation of and training on custom loss functions. JAX allows automatic differentiation and Just-In-Time (JIT) compilation of arbitrary functions using its NumPy drop-in replacement `jax.numpy`. It does this by making use of Autograd (2.2.4) to convert a function, e.g. a model's loss function, into another function that computes its gradient instead. This can be combined with JIT compilation to optimize such a gradient function itself using `jax.jit(jax.grad(loss_function))`. With JAX's built-in control flow such as `jax.lax.while_loop` for loops and `jax.logical_and` or `jax.tree.map` in combination with `jax.numpy.where` for conditions and `if`-statements, model code be executed fully on the GPU without requiring CPU synchronization between training epochs, massively speeding up the training process.

6. Discussion

In this thesis, we've combined modern machine learning techniques such as neural networks, dimensionality reduction and different types of regression to design a custom numerical model that jointly trains these different subparts on a combined loss function. It is able to learn a low-dimensional manifold signal responsible for the modulation of other signals that encode more concrete behavior. It is able to generically learn a modulatory relationship between these signals, including down-gating and up-modulation. Additionally, we designed an extension of the model that also enforces low-dimensionality regarding the concrete signal, making it able to find the subspace in which this signal operates. This way, it also restricts the gating dynamics themselves to a lower dimension, enabling further insights such as their intrinsic dimensionality.
We've designed and verified the models based on experimental GCaMP imaging data of *C. elegans* from our lab while ensuring generalizability. We then applied it to similarly acquired neuronal data of *Drosophila melanogaster* and mice that we researched from publicly available academic sources. We utilised exploratory data science methods to analyse the data and preprocess it for model compatibility.

For *C. elegans*, the results confirmed previously identified gated encodings of body-undulation behavior, present mostly in head motor neurons and motor neurons involved in forward locomotion. This is a step further into the direction of a better understanding of behavior-related information in the *C. elegans* brain and hierarchical interaction on a neuronal level. However, the causal mechanisms behind these signals still remain unclear. The models are able to pick up on any signal related to the behavioral quantifications used as input; therefore, it is uncertain whether the identified neurons play a role in causing the respective behavior or whether there are other reasons for this connection. Studies have confirmed that some motor neurons transduce a proprioceptive signal that is involved in propagating undulatory waves from head to tail during forward movement ([WPH$^+$12]). It is therefore well possible that identified gating mechanisms act on proprioception, rather than concrete motor commands.
The manifold signal found responsible for gating was noticeably similar to the globally widespread signal found to encode the action sequence of high-level motor commands. The gating dynamics arising from this signal exhibit two primary modes across worms, which show strong connections to its crawling direction and turn behavior related brain activity, respectively. In summary, it seems that the brain of *C. elegans* is pervaded by a dominant signal that fulfils multiple functions, driving locomotion on a higher level and interacting with lower-level signals regarding more concrete motor information in a hierarchical gating manner.
The degree by which these levels of activity are shared within neurons varies significantly,

with some neurons encoding almost exclusively the higher-level manifold, others mostly comprising the low-level information and a few showing a mixed representation of both. Different signals are present in the brain in a multiplexed manner and spread across neurons.

Moving from worms to model organisms with a significantly higher number of neurons poses a few challenges. The neuronal recording requires a head-fixed setup, which is not representative of a natural environment for the animal. Furthermore, neurons are not identifiable across sessions and due to the sheer number, the recordings are usually not whole-brain.
During research, we found that publicly available data with suitable granular behavioral quantifications were rare. We originally planned to include Zebrafish in our analysis, but dropped it due to this reason.
The only sufficient behavioral data regarding flies we were able to find was not public and we had to contact the authors for access. Unfortunately, our model analysis was unable to capture a meaningful connection between the neuronal and behavioral data in this case. This is likely to be a technical issue due to the slow character of nuclear-localized calcium imaging dynamics in spiking nervous systems that we encounter in the nls-GCaMP6s data for flies. Another reason could be that the circuitry that actually generates and coordinates the quantified motor patterns is located in areas that were not captured in the imaging experiment. The authors specify that they imaged the dorsal third of the central brain, which does not include e.g. the ventral nerve cord, a region classically associated with motor control. Lastly, it is always possible that the relationship between the specified behavior and the neuronal activity is too complex to be picked up by our model. Therefore, it does not follow from these results that such a connection, let alone a hierarchical organization, is not present in flies.

Despite difficulties, the preliminary results in mice were positive. Our most sophisticated model version was able to reliably find a latent, low-dimensional signal in the activity of the primary visual cortex that encodes granular face motion in a non-linear manner, by hierarchically integrating a higher-level signal that, among possible other things, encodes global arousal. This integration primarily happens by a positive modulation of the latent behavioral signal in 4 distinct modes, one of which also being strongly aligned with arousal. The way face motion is encoded in the mouse V1 appears to be significantly depending on the higher-level context of arousal. An obvious next step to consolidate these results would be to aim for a clearer separation between high-level arousal and low-level granular face motion, by constraining the neuronal manifold dimension to a lower value like 1 or 2, as well as removing the first PC from the behavior, as it is strongly correlated to arousal. This would allow a stronger biological interpretation of the uncovered modulatory dynamic.
Furthermore, there are a few open questions. Even though previous work and initial exploratory modelling on an analytical basis from our side revealed a linear encoding of face motion to be present in the data as well, none of the linear numerical model versions

were able to find it. This raises the question of which differences between the less complex analytical and the more complex numerical model result in this discrepancy. Further work should aim to isolate the point of divergence and close this gap of understanding. Moreover, the numerical models with a larger number of parameters performed worse on the data and it was the less powerful model design that revealed the nonlinear encoding. A possible reason could be that this model is by design differently regularised, enabling it to find the optimum. In this context, it's also important to keep in mind that machine learning models operate on a stochastic basis and results can vary depending on initial parameters and noise. Therefore, it is essential to further investigate these results and optimise the training process.
Lastly, the models were still primarily designed with *C. elegans* in mind and many organism-specific properties in mice, such as an intrinsic manifold dimension, still lack proper definition. Continuative work regarding mice could address this by tailoring the model more closely to murine data, guided by the existing body of neurobiological literature, while preserving the fundamental cross-organism generalizability.

In general, even though it is a rather strong assumption that effects observed in *C. elegans* can be generalised to other model organisms, such a case has already been confirmed regarding the brain-wide representation of ongoing behavior ([KZ20]). The next step was to build on this idea by searching for a generalizable function of these brain-wide signals, motivated by previous work in worms. In this thesis, we consolidated these previous results and additionally found an indication of a similar structure being present in mice, which should be further investigated.

Overview of generative AI tools used

In the preparation of the thesis, I utilized ChatGPT to improve the grammar, find spelling mistakes and help with translating the abstract.

Übersicht verwendeter Hilfmittel

Bei der Erstellung der Masterarbeit habe ich ChatGPT genutzt, um die Grammatik zu verbessern, Rechtschreibfehler zu finden und beim Übersetzen der Kurzfassung zu helfen.

List of Figures

1.1. **The rise of optical imaging.** The blue dots indicate electrophysiology data up until 2011, published in [SK11]. The blue squares indicate new data since then, compiled in [UDLC22]. Red dots indicate new optical imaging data. The black line denotes an exponential fit to the original data until 2011, extrapolated to the present/future where dashed/dotted. The figure was adapted from [UDLC22]. 2

1.2. **The neuronal manifold encodes the global action sequence in *C. elegans*.** Brainwide neuronal activity can be reduced to a low-dimensional brain state signal that encodes high level behavioral commands. Adapted from [KKS+15]. 3

1.3. **Context dependency in *Drosophila*.** Different looming stimuli can leave similar images on the retina - however, depending on the currently exhibited high-level state, the encoding of this sensory input changes, resulting in different behavioral responses downstream. Adapted from [OC22]. 4

2.1. **Eigenworm posture reconstruction.** The full body posture can be decomposed into modes by applying PCA to the curvature timeseries. . . 8

2.2. **DeepGraphPose markers.** A semi-supervised sequence model classifies marker positions into high-level behavioral states. Adapted from [SMW+23]. 9

2.3. **Facial video processing.** PCA is applied to the Motion Energy of the face video. Adapted from [SPS+19] 10

2.4. **Geometric intuition of PCA.** The variance of the data the latent space (orange) is maximized. 11

2.5. **Geometric intuition of CCA.** The correlation between the data two latent spaces (orange) is maximized. 12

2.6. Sigmoid Function $\sigma(x)$. 15

3.1. **Behavioral time-scale hierarchy of *C. elegans*.** The global manifold signal is known to encode the slow motor command action sequence. At a faster time scale, the worm exhibits ventral and dorsal body undulations during crawling movement. Nested within these undulation movements, even faster head-casts are observed. How does neuronal activity represent these granular behavior patterns whose encoding is not found in the manifold signal? Furthermore, does the hierarchical interaction between the levels extend to the encoding level and if yes, how? 20

3.2. **The global manifold signal gates residual activity.** Example traces of neurons of interest are shown, decomposed into a global (red) manifold part and the residual (purple). Triggered averages with respect to body undulation are shown below. The examples highlight the different cases of no encoding (D & H), linear encoding (C & G) and nonlinear encoding depending on the behavioral state encoded in the manifold (A & E, B & F). Adapted from [FLR$^+$25]. 20

4.1. **MCO-decomposition.** A raw trace (blue) is decomposed into a global shared part, the manifold (red), that potentially encodes high-level behavioral states and leaves the residual part (purple) when subtracted. The residual can in turn be decomposed again into a part (green) that encodes low-level, "concrete" behavior, as well as the rest of the trace that contains all other activity (gray). Periods during which reversals happen are colored in gray. 28

4.2. **MCM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The model is solely fit on the error of the residual prediction. 28

4.3. **Hierarchical organization of behavioral encoding.** Based on the observations in 3.2, the amplitude of the concrete component is significantly reduced during periods of reversals (gray). In order to model this feature, the decomposition is combined with a nonlinear gating interaction between the manifold and concrete component. 30

4.4. **Gating via multiplication with the manifold.** The MCM (top) predicts the concrete component linearly from low-level behavior. The HGM (bottom) extends this prediction by multiplying it with a sigmoid of the manifold term. This is a visual depiction of the idea formulated in Equation 3.2 (we omitted the weight matrices in this figure). 30

4.5. **HGM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The circle symbol indicates element-wise multiplication. The model is solely fit on the error of the residual prediction. 31

4.6. **SS-MCM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The model is solely fit on the error of the residual prediction. 32

4.7. **SS-HGM.** Smaller boxes indicate lower-dimensional data matrices. Circles indicate transformation matrices. The circle symbol indicates element-wise multiplication. The model is solely fit on the error of the residual prediction. 33

4.8. ***C. elegans* manifolds.** The axis show the relative variance the respective mode explains. The total explained variance is given at the top. Latent dimension is set to 2. The plots are colored by the high level state quantifications to visualize their connection to the signal. 35

4.9. ***C. elegans* trace heatmap.** Neurons are sorted by PCA loadings. Discrete states are sorted manually. 36

4.10. ***C. elegans* residual heatmap.** Neurons are sorted by CCA loadings. Eigenworms are inversely sorted by CCA target loadings. 37
4.11. ***C. elegans* MCO ratios.** x-axis shows neurons, y-axis shows the relative variance explained by each component of the given neuron, averaged across all datasets. M component in red, C in green, O in gray. Neurons are sorted by C. The per-neuron standard deviations of the relative variance explained for each component are overlaid on their bar segments. 38
4.12. **Hyperparameter sweeps.** The validation loss (y-axis) shows clear minima for all hyperparameters in both model types. The green lines indicate the mean parameter value of the 5% best runs for the depicted dataset (green) and its average across all swept datasets (orange). 39
4.13. **HGM R^2 control distribution examples.** The histograms show the distribution of control scores and the observed score in comparison for a significant (bottom) and a non-significant (top) example neuron. 40
4.14. **HGM R^2 control percentile distribution over all neurons.** The histogram shows the distribution of percentiles in which a neuron's HGM R^2 falls, with respect to its shuffle control distribution, over all neurons of dataset 19. In this dataset, 60 out of 121 neurons land in the 100th percentile. 41
4.15. **HGM R^2 control percentile distributions for all datasets.** The boxplots show the distribution of HGM R^2 control percentiles as in 4.14 for all datasets. The upper bars show the respective relative share of significant neurons. 42
4.16. **Single-neuron manifold prediction examples.** Excerpts of the neuronal traces of AVAL and DD01 are shown in blue. The respective manifold component as predicted by the HGM is overlaid in red, the resulting residual in purple. Gray bars indicate periods of reversals. 43
4.17. **Single-neuron residual prediction examples.** Similar to Figure 4.16, the residual activity of AVAL and DD01 as predicted by the HGM is shown in purple. The respective predictions of such residual activity by the HGM are overlaid in green. Gray bars indicate periods of reversals. . 43
4.18. **HGM vs. MCM R^2 for all neurons with controls.** The plot shows MCM and HGM R^2's for all neurons, including all shuffle controls (pooled) for dataset 19. Significant neurons are colored green, non significant neurons gray, controls red. Significant neurons which are identified are labeled. 44
4.19. **HGM and MCM R^2 distributions for all datasets**. Distributions only take significant neurons into account. 44
4.20. **R^2 improvement against shuffle control R^2 improvement distribution sample p-value.** Showing dataset 19, and only significant neurons. Neurons whose improvement is significant are colored green, others gray. Neurons with significant improvement are labeled if they are identified. . . 46

4.21. R^2 **improvement distributions for all datasets**. The distributions only take significant neurons with significant improvement into account. (In dataset 9, all neurons happened to be filtered out by at least one of the two significance assessments.) 46

4.22. **Distribution of HGM R^2 medians for identified neurons, with GFP control, across datasets.** GFP control distribution in red, GCaMP in green. The black line indicates the 100 percentile threshold of the control distribution. Neurons that pass this threshold are labeled. 47

4.23. **Distribution of HGM R^2 improvements for identified neurons, across datasets.** Showing only significant neurons with respect to GFP control. Boxplot are placed on the x-axis at the median of their corresponding MCM R^2 distribution. Individual neuron R^2's are indicated by dots. The black line indicates the 100th percentile threshold of distribution of GFP control improvement medians. 48

4.24. **Manifold prediction examples for modulated neurons.** Excerpts of the neuronal traces of DB01 and VB02 are shown in blue. The respective manifold component as predicted by the HGM is overlaid in red, the resulting residual in purple. Transparent red/purple denote the respective traces as resulting from the MCM. Gray bars indicate periods of reversals. 49

4.25. **Residual prediction examples for modulated neurons.** Similar to Figure 4.24, the residual activity of DB01 and VB02 is shown in purple. The respective predictions of such residual activity by the HGM is overlaid in green, the respective gating term prediction in black. Transparent green denotes the residual prediction as resulting from the MCM. Gray bars indicate periods of reversals. A corresponding plot for the (unmodulated) neuron DD01 can be found in the appendix (Figure A.5). 49

4.26. **HGM manifolds.** Color coding and labelling is the same as in Figure 4.8. Latent dimension is also set to 2. 50

4.27. **HGM manifolds by gating of different neurons.** Brighter colors indicate a larger sigmoid term and therefore less gating. Plots are titled by the neuron that is modulated in the depicted way. The respective improvement of their R^2 is given in the brackets, followed by 'sig' if the improvement was significant with respect to the shuffle controls. We show the same neurons for both datasets except for the middle column, where we picked DD01 instead of DB01 for dataset 14. 51

4.28. **Gating subspace.** The black traces depict the modes of the gating subspace itself. Projections of the residual predictions in green, projections of the actual residuals in purple. Transparent lines denote the SS-MCM prediction, SS-HGM in opaque. As always, gray bars indicate periods of reversals. The oscillations appearing centered below 0 is merely a side effect of transforming data with PCA fitted on other data. 53

4.29. **HGM manifolds by different gating modes.** Only the first three PC's are shown, as the others behave like PC 3, capturing no relevant gating activity. 54
4.30. **Gating dimensionality across datasets.** Average cumulative variance explained maxes out at 2. 54
4.31. ***Drosophila* manifolds.** Latent dimension is set to 4. 57
4.32. **State transitions.** Transition probabilities are averaged across datasets. Reflexive transitions are excluded. 57
4.33. ***Drosophila* trace heatmap.** Neurons are sorted by PCA loadings. Discrete states are sorted manually. 'still' refers to quiescence, 'walk' refers to running. 58
4.34. ***Drosophila* residual heatmap.** Neurons are sorted by CCA loadings. Markers are inversely sorted by CCA target loadings. 59
4.35. **Marker vs. Neuron timescale.** The example neuron's time-scale is representative of the general neuronal residual dynamics. 59
4.36. ***Drosophila* MCO ratios.** Color coding is the same as in Figure 4.11. Plot only shows data for dataset 1, as missing neuron ID's do not allow cross-dataset mean and standard deviation analysis. 61
4.37. **Hyperparameter sweeps for *Drosophila*.** Labeling is the same as in Figure 4.12. The validation loss (y-axis) is consistently minimized at values of 1 for W_B. This indicates that the model did not find the desired structure in the data and performs best by forcing the weights to be 0, simply predicting a flat line average. This also renders the result of the W_Z sweep rather meaningless. 62
4.38. ***Drosophila* HGM vs. MCM R^2.** Not a single neuron is significant. . . 63
4.39. **Residual prediction example.** As always, the residual activity is shown in purple. The residual prediction by the HGM is overlaid in green, MCM in transparent green. Gray bars indicate periods of quiescence. 63
4.40. **Exponential filter to "reconvolve" spikes.** The gray trace is the GCaMP data as deconvolved in [SPS+19]. Blue is the result of the exponential filter, transforming it back into continuous calcium dynamics. The continuous gray bars indicate pupil area. 65
4.41. **Mouse manifold.** Latent dimensionality is set to 16. Manifolds are colored by the arousal variables, with brighter colors corresponding to larger values. 66
4.42. **Mouse trace heatmap.** The first 1000 traces are shown, as sorted by PCA loadings. The three general arousal variables are sorted manually. . 67
4.43. **Mouse residual heatmap** The first 1000 traces are shown, as sorted by CCA loadings. The 10 best-fit face motion PCs are shown, sorted by CCA target loadings. 68
4.44. **Mouse MCO ratios.** Color coding is the same as in Figure 4.36. Plot only shows data for dataset 2. The latent dimension of CCA between residuals and face motion PCs is set to 16. 69

4.45. **SS-Model sweeps.** Both models show validation loss minima for all hyperparameters. Due to computational limitations, only one dataset was used for sweeping. 70
4.46. **SS-HGM R^2 control percentile distribution over all neurons.** In this dataset (ID 2), about a third of all neurons land in the 100th percentile. 72
4.47. **SS-HGM R^2 control percentile distributions for all datasets.** The boxplots show the distribution of HGM R^2 control percentiles for all datasets. The upper bars show the respective relative share of significant neurons, i.e. neurons that land in the 100th percentile of HGM controls and having an R^2 above zero for both models. 72
4.48. **SS-HGM vs. SS-MCM R^2 for all neurons with controls.** Showing all neurons, including all shuffle controls (pooled) for dataset 2. Significant neurons are colored green, non significant neurons gray, controls red. . . . 73
4.49. **SS-HGM and SS-MCM R^2 distributions for all datasets.** Distributions only take significant neurons into account. 73
4.50. **R^2 improvement against shuffle control R^2 improvement distribution sample p-value.** The plot shows only significant neurons. Neurons whose improvement/non-improvement is significant are colored green/blue, others gray. 74
4.51. **R^2 improvement distributions for all datasets.** The distributions only take significant neurons with significant improvement into account. . 74
4.52. **SS-HGM Manifold prediction examples.** Showing a few examples of high R^2 neurons. Original traces in blue, manifold components in red, residuals in purple. Transparent lines denote the SS-MCM prediction, SS-HGM in opaque. Gray bars represent pupil area. 76
4.53. **SS-HGM Residual predictions examples.** Showing a few examples of high R^2 neurons. Residuals in purple, residual predictions in green. Transparent lines denote the SS-MCM prediction, SS-HGM in opaque. The subspace gating term when projected into the single-neuron space is indicated in black. Gray bars represent pupil area. 76
4.54. **SS-HGM manifolds.** Manifolds are colored by the arousal variables, with brighter colors corresponding to larger values. 77
4.55. **Gating subspace.** Black traces depict gating modes. Residual prediction modes in green, actual residual modes in purple. Transparent lines denote SS-MCM prediction, SS-HGM in opaque. Gray bars show pupil area. The off-centering of the traces is a side effect of transforming data with PCA fitted on other data. . . 78
4.56. **SS-HGM manifolds by different gating modes.** Manifolds are colored by modulation subspace activity, with brighter colors corresponding to larger values. Only the first 6 out of all 16 PC's are shown, as modulation only happens in the first few. 79
4.57. **Gating dimensionalities across datasets.** Zooming in on the top 10% (right) shows the average cumulative variance explained continuing to weakly increase up until dimension 4. 79

5.1. **MAE vs. MSE.** When adding the flat errors for both targets (L_1-norm), the minimum is achieved by concentrating on E_2. When adding the squared losses however (L_2-norm), the optimal parameters fall somewhere between satisfying E_1 and E_2. 84
5.2. **Stratified k-Fold Cross-Validation.** Adapted from the SciKit-Learn documentation ([PVG+11]) 89

A.1. ***C. elegans* HGM & MCM train losses.** HGM opaque, MCM transparent. Some loss curves show discontinuity, suddenly jumping to a larger value. This is due to early stopping reloading the parameters of the last best validation loss. 117
A.2. ***C. elegans* PCA manifolds.** 118
A.3. ***C. elegans* HGM manifolds by gating of significantly improved neurons.** Showing dataset 19. 119
A.4. ***C. elegans* HGM manifolds by gating of significantly improved neurons.** Showing dataset 14. 120
A.5. ***C. elegans* DD01 residual prediction example.** 121
A.6. ***C. elegans* SS-HGM R^2 percentile distributions across datasets.** . 121
A.7. ***C. elegans* SS-HGM & SS-MCM R^2 distributions across datasets.** 121
A.8. ***C. elegans* SS-Model R^2 improvement distributions across datasets.** 121
A.9. ***C. elegans* behavioral subspace.** Showing dataset 19. 122
A.10. ***C. elegans* gating subspace.** Showing dataset 14. 123
A.11. ***Drosophila* raw trace heatmaps.** 124
A.12. ***Drosophila* PCA manifolds.** 125
A.13. ***Drosophila* high-level state distributions.** 126
A.14. ***Drosophila* PCA of hind leg marker.** 126
A.15. ***Drosophila* residual regression from markers without cross-validation.** Showing best-scoring residual, in purple. Gradients in orange. 126
A.16. ***Drosophila* Hyperparameter sweeps for subspace models.** 127
A.17. **Mouse Hyperparameter sweeps for standard models.** 127
A.18. **Mouse SS-HGM & SS-MCM train losses.** SS-HGM opaque, SS-MCM transparent. 128
A.19. **Mouse PCA manifolds.** Colored by pupil area. 128
A.20. **Mouse SS-HGM behavioral subspace.** Showing dataset 2. 129
A.21. **Mouse neuron positions, colored by improvement.** Brighter green denotes stronger improvement. Non-significant neurons in gray. 130

List of Tables

3.1. **Model landscape**. Vertical axis categorizes by modulation complexity, horizontal axis categorizes by captured biological features. The displayed models cover a mixture of different approaches, some modeling time dynamics while others treat time points as independent samples. In addition, black entries indicate modelling activity from latent variables (i.e. making use of manifolds) while gray entries do not. Behavior is modeled in a multiplexed manner unless stated otherwise. 21

A.1. **Dataset research results.** Accessible at `https://docs.google.com/spreadsheets/d/19kfHLkAmDi7FF740qnZWUh5K01zMpTaxnTF5DWbqjoc` . . 116

Bibliography

[AKJ+19] Sophie Aimon, Takeo Katsuki, Tongqiu Jia, Logan Grosenick, Michael Broxton, Karl Deisseroth, Terrence J. Sejnowski, and Ralph J. Greenspan. Fast near-whole–brain imaging in adult Drosophila during responses to stimuli and behavior. *PLOS Biology*, 17(2):e2006732, February 2019.

[ALS+25] Shailaja Akella, Peter Ledochowitsch, Joshua H. Siegle, Hannah Belski, Daniel D. Denman, Michael A. Buice, Severine Durand, Christof Koch, Shawn R. Olsen, and Xiaoxuan Jia. Deciphering neuronal variability across states reveals dynamic sensory encoding. *Nature Communications*, 16(1):1768, February 2025.

[Ben66] M. V. L. Bennett. Physiology of Electrotonic Junctions. *Annals of the New York Academy of Sciences*, 137(2):509–539, 1966.

[BFH+18] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

[Bie20] Lukas Biewald. Experiment tracking with weights and biases, 2020. Software available from wandb.com.

[But30] S. Butterworth. On the Theory of Filter Amplifiers. *Experimental Wireless & the Wireless Engineer*, 7:536–541, October 1930.

[CCK+12] Mark M. Churchland, John P. Cunningham, Matthew T. Kaufman, Justin D. Foster, Paul Nuyujukian, Stephen I. Ryu, and Krishna V. Shenoy. Neural population dynamics during reaching. *Nature*, 487(7405):51–56, July 2012.

[CSL+24] Michał Ciszewski, Jakob Söhl, Ton Leenen, Bart van Trigt, and Geurt Jongbloed. Testing for no effect in regression problems: A permutation approach, April 2024.

[CT08] John Y. Campbell and Samuel B. Thompson. Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average? *The Review of Financial Studies*, 21(4):1509–1531, July 2008.

[DMS+24] Sven Dorkenwald, Arie Matsliah, Amy R. Sterling, Philipp Schlegel, Szichieh Yu, Claire E. McKellar, Albert Lin, Marta Costa, Katharina Eichler,

Yijie Yin, Will Silversmith, Casey Schneider-Mizell, Chris S. Jordan, Derrick Brittain, Akhilesh Halageri, Kai Kuehner, Oluwaseun Ogedengbe, Ryan Morey, Jay Gager, Krzysztof Kruk, Eric Perlman, Runzhe Yang, David Deutsch, Doug Bland, Marissa Sorek, Ran Lu, Thomas Macrina, Kisuk Lee, J. Alexander Bae, Shang Mu, Barak Nehoran, Eric Mitchell, Sergiy Popovych, Jingpeng Wu, Zhen Jia, Manuel A. Castro, Nico Kemnitz, Dodam Ih, Alexander Shakeel Bates, Nils Eckstein, Jan Funke, Forrest Collman, Davi D. Bock, Gregory S. X. E. Jefferis, H. Sebastian Seung, and Mala Murthy. Neuronal wiring diagram of an adult brain. *Nature*, 634(8032):124–138, October 2024.

[EH21] R. Becket Ebitz and Benjamin Y. Hayden. The population doctrine in cognitive neuroscience. *Neuron*, 109(19):3055–3068, October 2021.

[FLR+25] Charles Fieseler, Itamar Lev, Ulises Rey, Lukas Hille, Hannah Brenner, and Manuel Zimmer. An intrinsic neuronal manifold underlies brain-wide hierarchical organization of behavior in C. elegans, March 2025.

[FZP17] Johannes Friedrich, Pengcheng Zhou, and Liam Paninski. Fast online deconvolution of calcium imaging data. *PLOS Computational Biology*, 13(3):e1005423, March 2017.

[GMS14] Robbe L. T. Goris, J. Anthony Movshon, and Eero P. Simoncelli. Partitioning neuronal variability. *Nature Neuroscience*, 17(6):858–865, June 2014.

[GVL96] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 3 edition, 1996.

[HMvdW+20] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[HTF09a] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference and prediction*. Springer, 2 edition, 2009.

[HTF09b] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009.

[JFL+] Hongfei Ji, Anthony D. Fouad, Zihao Li, Andrew Ruba, and Christopher Fang-Yen. A proprioceptive feedback circuit drives Caenorhabditis elegans locomotor adaptation through dopamine signaling. *Proceedings of the National Academy of Sciences of the United States of America*, 120(20):e2219341120.

[Jol02] I. T. Jolliffe. *Principal Component Analysis*. Springer Series in Statistics. Springer-Verlag, New York, 2002.

[Kah11] Daniel Kahneman. *Thinking, fast and slow*. Farrar, Straus and Giroux, New York, 2011.

[KB15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[Kha22] Anjul Khadria. Tools to measure membrane potential of neurons. *Biomedical Journal*, 45(5):749–762, October 2022.

[KKS+15] Saul Kato, Harris S. Kaplan, Tina Schrödel, Susanne Skora, Theodore H. Lindsay, Eviatar Yemini, Shawn Lockery, and Manuel Zimmer. Global Brain Dynamics Embed the Motor Command Sequence of Caenorhabditis elegans. *Cell*, 163(3):656–669, October 2015.

[KM65] B. Katz and R. Miledi. The Measurement of Synaptic Delay, and the Time Course of Acetylcholine Release at the Neuromuscular Junction. *Proceedings of the Royal Society of London. Series B, Biological Sciences*, 161:483–495, February 1965.

[KTKZ20] Harris S. Kaplan, Oriana Salazar Thula, Niklas Khoss, and Manuel Zimmer. Nested Neuronal Dynamics Orchestrate a Behavioral Hierarchy across Timescales. *Neuron*, 105(3):562–576.e9, February 2020.

[KW78] Jae-On Kim and Charles W.Mueller. *Introduction to Factor Analysis*. SAGE Publications, Inc., 1978.

[KZ20] Harris S Kaplan and Manuel Zimmer. Brain-wide representations of ongoing behavior: A universal principle? *Current Opinion in Neurobiology*, 64:60–69, October 2020.

[LNB+19] Scott Linderman, Annika Nichols, David Blei, Manuel Zimmer, and Liam Paninski. Hierarchical recurrent state space models reveal discrete and continuous dynamics of neural activity in C. elegans, April 2019.

[LS16] Michael Z Lin and Mark J Schnitzer. Genetically encoded indicators of neuronal activity. *Nature neuroscience*, 19(9):1142–1153, August 2016.

[LWH+22] Albert Lin, Daniel Witvliet, Luis Hernandez-Nunez, Scott W. Linderman, Aravinthan D. T. Samuel, and Vivek Venkatachalam. Imaging whole-brain activity to understand behaviour. *Nature Reviews Physics*, 4(5):292–305, May 2022.

[Mar12] Eve Marder. Neuromodulation of Neuronal Circuits: Back to the Future. *Neuron*, 76(1):1–11, October 2012.

[MKJ+19] Simon Musall, Matthew T. Kaufman, Ashley L. Juavinett, Steven Gluf, and Anne K. Churchland. Single-trial neural dynamics are dominated by richly varied movements. *Nature Neuroscience*, 22(10):1677–1686, October 2019.

[OC22] Tess B. Oram and Gwyneth M. Card. Context-dependent control of behavior in *Drosophila*. *Current Opinion in Neurobiology*, 73:102523, April 2022.

[PVG+11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[RW05] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, November 2005.

[SC19] Shreya Saxena and John P Cunningham. Towards the neural population doctrine. *Current Opinion in Neurobiology*, 55:103–111, April 2019.

[SK11] Ian H. Stevenson and Konrad P. Kording. How advances in neural recording affect data analysis. *Nature Neuroscience*, 14(2):139–142, February 2011.

[SMW+23] Evan S. Schaffer, Neeli Mishra, Matthew R. Whiteway, Wenze Li, Michelle B. Vancura, Jason Freedman, Kripa B. Patel, Venkatakaushik Voleti, Liam Paninski, Elizabeth M. C. Hillman, L. F. Abbott, and Richard Axel. The spatial and temporal structure of neural activity across the fly brain. *Nature Communications*, 14(1):5572, September 2023.

[SPS+19] Carsen Stringer, Marius Pachitariu, Nicholas Steinmetz, Charu Bai Reddy, Matteo Carandini, and Kenneth D. Harris. Spontaneous behaviors drive multidimensional, brainwide activity. *Science*, 364(6437):eaav7893, April 2019.

[UDLC22] Anne E. Urai, Brent Doiron, Andrew M. Leifer, and Anne K. Churchland. Large-scale neural recordings call for new insights to link brain and behavior. *Nature Neuroscience*, 25(1):11–19, January 2022.

[VGO+20] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[WB17] Matthew R. Whiteway and Daniel A. Butts. Revealing unobserved factors underlying cortical activity with a rectified latent variable model applied to neural population recordings. *Journal of Neurophysiology*, 117(3):919–936, March 2017.

[Weg00] Jacob A. Wegelin. A survey of partial least squares (pls) methods, with emphasis on the two-block case. Technical Report 371, University of Washington, Department of Statistics, Seattle, WA, 2000. Technical Report.

[WPH+12] Quan Wen, Michelle D. Po, Elizabeth Hulme, Sway Chen, Xinyu Liu, Sen Wai Kwok, Marc Gershow, Andrew M. Leifer, Victoria Butler, Christopher Fang-Yen, Taizo Kawano, William R. Schafer, George Whitesides, Matthieu Wyart, Dmitri B. Chklovskii, Mei Zhen, and Aravinthan D. T. Samuel. Proprioceptive Coupling within Motor Neurons Drives C. elegans Forward Locomotion. *Neuron*, 76(4):750–761, November 2012.

[WSTB86] J. G. White, E. Southgate, J. N. Thomson, and S. Brenner. The structure of the nervous system of the nematode Caenorhabditis elegans. *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences*, 314(1165):1–340, November 1986.

[YCS+09] Byron M. Yu, John P. Cunningham, Gopal Santhanam, Stephen I. Ryu, Krishna V. Shenoy, and Maneesh Sahani. Gaussian-Process Factor Analysis for Low-Dimensional Single-Trial Analysis of Neural Population Activity. *Journal of Neurophysiology*, 102(1):614–635, July 2009.

A. Appendix

DATASETS	*data availability*	**available**	**available, but incomplete**	**"upon request"**	**no mention**
behavior quantification	*organism*				
Sufficient	**Fly**		**Schaffer, E. S. et al. The spatial and temporal structure of neural activity across the fly brain. Nat Commun 14, 5572 (2023).**		
	Fish		Cong, L. et al. Rapid whole brain imaging of neural activity in freely behaving larval zebrafish (Danio rerio). eLife 6, e28158 (2017).	**Marques, J. C., Li, M., Schaak, D., Robson, D. N. & Li, J. M. Internal state dynamics shape brainwide activity and foraging behaviour. Nature 577, 239–243 (2020).**	
	Mouse	**Stringer, C. et al. Spontaneous behaviors drive multidimensional, brainwide activity. Science 364, eaav7893 (2019).** **Musall, S., Kaufman, M. T., Juavinett, A. L., Gluf, S. & Churchland, A. K. Single-trial neural dynamics are dominated by richly varied movements. Nat Neurosci 22, 1677–1686 (2019).** **Vickers, E. D. & McCormick, D. A. Pan-cortical 2-photon mesoscopic imaging and neurobehavioral alignment in awake, behaving mice. eLife 13, RP94167 (2024).** **Meshulam, L. et al. A brain-wide map of neural activity during complex behaviour. Nature 645, 177–191 (2025).**	**Akella, S. et al. Deciphering neuronal variability across states reveals dynamic sensory encoding. Nat Commun 16, 1768 (2025).**		
Insufficient	**Fly**	Brezovec, B. E. et al. Mapping the neural dynamics of locomotion across the Drosophila brain. Current Biology 34, 710-726.e4 (2024).	Aimon, S. et al. Fast near-whole–brain imaging in adult Drosophila during responses to stimuli and behavior. PLOS Biology 17, e2006732 (2019).	Mann, K., Deny, S., Ganguli, S. & Clandinin, T. R. Coupling of activity, metabolism and behaviour across the Drosophila brain. Nature 593, 244–248 (2021).	Pacheco, D. A., Thiberge, S. Y., Pnevmatikakis, E. & Murthy, M. Auditory activity is diverse and widespread throughout the central brain of Drosophila. Nat Neurosci 24, 93–104 (2021).
	Fish	Chen, X. et al. Brain-wide Organization of Neuronal Activity and Convergent Sensorimotor Transformations in Larval Zebrafish. Neuron 100, 876-890.e5 (2018). Mu, Y. et al. Glia Accumulate Evidence that Actions Are Futile and Suppress Unsuccessful Behavior. Cell 178, 27-43.e19 (2019).	Ahrens, M. B. et al. Brain-wide neuronal dynamics during motor adaptation in zebrafish. Nature 485, 471–477 (2012). Dunn, T. W. et al. Brain-wide mapping of neural activity controlling zebrafish exploratory locomotion. eLife 5, e12741 (2016).	Kim, D. H. et al. Pan-neuronal calcium imaging with cellular resolution in freely swimming zebrafish. Nat Methods 14, 1107–1114 (2017). Yang, C. et al. A population code for spatial representation in the zebrafish telencephalon. Nature 634, 397–406 (2024).	
	Mouse	Das, A. et al. Large-scale recording of neuronal activity in freely-moving mice at cellular resolution. Nat Commun 14, 6399 (2023). Hope, J. et al. Brain-wide neural recordings in mice navigating physical spaces enabled by robotic neural recording headstages. Nat Methods 21, 2171–2181 (2024).		Jun, J. J. et al. Fully integrated silicon probes for high-density recording of neural activity. Nature 551, 232–236 (2017). Weisenburger, S. et al. Volumetric Ca2+ Imaging in the Mouse Brain Using Hybrid Multiplexed Sculpted Light Microscopy. Cell 177, 1050-1066.e14 (2019). Obaid, A. et al. Massively parallel microwire arrays integrated with CMOS chips for neural recording. Science Advances 6, eaay2789 (2020).	Melin, M. D. et al. Large scale, simultaneous, chronic neural recordings from multiple brain areas. 2023.12.22.572441 Preprint at https://doi.org/10.1101/2023.12.22.572441 (2024).

Table A.1.: **Dataset research results.** Accessible at `https://docs.google.com/spreadsheets/d/19kfHLkAmDi7FF740qnZWUh5K01zMpTaxnTF5DWbqjoc`

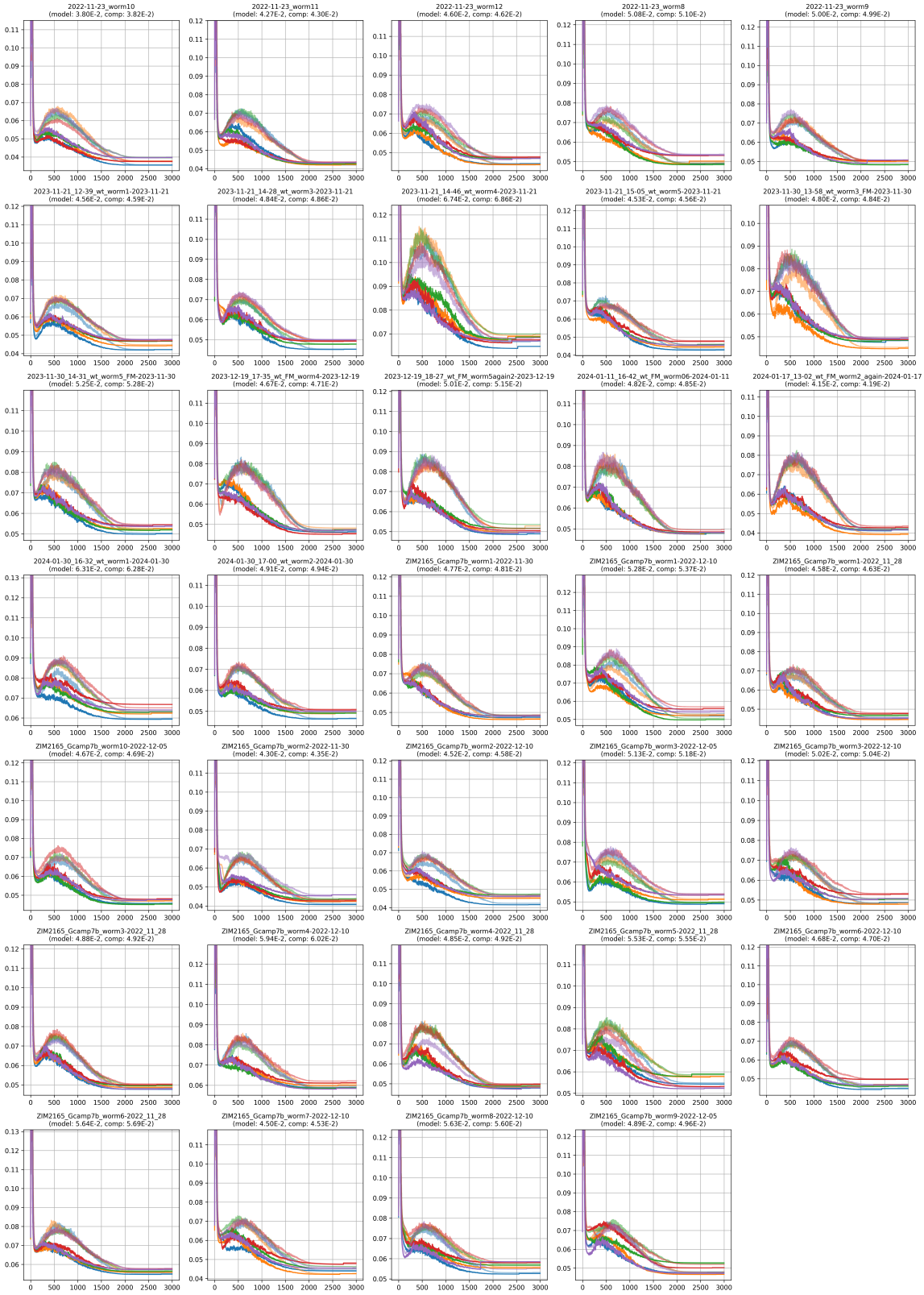


Figure A.1.: ***C. elegans* HGM & MCM train losses.** HGM opaque, MCM transparent. Some loss curves show discontinuity, suddenly jumping to a larger value. This is due to early stopping reloading the parameters of the last best validation loss.

A. Appendix

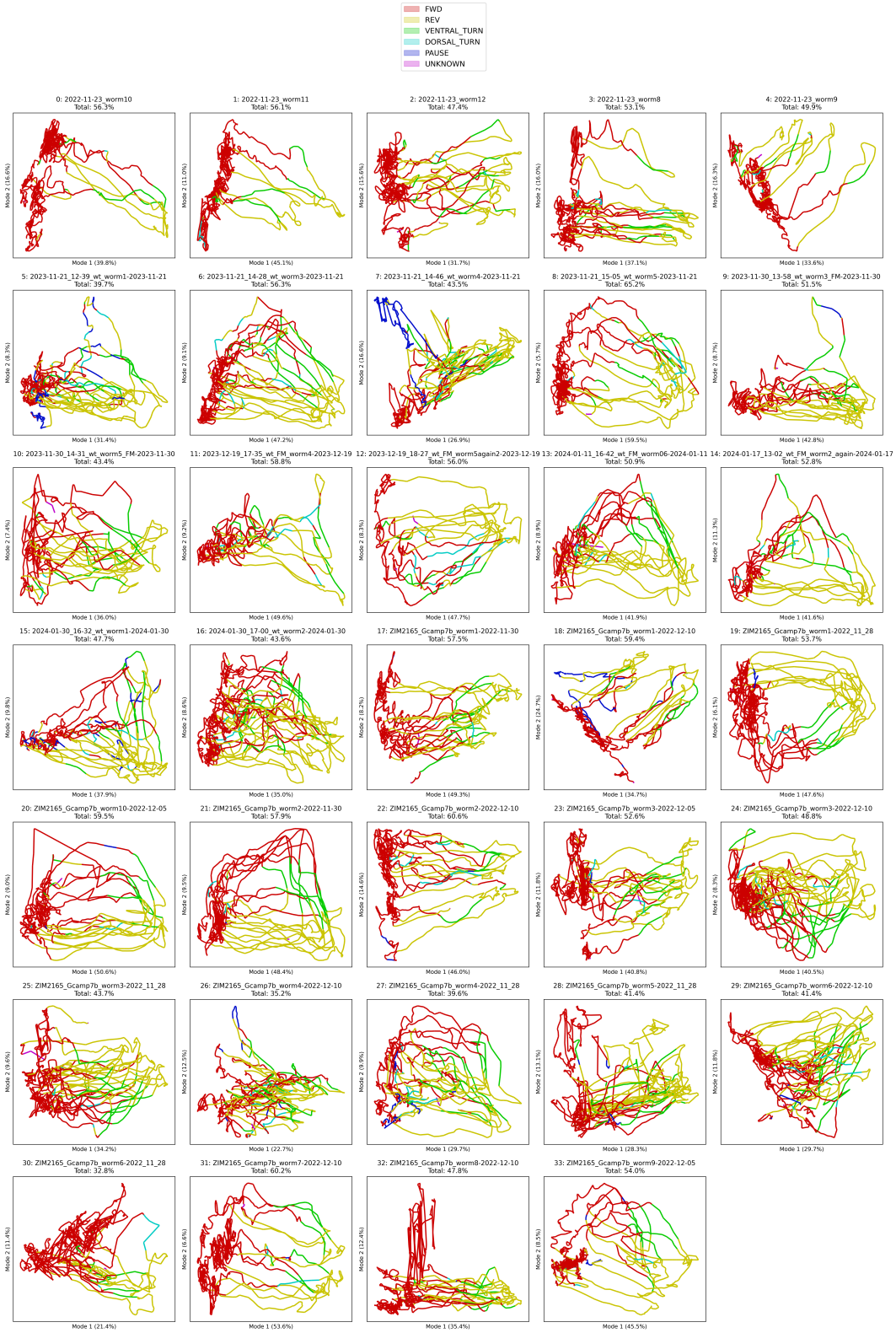


Figure A.2.: ***C. elegans* PCA manifolds.**

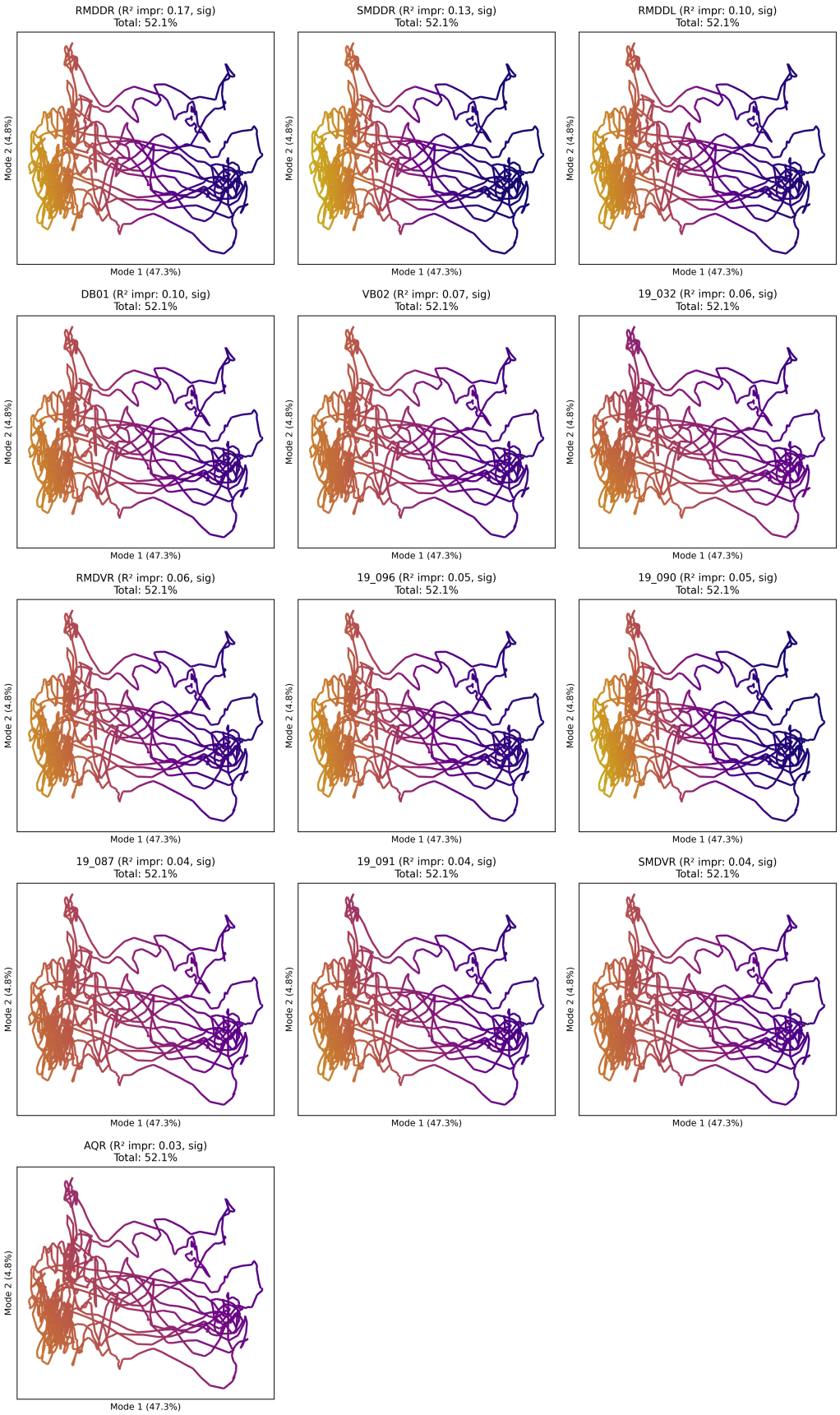


Figure A.3.: ***C. elegans* HGM manifolds by gating of significantly improved neurons.** Showing dataset 19.

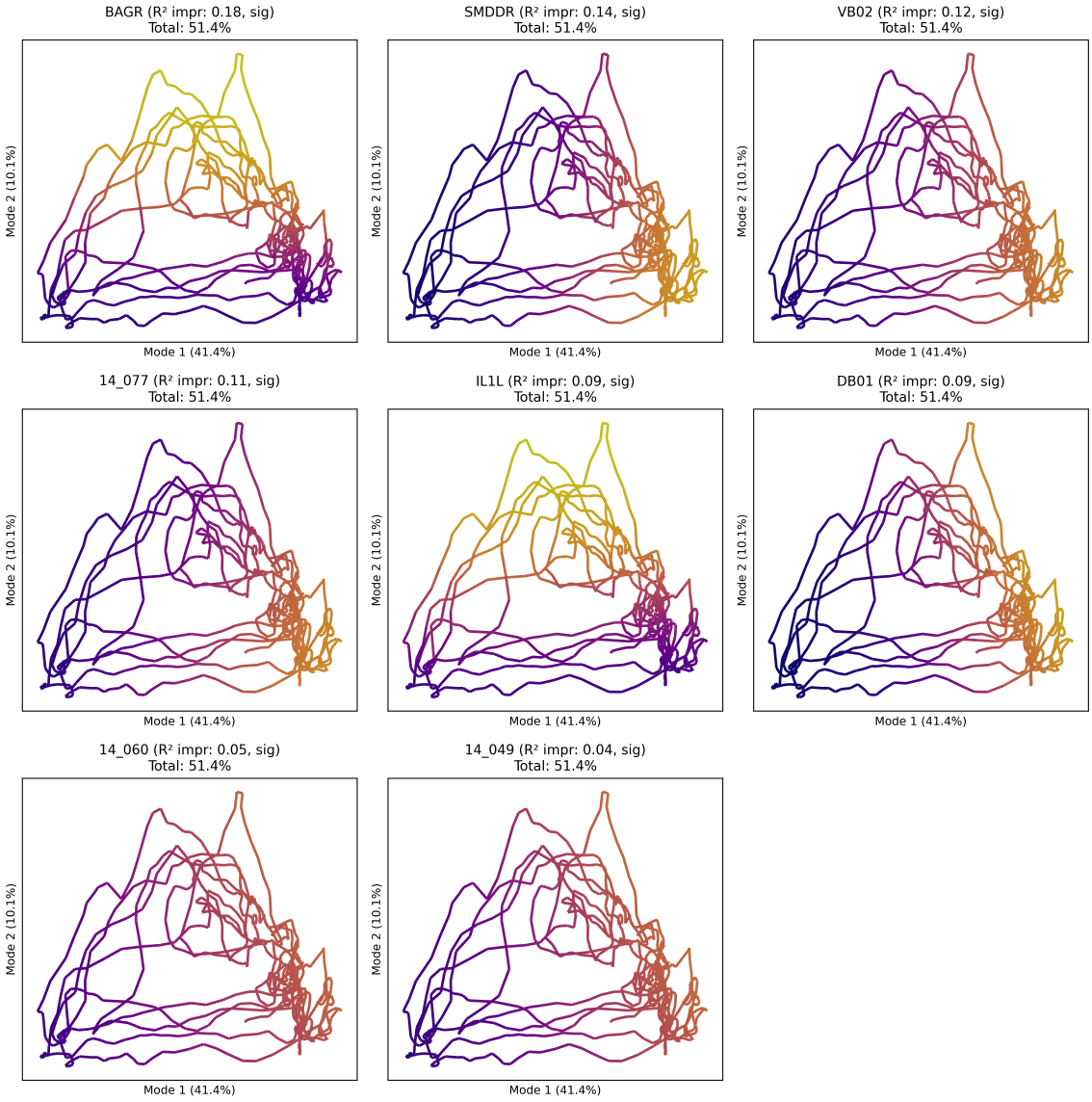


Figure A.4.: ***C. elegans* HGM manifolds by gating of significantly improved neurons.** Showing dataset 14.

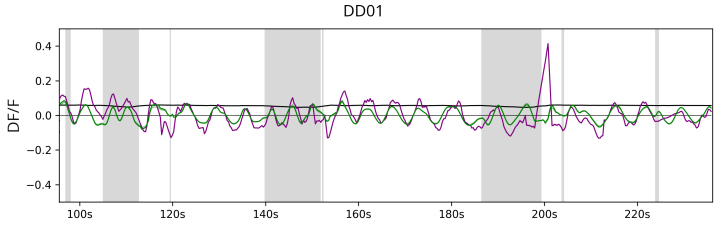


Figure A.5.: ***C. elegans* DD01 residual prediction example.**

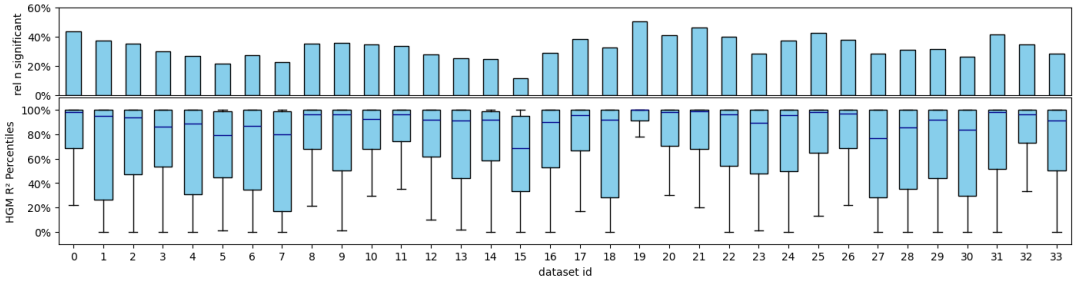


Figure A.6.: ***C. elegans* SS-HGM R^2 percentile distributions across datasets.**

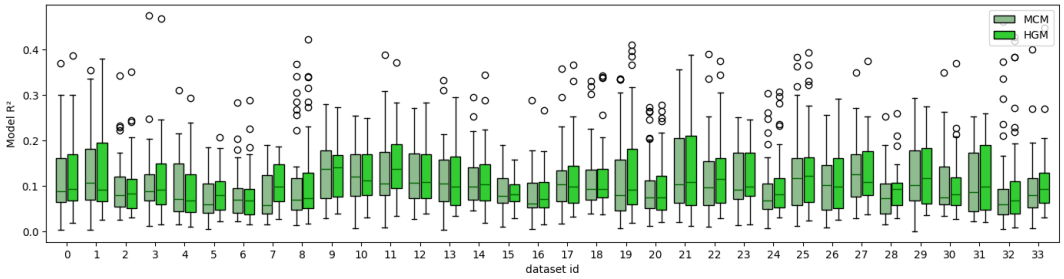


Figure A.7.: ***C. elegans* SS-HGM & SS-MCM R^2 distributions across datasets.**

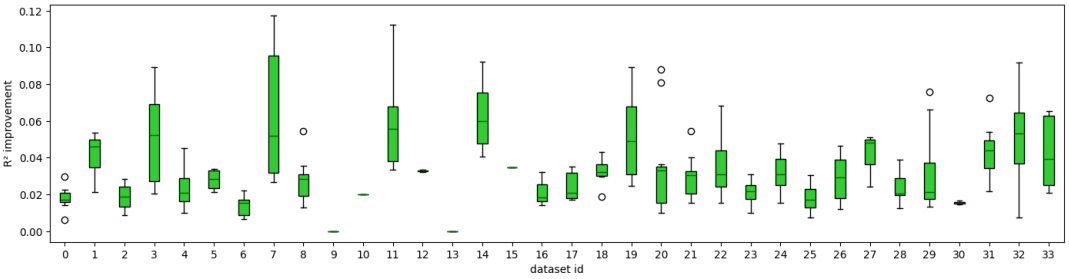


Figure A.8.: ***C. elegans* SS-Model R^2 improvement distributions across datasets.**

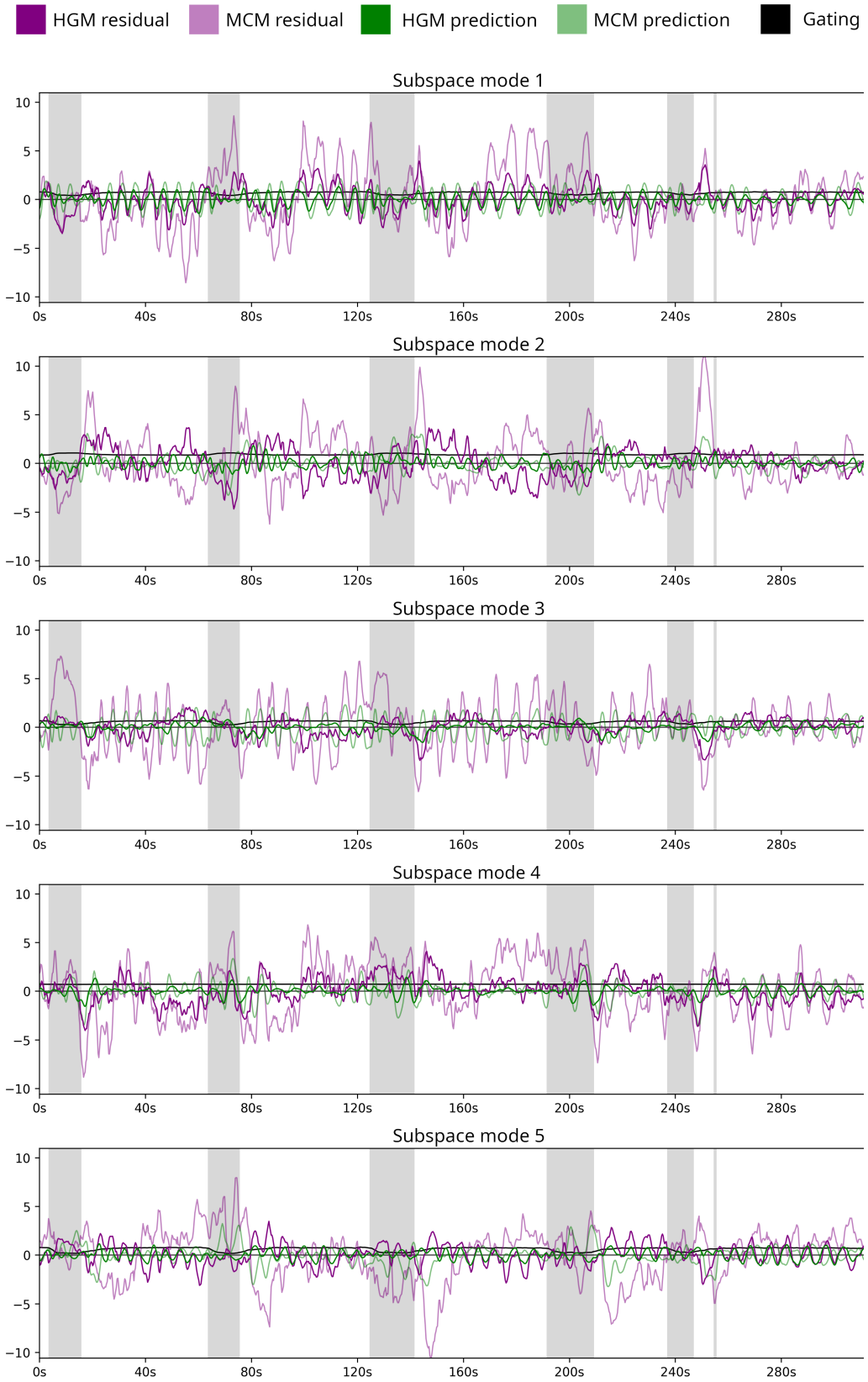


Figure A.9.: ***C. elegans* behavioral subspace.** Showing dataset 19.

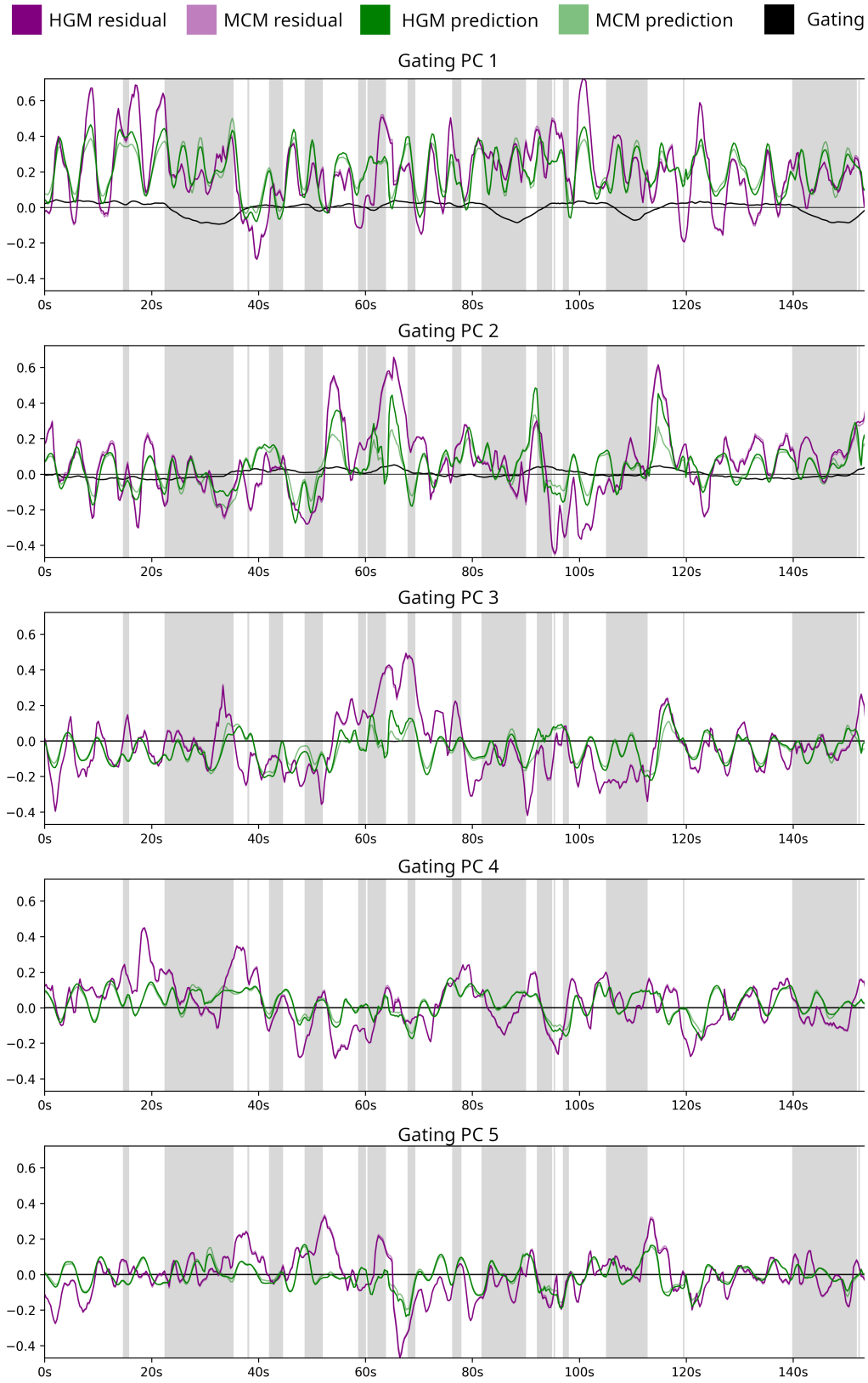


Figure A.10.: ***C. elegans* gating subspace.** Showing dataset 14.

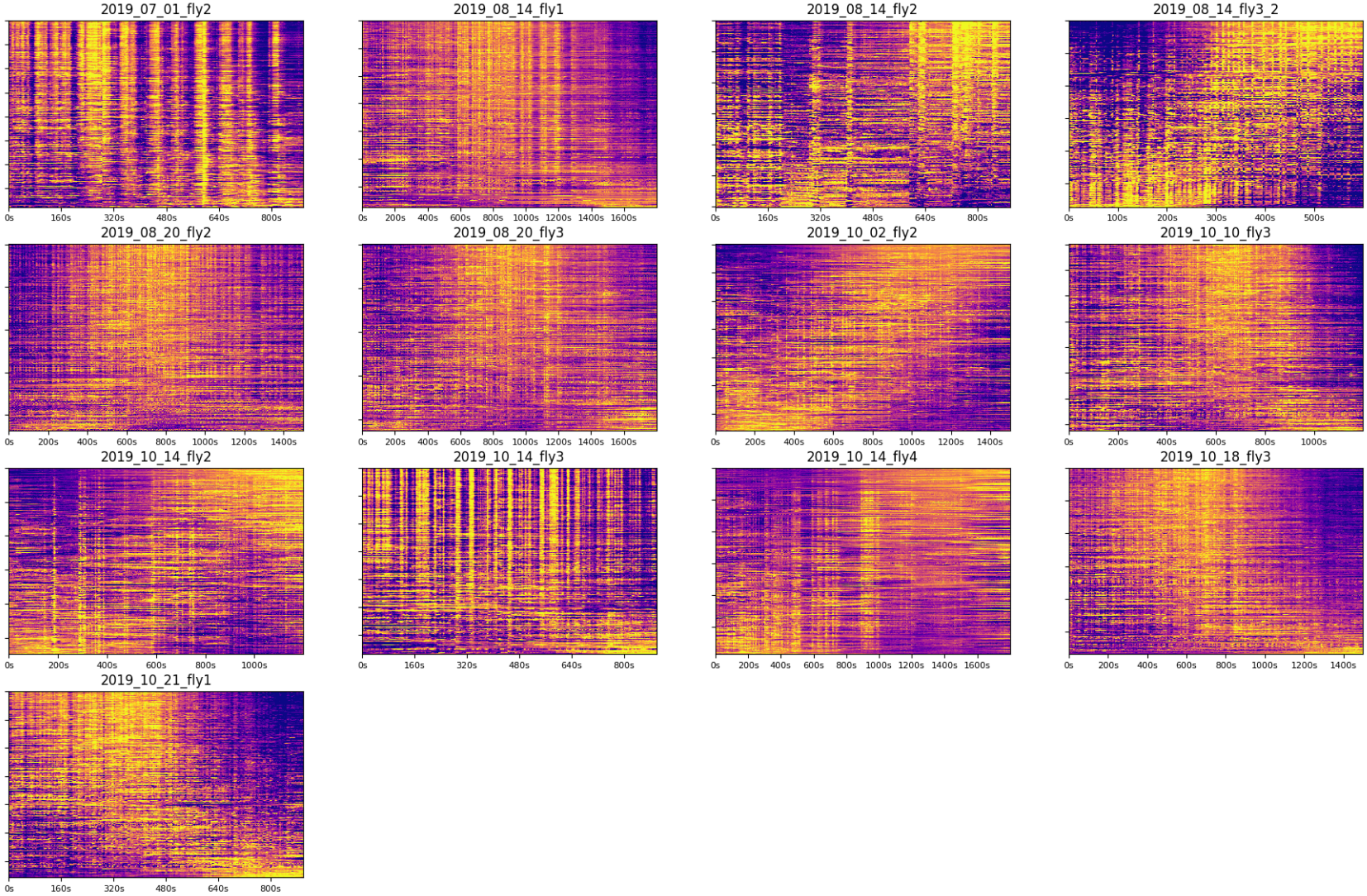


Figure A.11.: ***Drosophila* raw trace heatmaps.**

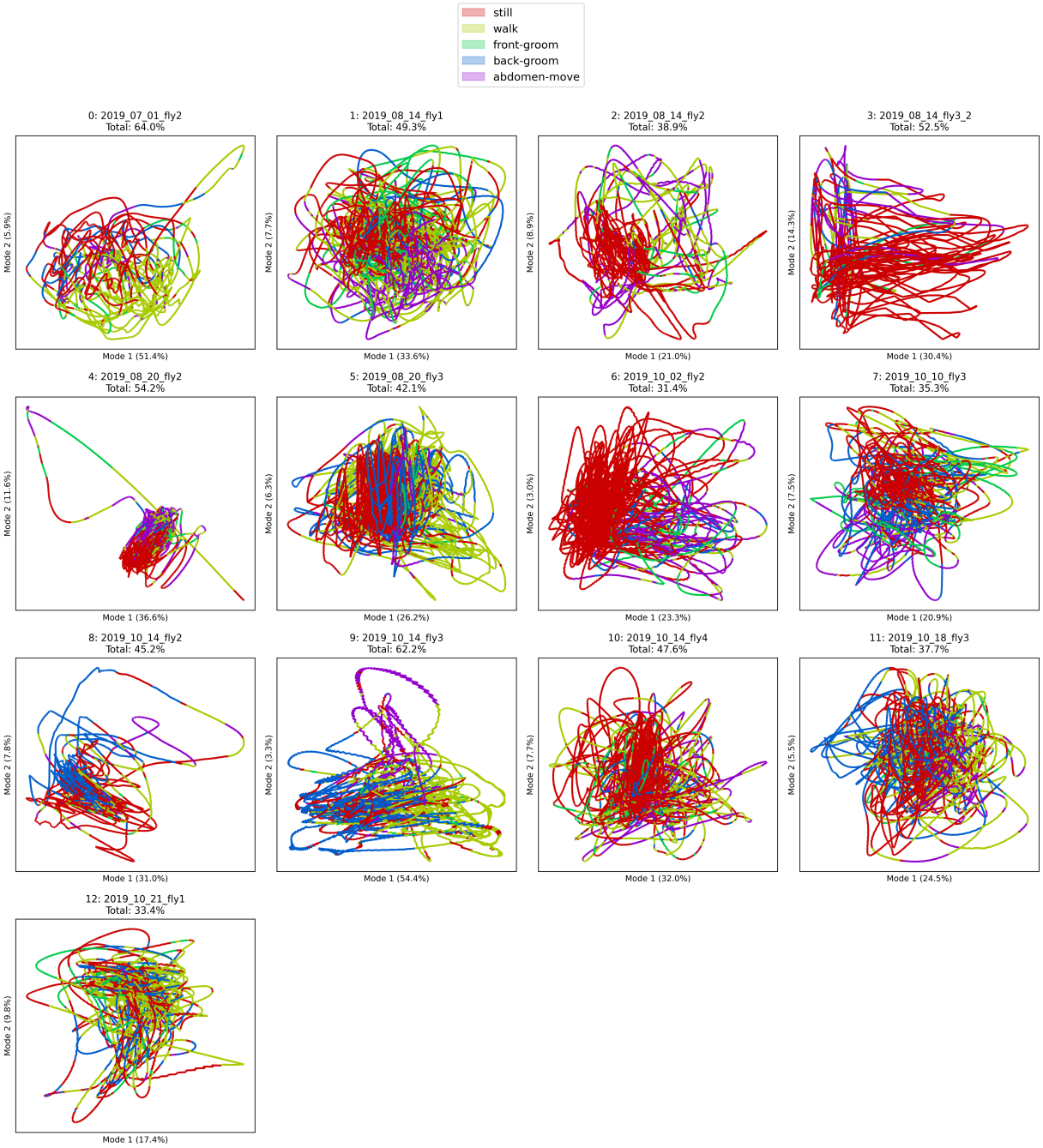


Figure A.12.: ***Drosophila* PCA manifolds.**

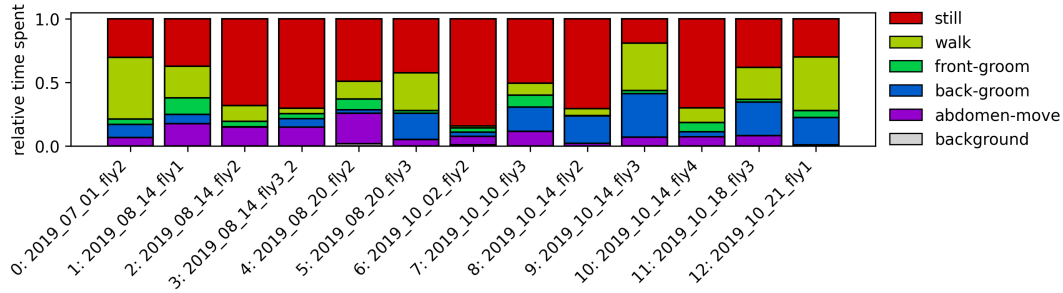


Figure A.13.: ***Drosophila* high-level state distributions.**

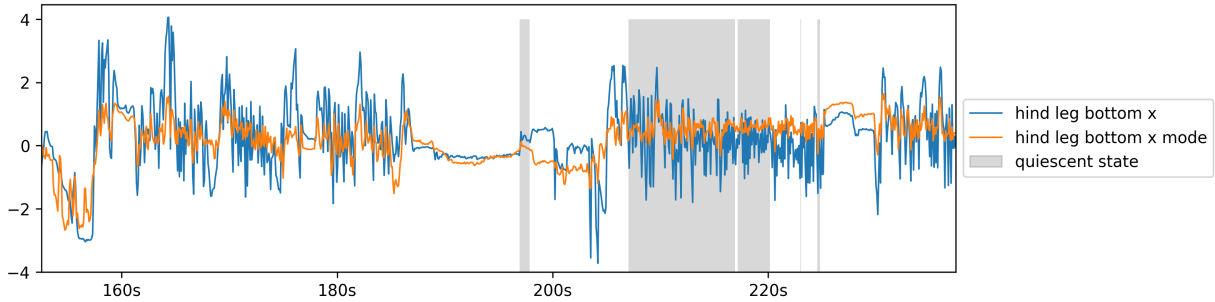


Figure A.14.: ***Drosophila* PCA of hind leg marker.**

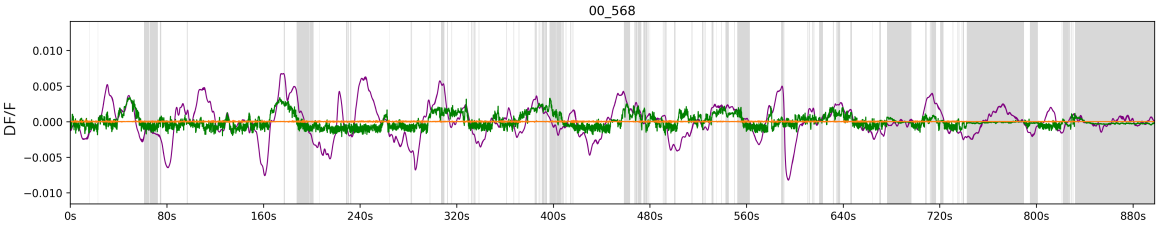


Figure A.15.: ***Drosophila* residual regression from markers without cross-validation.** Showing best-scoring residual, in purple. Gradients in orange.

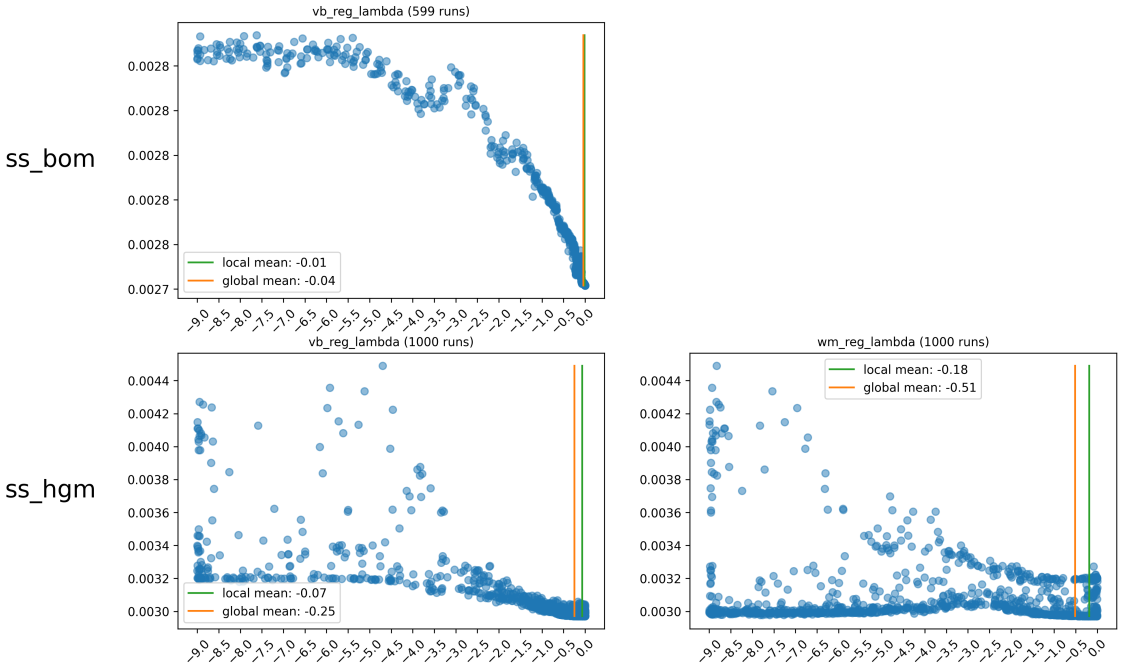


Figure A.16.: ***Drosophila*** **Hyperparameter sweeps for subspace models.**

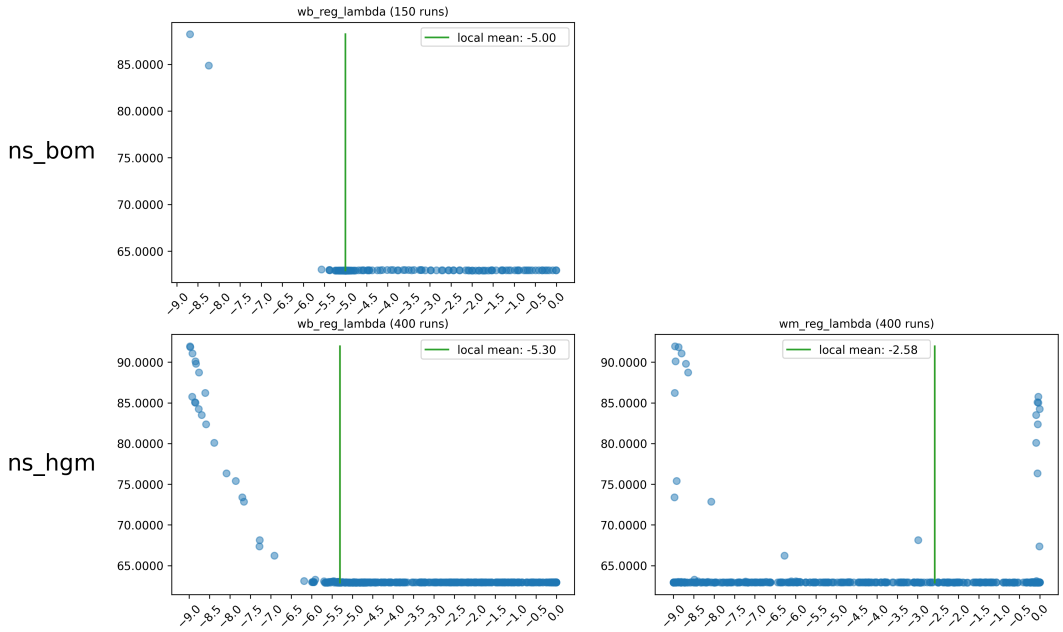


Figure A.17.: **Mouse Hyperparameter sweeps for standard models.**

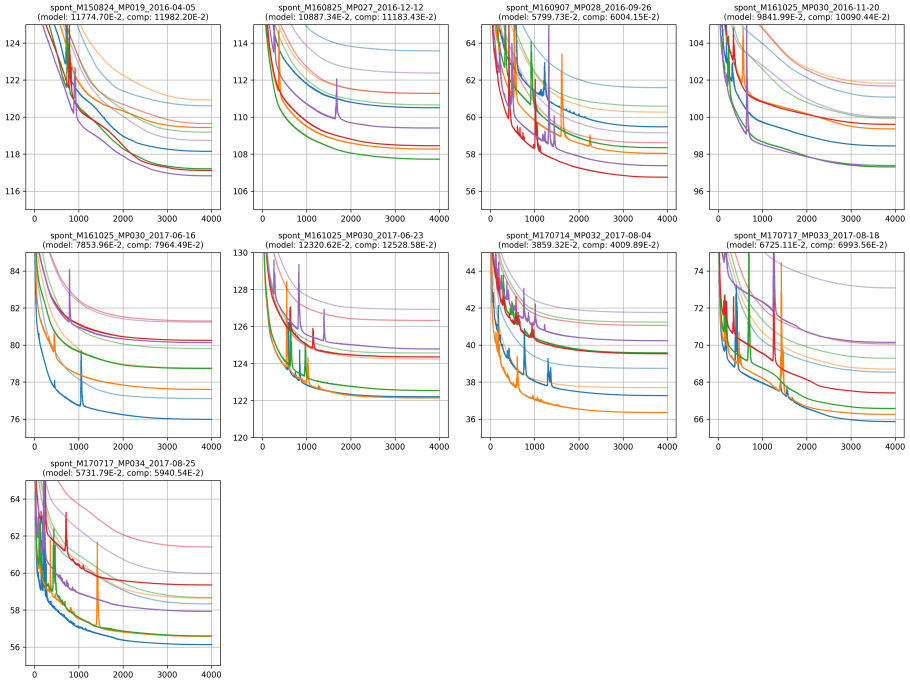


Figure A.18.: **Mouse SS-HGM & SS-MCM train losses.** SS-HGM opaque, SS-MCM transparent.

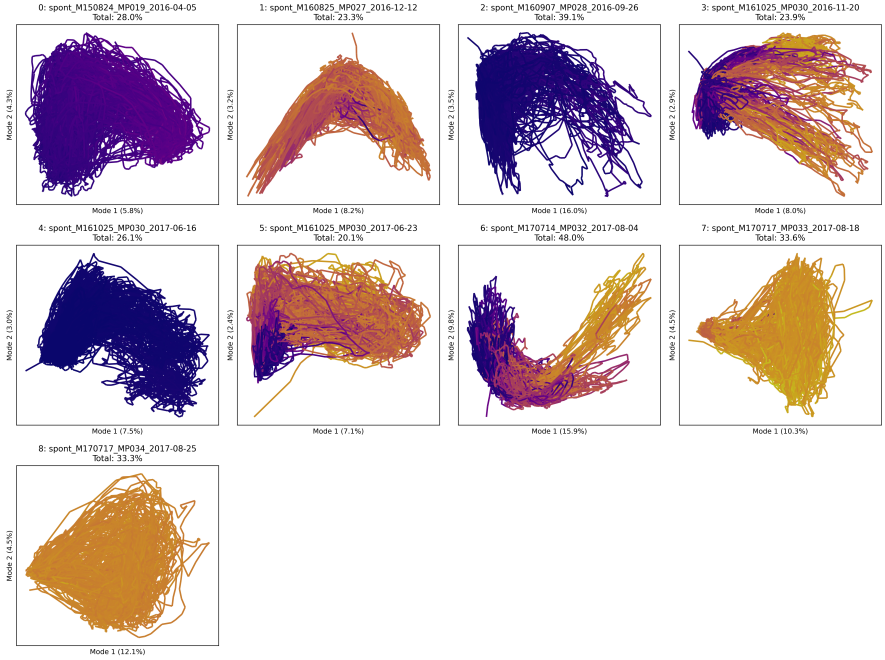


Figure A.19.: **Mouse PCA manifolds.** Colored by pupil area.

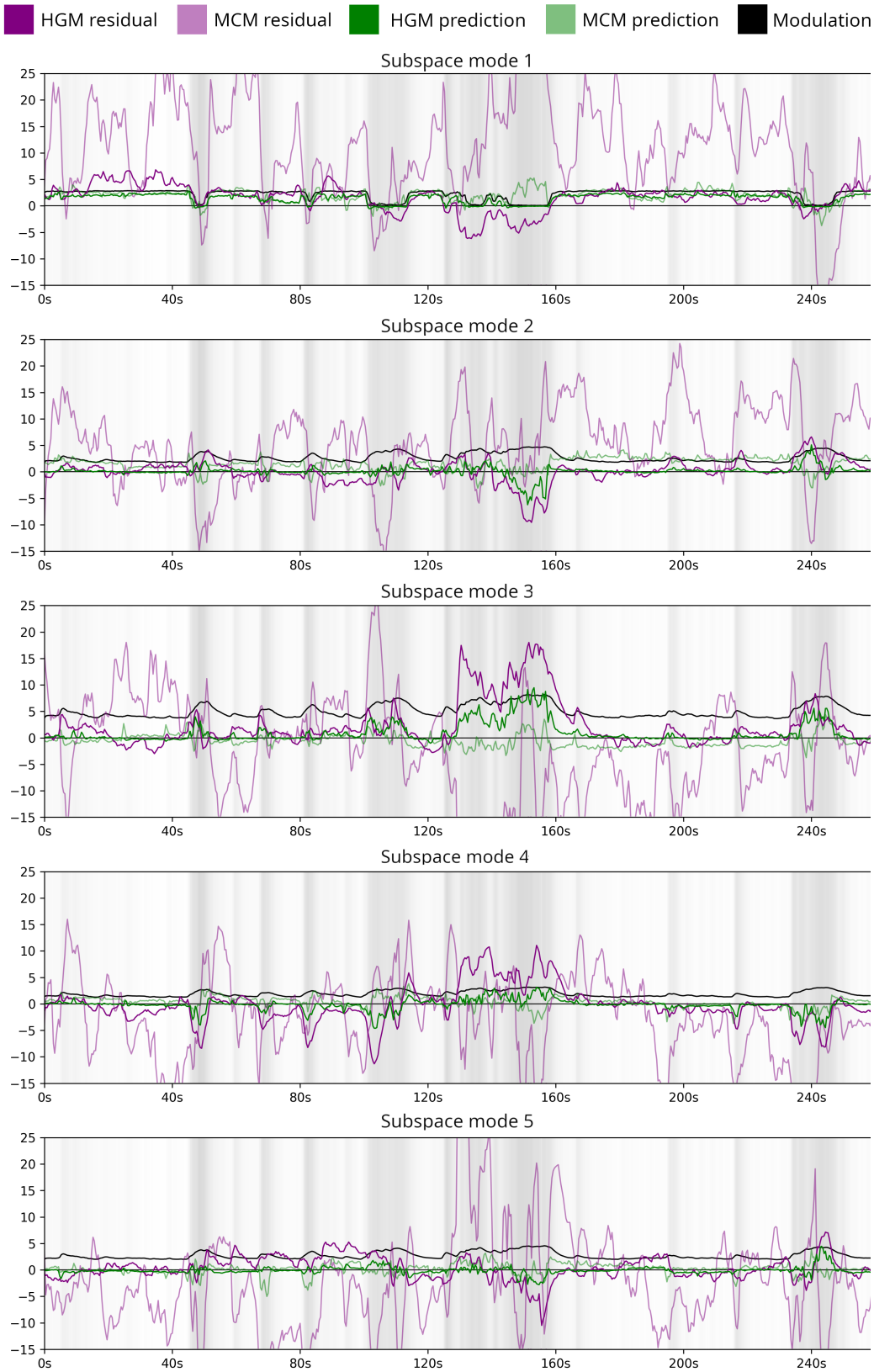


Figure A.20.: **Mouse SS-HGM behavioral subspace.** Showing dataset 2.

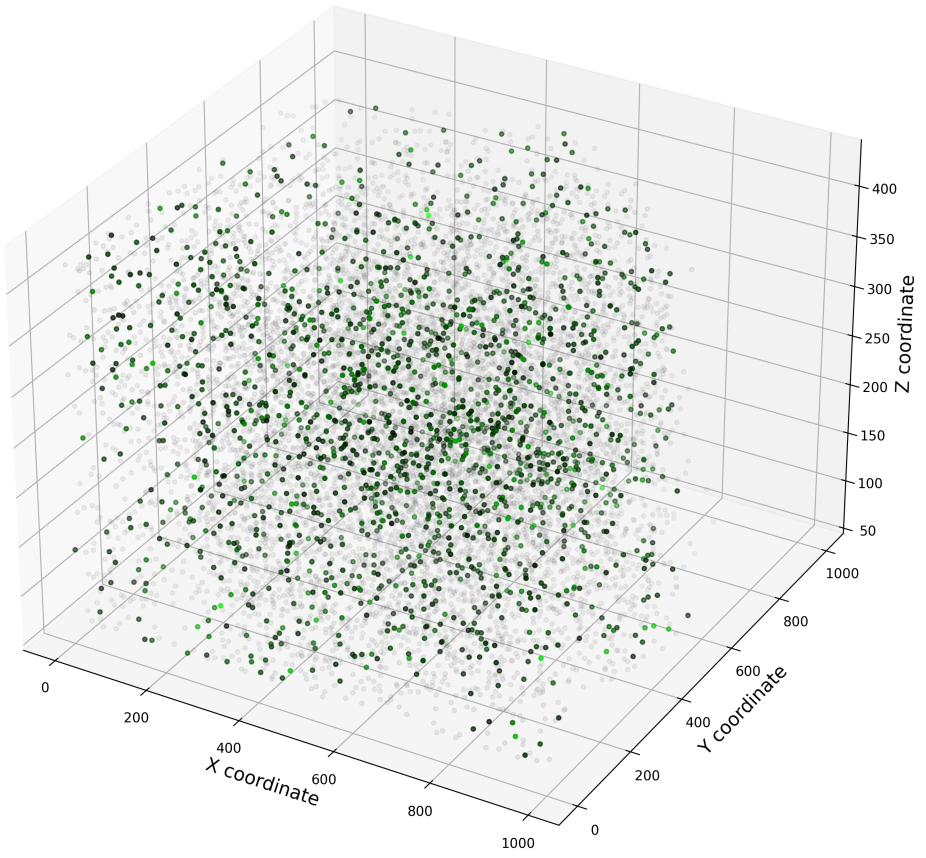


Figure A.21.: **Mouse neuron positions, colored by improvement.** Brighter green denotes stronger improvement. Non-significant neurons in gray.